

EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING



PROJECT CSW306 REPORT

E-Commerce Management System

Students

Trương Thị Vân – 2331220034
Nguyễn Thị Diễm My – 2331200069
Bùi Nhật Linh – 2331200158
Trương Châu Phước Lộc – 2331200066
Nguyễn Đình Duy Linh – 2331200198
Nguyễn Chí Đại – 2331200018

Supervisor

Nguyễn Xuân Cường

Ho Chi Minh, June, 2026

Abstract

To this day, managing sales, tracking orders, and controlling inventory manually or through disjointed tools often causes numerous issues for businesses, leading to inventory errors, lost orders, or poor customer service. Additionally, both store owners and staff spend too much time organizing information without a centralized platform.

For this reason, we designed an E-Commerce system—an online platform that helps businesses and individuals sell products over the internet while managing products, orders, customers, and revenue. This system provides a comprehensive tool for store owners to control the entire business operation, staff to process orders efficiently, and customers to shop online easily.

Technically, this project focuses on Backend architecture and RESTful APIs using ASP.NET Core and Entity Framework Core. The system ensures security and Role-based Authorization via JWT Authentication, while fully supporting CRUD operations to manage databases related to users, products, and orders.

This project was implemented to speed up and increase the reliability of the online shopping process. Although it is a basic system, it provides administrators, staff, and customers with an effective, secure environment to interact and significantly minimizes mistakes in daily business operations.

Acknowledgements

We would like to sincerely thank our supervisor, Lecturer Nguyen Xuan Cuong, for his kind guidance and support throughout this project. His advice helped us understand the problems better and improve our ideas step by step.

During the whole process, from the first research to finishing the E-Commerce backend system, he was always willing to answer our questions and guide us when we were stuck. Thanks to his support, we were able to complete this CSW306 project successfully and gain valuable experience in developing RESTful APIs and working with Entity Framework.

We truly appreciate the time and effort he spent helping us, and we are grateful for everything we have learned from him.

Contents

Abstract	i
Acknowledgements	ii
List of Tables	x
List of Figures	xiii
1 Introduction	1
1.1 Motivation	1
1.2 Objective	2
1.3 Project Overview	2
1.4 Scope of System	4
1.5 Actors and Roles	4
1.6 Technology Stack and Development Environment	5
2 System Design	6
2.1 System Workflow	6
2.1.1 General Use Case Diagram	6
2.1.2 Detailed Use Case Diagram	10
2.1.2.1 Authentication Use Case Diagram	10
2.1.2.2 Product Management Use Case Diagram	13
2.1.2.3 Shopping Cart Use Case Diagram	17
2.1.2.4 Order Management Use Case Diagram	20

2.1.2.5	Review Management Use Case Diagram	24
2.1.2.6	Voucher Management Use Case Diagram	27
2.1.2.7	Support Request Use Case Diagram	33
2.1.2.8	Administration & System Management Use Case Diagram	35
2.1.3	Activity Diagrams	36
2.1.3.1	User Registration Activity Diagram	36
2.1.3.2	User Login Activity Diagram	39
2.1.3.3	Role-based Access Control Flow	42
2.1.3.4	Product Management Activity Diagram	46
2.1.3.5	Product Variant Management Activity Diagram	50
2.1.3.6	Product Image Management Activity Diagram	53
2.1.3.7	Shopping Cart Activity Diagram	56
2.1.3.8	Voucher Validation Activity Diagram	59
2.1.3.9	Order Creation Activity Diagram	62
2.1.3.10	Payment Processing Activity Diagram	65
2.1.3.11	VNPay Activity Diagram	69
2.1.3.12	Order Cancellation Activity Diagram	73
2.1.3.13	Review Management Activity Diagram	76
2.1.3.14	Support Request Activity Diagram	79
2.1.3.15	Staff Order Processing Activity Diagram	82
2.2	Database	86
2.2.1	Users Table	86
2.2.2	Roles Table	88
2.2.3	Products Table	88
2.2.4	Product Variants Table	89

2.2.5	Product Images Table	90
2.2.6	Brands Table	91
2.2.7	Categories Table	92
2.2.8	Shopping Cart Table	93
2.2.9	Cart Items Table	93
2.2.10	Orders Table	94
2.2.11	Order Details Table	95
2.2.12	Vouchers Table	96
2.2.13	Reviews Table	97
2.2.14	Support Requests Table	98
2.2.15	Notifications Table	99
2.2.16	Payment Methods Table	100
2.2.17	Entity Relationships Overview (ERD)	102
3	Application Functions	104
3.1	Authentication & User Account	104
3.1.1	Login Screen	104
3.1.2	Register Screen	104
3.1.3	My Account Screen	105
3.1.4	Manage Shipping Addresses	105
3.1.5	Change Password Function	106
3.2	Product Catalog	107
3.2.1	Home Screen	107
3.2.2	Product Listing Screen	107
3.2.3	Product Detail Screen	108

3.3	Shopping Cart & Checkout	109
3.3.1	Shopping Cart Screen	109
3.3.2	Checkout Screen	110
3.3.3	Payment Selection	111
3.3.4	Payment Gate	112
3.3.5	Payment Success Notification	113
3.4	Order Management	114
3.4.1	My Orders Screen	114
3.4.2	Admin Order Management Screen	115
3.4.3	Staff Order Processing Screen	115
3.5	Customer Support	116
3.5.1	Support Request Screen	116
3.5.2	View Support Tickets Screen	116
3.5.3	Staff Support Management Screen	117
3.6	Administration	117
3.6.1	Admin Dashboard Screen	117
3.6.2	Product Management Screen	118
3.6.3	Customer Management Screen	118
3.6.4	Category Management Screen	119
3.6.5	Promotion Management Screen	119
3.6.6	Payment Method Management Screen	120
3.6.7	Staff Management Screen	120
3.7	Inventory Management	121
3.7.1	Inventory Management Screen	121
3.8	Reports & Analytics	121

3.8.1	Revenue Report Screen	121
3.8.2	Order Statistics Screen	122
3.8.3	Customer Statistics Screen	122
4	Related Work	124
4.1	Existing E-Commerce Platforms	126
4.1.1	Shopify and Shopify Plus	126
4.1.2	WooCommerce	127
4.1.3	BigCommerce	129
4.1.4	Adobe Commerce (Magento)	130
4.1.5	Salesforce Commerce Cloud (SFCC)	131
4.2	Feature Comparison	133
4.3	Competitive Advantages of the System	135
4.3.1	Uncompromising Reliability through Advanced Testing Frameworks	136
4.3.2	Absolute Economic Control and Sovereign Security	136
4.3.3	Next-Generation Behavioral Analytics via HT-Transformer Inte- gration	137
5	Conclusion and Future Works	139
5.1	Conclusion	139
5.2	Future Works	140
5.2.1	Remediation of Security Vulnerabilities and Identity Management	141
5.2.2	User Experience Optimization and Workflow Friction Reduction .	143
5.2.3	API Stability, Input Validation, and Data Integrity	145
5.2.4	Expanding Functional Completeness: Support Request Lifecycle .	146
5.2.5	Testing Infrastructure, Security Scanning, and Deployment Stan- dardization	148

5.2.6	Preparing for Advanced Analytics and Performance Scaling	149
A	Appendix	151
A.1	API Documentation	151

List of Tables

2.1	Structure of Users Table	87
2.2	Structure of Roles Table	88
2.3	Structure of Products Table	89
2.4	Structure of Product Variants Table	90
2.5	Structure of Product Images Table	90
2.6	Structure of Brands Table	92
2.7	Structure of Categories Table	92
2.8	Structure of Shopping Cart Table	93
2.9	Structure of Cart Items Table	94
2.10	Structure of Orders Table	95
2.11	Structure of Order Details Table	96
2.12	Structure of Vouchers Table	97
2.13	Structure of Reviews Table	98
2.14	Structure of Support Requests Table	99
2.15	Structure of Notifications Table	100
2.16	Structure of Payment Methods Table	102
4.1	High-Level Platform Architecture, Demographic Fit, and Implementation Velocity	133
4.2	Economic Models and Three-Year Total Cost of Ownership (TCO)	134
4.3	Technical Capabilities, Ecosystem Depth, and Omnichannel Readiness	134
5.1	Security Vulnerabilities and Proposed Remediations	143

5.2	API Stability Issue and Proposed Remediation	146
A.1	API Documentation	157

List of Figures

1.1	Ecommerce Problem Context	2
1.2	System Architecture Overview	3
2.1	General Use Case Diagram of the E-Commerce Management System . . .	7
2.2	Authentication Use Case Diagram	10
2.3	Product Management Use Case Diagram	13
2.4	Shopping Cart Use Case Diagram	17
2.5	Order Management Use Case Diagram	20
2.6	Review Management Use Case Diagram	24
2.7	Voucher Management Use Case Diagram	27
2.8	Support Request Use Case Diagram	33
2.9	Admin System Management Use Case Diagram	35
2.10	User Registration Activity Diagram	37
2.11	User Login Activity Diagram	40
2.12	Role Based Access Control Activity Diagram	43
2.13	Product Management Activity Diagram	47
2.14	Product Variant Management Activity Diagram	51
2.15	Product Image Management Activity Diagram	54
2.16	Shopping Cart Activity Diagram	57
2.17	Voucher Validation Activity Diagram	60
2.18	Order Creation Activity Diagram	63
2.19	Payment Processing Activity Diagram	66

2.20	VNPay Activity Diagram	70
2.21	Order Cancellation Activity Diagram	74
2.22	Review Management Activity Diagram	77
2.23	Support Request Activity Diagram	80
2.24	Staff Order Processing Activity Diagram	83
2.25	Entity Relationship Diagram	103
3.1	Login Screen	104
3.2	Register Screen	105
3.3	My Account Screen	105
3.4	Manage Shipping Addresses	106
3.5	Change Password Function	106
3.6	Home Screen	107
3.7	Product Listing Screen	108
3.8	Product Detail Screen	109
3.9	Shopping Cart Screen	110
3.10	Checkout Screen	111
3.11	Payment Selection	112
3.12	Payment Gate	113
3.13	Payment Success Notification	114
3.14	My Orders Screen	114
3.15	Admin Order Management Screen	115
3.16	Staff Order Processing Screen	115
3.17	Support Request Screen	116
3.18	View Support Tickets Screen	116

3.19 Staff Support Management Screen	117
3.20 Admin Dashboard Screen	117
3.21 Product Management Screen	118
3.22 Customer Management Screen	118
3.23 Category Management Screen	119
3.24 Promotion Management Screen	119
3.25 Payment Method Management Screen	120
3.26 Staff Management Screen	120
3.27 Inventory Management Screen	121
3.28 Revenue Report Screen	122
3.29 Order Statistics Screen	122
3.30 Customer Statistics Screen	123
4.1 The Shopify admin overview dashboard.	126
4.2 The WooCommerce product management interface integrated within the WordPress CMS.	128
4.3 The BigCommerce Theme Marketplace.	129
4.4 The Adobe Commerce backend interface.	130
4.5 The Salesforce Commerce Cloud Page Designer.	132

Chapter 1

Introduction

1.1 Motivation

The rapid growth of online shopping has fundamentally changed consumer behavior, with millions of transactions occurring daily across digital platforms. However, many small and medium-sized businesses still struggle to manage their operations efficiently due to fragmented tools, manual processes, and lack of an integrated digital storefront.

Traditional retail faces challenges such as limited geographic reach, high operational costs, and difficulty tracking inventory and customer data in real time. Meanwhile, customers expect seamless experiences — including easy product discovery, secure checkout, order tracking, and responsive support.

This project addresses these challenges by developing a full-featured E-Commerce Management System that unifies product management, customer accounts, order processing, payment handling, review systems, promotional vouchers, and customer support into a single web-based platform. The system aims to reduce operational friction for administrators while delivering a smooth and trustworthy shopping experience for end users..

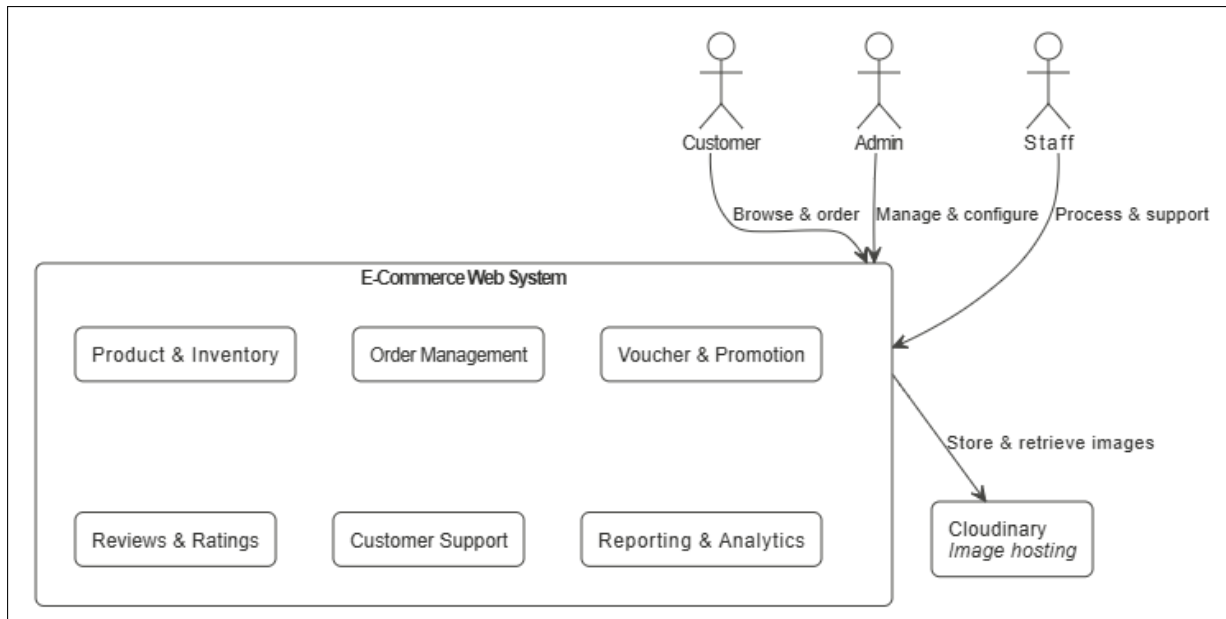


Figure 1.1: Ecommerce Problem Context

1.2 Objective

The main objective of this project is to design and implement a scalable, secure, and maintainable e-commerce management system for handling online retail operations.

Specific Objectives

- Provide product, brand, category, and variant management features.
- Support customer shopping carts, orders, payments, vouchers, and reviews.
- Implement authentication and authorization using JWT-based security.
- Provide API documentation and testing support through Swagger.

1.3 Project Overview

The E-Commerce Management System contains modules for user management, product catalog management, shopping cart management, order processing, voucher management, payment processing, review management, and customer support.

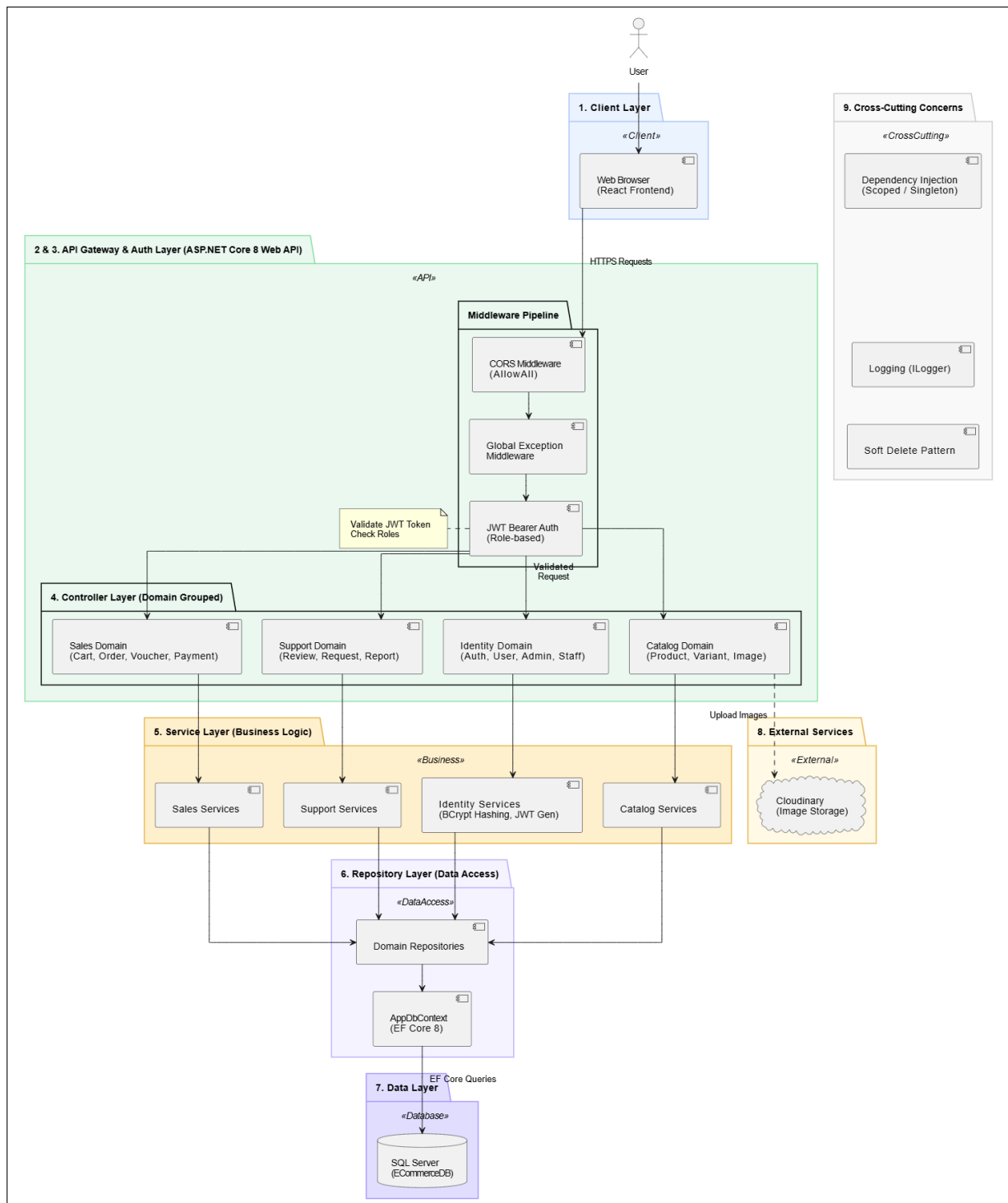


Figure 1.2: System Architecture Overview

1.4 Scope of System

The system scope includes customer registration, authentication, product browsing, cart operations, order management, payment workflow, voucher application, reviews, support requests, and administrative management features.

In Scope

- Customer-facing e-commerce features.
- Administrative product and order management.
- Database design and REST API workflow documentation.

Out of Scope

- Native mobile applications.
- Third-party logistics integration.
- Advanced recommendation engine.

1.5 Actors and Roles

The E-Commerce Management System supports several actors who interact with the platform according to their responsibilities and access permissions.

- **Customer:** Browses products, manages a cart, places orders, makes payments, tracks orders, writes reviews, and submits support requests.
- **Administrator:** Manages products, categories, orders, customers, staff accounts, promotions, inventory, reports, and system settings.
- **Staff:** Supports daily operations such as order processing, customer support, product updates, and inventory monitoring.
- **Payment Gateway:** Processes online payment transactions and returns payment results to the system.
- **System:** Handles authentication, authorization, notifications, logging, and automated business rule validation.

1.6 Technology Stack and Development Environment

To ensure a robust, scalable, and secure architecture, the system is built using a modern full-stack approach. The codebase is strictly organized into separate directories for the frontend and backend environments to prevent path conflicts, streamline the development workflow, and ensure maintainability.

Core Technologies

- **Frontend Ecosystem:** The client-side application is developed using React 18 and bundled with Vite for optimal rendering performance and rapid hot-module replacement. The user interface is heavily styled using a combination of Tailwind CSS v4 for utility-first styling and Ant Design for robust, pre-built enterprise components.
- **Backend Architecture:** The server-side operations are powered by ASP.NET Core 8.0 Web API. The backend utilizes a layered architectural pattern consisting of Controllers, Services, and Repositories to separate business logic from routing. Entity Framework Core 8.0 serves as the Object-Relational Mapper (ORM), handling data querying and database migrations.
- **Database:** SQL Server is used as the primary relational database. It is deployed locally via LocalDB during development and containerized for testing.
- **External Integrations:** The system integrates Cloudinary through Cloudinary-DotNet for optimized cloud-based image hosting and delivery. Financial transactions are processed securely through the VNPay Sandbox, using HMAC-SHA512 cryptographic signatures.

Deployment and Infrastructure

The application environment is fully containerized. Developers utilize `docker-compose` to orchestrate the backend API, frontend client, and SQL Server instances simultaneously. Database initialization and restoration are managed via SQL Server Management Studio (SSMS), using `.bak` backup files mapped directly into Linux-based Docker containers.

Chapter 2

System Design

2.1 System Workflow

2.1.1 General Use Case Diagram

The general use case diagram summarizes the main actors and high-level functions of the E-Commerce Management System.

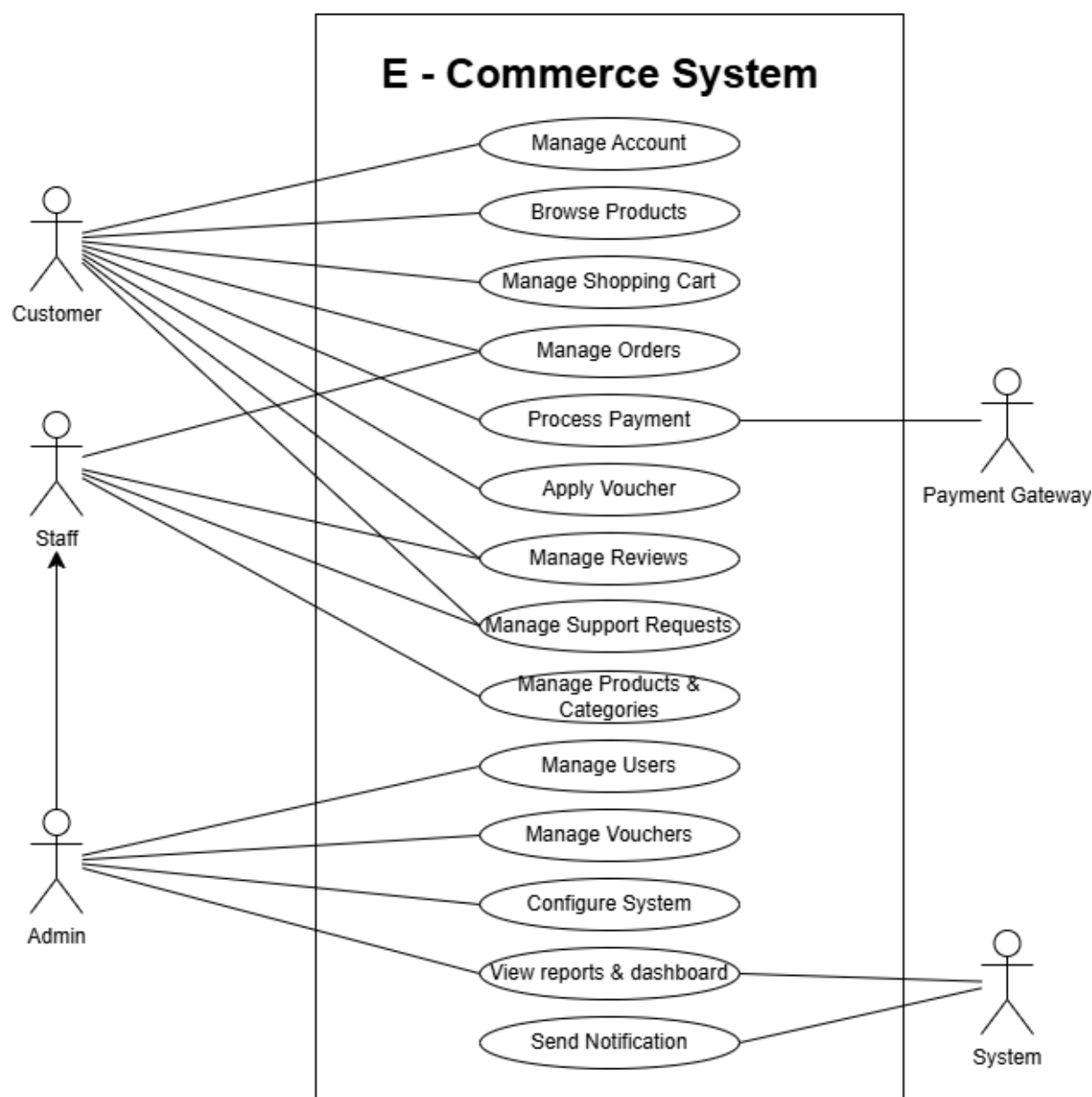


Figure 2.1: General Use Case Diagram of the E-Commerce Management System

User (Customer): The primary actor of the system who uses the application to browse products, place orders, and manage their shopping experience.

- Manage account: register, login, update profile, and change password.
- Browse products: search by keyword, filter by category or price, and view product details.
- Manage shopping cart: add or remove products, update quantities, and view total amount.

- Manage orders: place orders, track order status, cancel orders, and request refunds.
- Process payment: select payment method and complete transaction via Payment Gateway.
- Apply voucher: enter voucher code and apply discount to order total.
- Manage reviews: write reviews, rate products, and edit or delete own reviews.
- Manage support requests: submit support tickets, track ticket status, and reply to staff responses.

User (Staff): The operational actor responsible for day-to-day management of products, orders, and customer interactions.

- Manage products & categories: add, edit, or remove products and manage category structure.
- Manage orders: confirm orders, update delivery status, and handle cancellation and return requests.
- Manage reviews: approve or remove reviews that violate content policy.
- Manage support requests: receive and respond to customer tickets, update ticket status, and escalate complex issues to Admin.

User (Admin): The administrator responsible for maintaining and supervising the entire system, inheriting all Staff permissions with additional elevated access.

- Manage users: view, create, and deactivate Customer and Staff accounts, assign roles and permissions.
- Manage products & categories: full control over product catalog and category hierarchy.
- Manage orders: oversee all orders across the system, resolve disputes, and generate order reports.
- Manage vouchers: create and configure voucher campaigns, set conditions and expiry, and monitor usage statistics.
- Manage reviews: moderate all reviews platform-wide and remove policy-violating content.
- View reports & dashboard: monitor revenue, sales performance, user activity, and product analytics.

- **Configure system:** adjust platform settings, configure payment gateways, set shipping policies, and manage notification templates.

System: The core backend of the application that processes data and handles communication between all actors.

- Stores and updates order data, payment records, product inventory, and user information.
- Provides interfaces for Customers, Staff, and Admin to interact with the platform.
- Executes automated functions such as payment confirmation, order status updates, low-stock alerts, and scheduled report generation.
- Sends notifications to Customers and Staff triggered by system events, such as order confirmation, ticket replies, and voucher expiry reminders.

Payment Gateway: Represents external payment services (Cash, Momo, VNPay, credit card) used by Customers to complete transactions through the system.

- Receives payment requests initiated by Customers during the checkout process.
- Processes and verifies transactions securely.
- Returns transaction results, such as success or failure, to the system for order status updates.

Relationship Between Roles and Functionalities:

- Customer focuses on the end-to-end shopping journey: browsing, carting, ordering, paying, reviewing, and seeking support.
- Staff handles daily operational tasks including product updates, order processing, review moderation, and customer support resolution.
- Admin oversees the entire platform with full system access: user governance, voucher management, analytics, and system configuration. Admin inherits all Staff capabilities via generalization.
- System provides backend processes and handles data flow between all actors to ensure smooth and automated operation of the platform.
- Payment Gateway acts as a trusted external intermediary that securely bridges the Customer's payment action and the system's order confirmation logic.

2.1.2 Detailed Use Case Diagram

This subsection provides detailed use case diagrams for the major functional areas of the E-Commerce Management System. Each diagram focuses on the relationship between actors and system functions.

2.1.2.1 Authentication Use Case Diagram

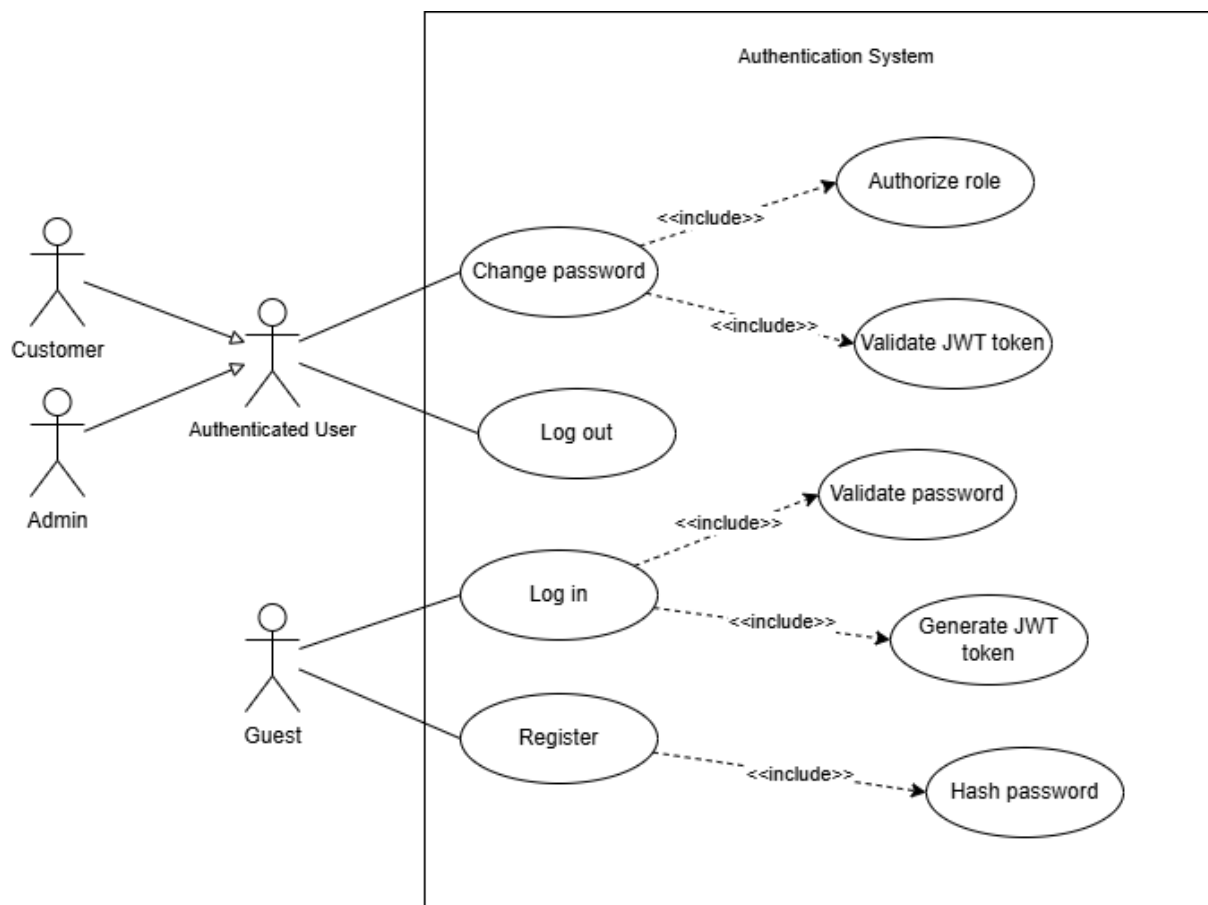


Figure 2.2: Authentication Use Case Diagram

Use Case 1: Register

- **Actor:** Guest.
- **Description:** Allows a new user to create an account in the system.
- **Pre-conditions:** The guest is not logged in and does not already have an active account using the submitted email address.

- **Basic Flow:**

1. The guest enters registration information, submits the form, and the system validates the input.
2. The system hashes the password using BCrypt before saving the account record.

- **Exceptions:** If required information is missing, the email is already registered, or validation fails, the system rejects the request and returns an error message.

- **Post-conditions:** A new user account is created and stored securely in the database.

Use Case 2: Log in

- **Actor:** Guest.

- **Description:** Authenticates a registered user and grants access to protected system functions.

- **Pre-conditions:** The user has an existing active account and is not currently authenticated.

- **Basic Flow:**

1. The guest submits email and password credentials.
2. The system verifies the password against the stored hash and generates a signed JWT containing user claims such as ID, Email, and Role.

- **Exceptions:** If the email does not exist, the password is incorrect, or the account is inactive, the system rejects the login request and does not generate a token.

- **Post-conditions:** The user receives a valid JWT and is recognized as an authenticated user.

Use Case 3: Log out

- **Actor:** Authenticated User.

- **Description:** Terminates the current authenticated session on the client side.

- **Pre-conditions:** The user is logged in with an active JWT-based session.

- **Basic Flow:** The user selects the logout function, and the client removes the stored authentication token and redirects the user to the guest state.

- **Exceptions:** If the token is already expired or missing, the system still clears local session data and treats the user as logged out.
- **Post-conditions:** The user session is cleared, and protected functions are no longer accessible without logging in again.

Use Case 4: Change Password

- **Actor:** Authenticated User.
- **Description:** Allows a logged-in user to securely update the account password.
- **Pre-conditions:** The user provides a valid JWT and the current password.
- **Basic Flow:** The system validates the JWT, authorizes the user role, verifies the current password, hashes the new password, and saves the updated password hash.
- **Exceptions:**
 - If the JWT is expired, invalid, or unauthorized, the system returns a 401 Unauthorized or 403 Forbidden response.
 - If the current password is incorrect or the new password violates policy, the update is rejected.
- **Post-conditions:** The password is updated and stored as a new secure hash in the database.

2.1.2.2 Product Management Use Case Diagram

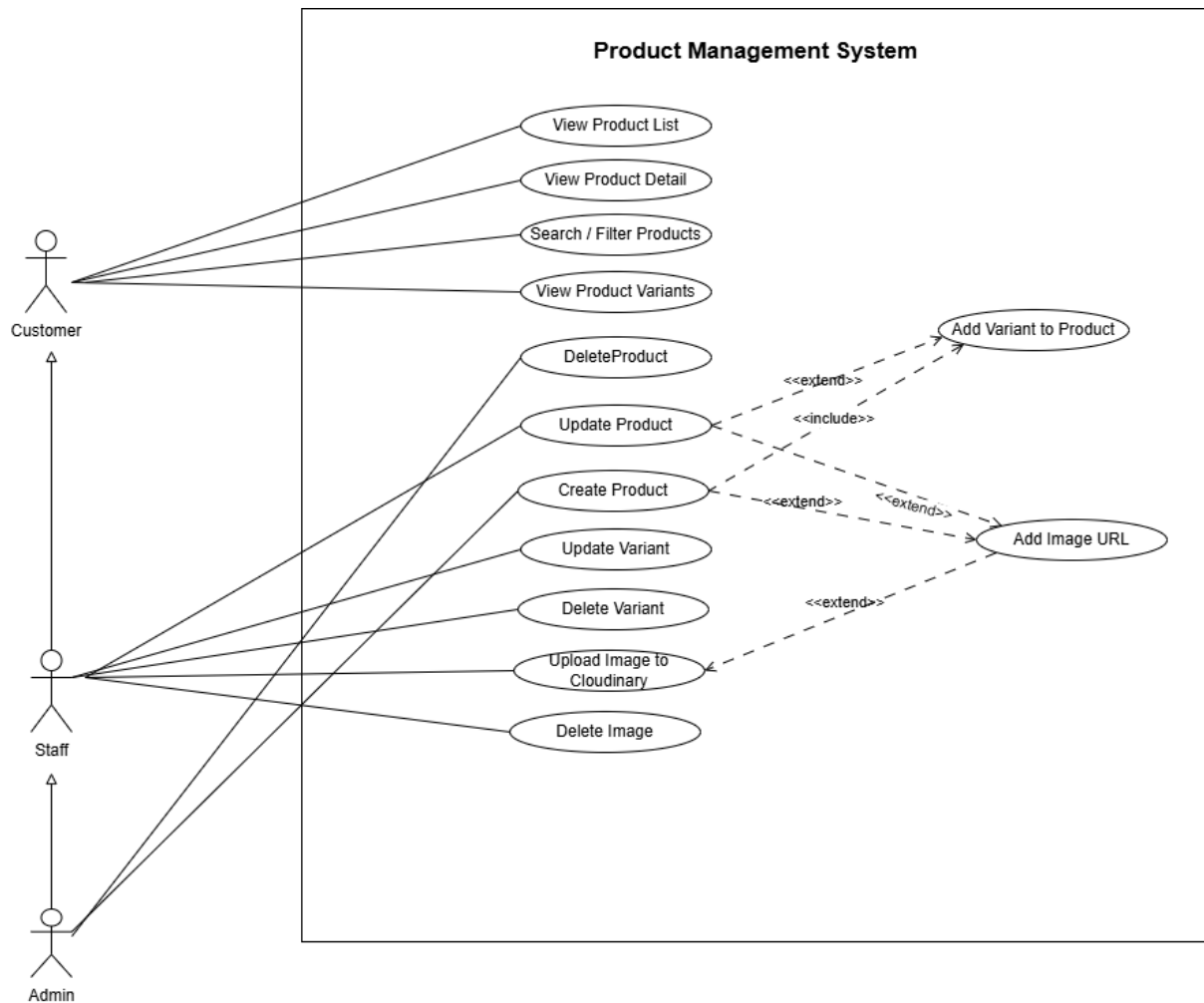


Figure 2.3: Product Management Use Case Diagram

Use Case 1: View Product List

- **Actor:** Customer.
- **Description:** Allows customers to browse available products in the catalog.
- **Pre-conditions:** Products exist in the system and are not marked as deleted.
- **Basic Flow:** The customer opens the product listing page, and the system displays paginated products with name, price, rating, and thumbnail image.
- **Exceptions:** If no products match the current listing criteria, the system displays an empty result message.

- **Post-conditions:** The customer can review the visible product list and select a product for more details.

Use Case 2: View Product Detail

- **Actor:** Customer.
- **Description:** Allows customers to inspect full information for a selected product.
- **Pre-conditions:** The selected product exists and is visible to customers.
- **Basic Flow:** The customer selects a product, and the system displays its description, average rating, brand, category, images, and available variants (*«include» View Product Variants*).
- **Exceptions:** If the product does not exist or has been soft-deleted, the system returns a not-found response.
- **Post-conditions:** The product detail page is displayed with current product and variant information.

Use Case 3: Search / Filter Products

- **Actor:** Customer.
- **Description:** Allows customers to locate products using search and filter criteria.
- **Pre-conditions:** The product catalog is available for browsing.
- **Basic Flow:**
 1. The customer enters keywords or selects filters such as category, brand, and price range.
 2. The system applies filtering and returns paginated results.
- **Exceptions:** If no products match the criteria, the system displays an empty result set without changing catalog data.
- **Post-conditions:** Filtered product results are shown to the customer.

Use Case 4: Create Product

- **Actor:** Staff, Admin.
- **Description:** Allows authorized personnel to create a new product in the system.

- **Pre-conditions:** The actor is authenticated and has product management permission. Required category and brand records exist.
- **Basic Flow:**
 1. The actor enters product name, description, category, brand, and variant information.
 2. The system validates the request, creates the product, and can add variants (*«extend» Add Variant to Product*) and image URLs (*«extend» Add Image URL*).
- **Exceptions:** If required fields are missing, category or brand is invalid, variant data is invalid, or database persistence fails, the system rejects the request and displays an error.
- **Post-conditions:** A new product record is created and becomes available according to catalog visibility rules.

Use Case 5: Update Product

- **Actor:** Staff, Admin.
- **Description:** Allows authorized personnel to modify an existing product.
- **Pre-conditions:** The actor is authenticated, authorized, and the target product exists.
- **Basic Flow:**
 1. The actor edits product information, submits changes, and the system validates and saves the update.
 2. During the update, the actor may add new variants (*«include» Add Variant to Product*) and attach additional image URLs (*«extend» Add Image URL*).
- **Exceptions:** If the product is not found, validation fails, or the update conflicts with business rules, the system cancels the operation and returns an error.
- **Post-conditions:** The product record is updated while relational integrity is preserved.

Use Case 6: Delete Product

- **Actor:** Staff, Admin.
- **Description:** Allows authorized personnel to remove a product from the public catalog using soft delete.

- **Pre-conditions:** The actor is authenticated, authorized, and the product exists.
- **Basic Flow:** The actor selects a product for deletion, and the system sets its IsDeleted flag to true.
- **Exceptions:** If the product does not exist or cannot be deleted because of business constraints, the system returns an error.
- **Post-conditions:** The product is hidden from customer listings while historical data remains preserved.

Use Case 7: Manage Product Variants

- **Actor:** Staff, Admin.
- **Description:** Allows authorized personnel to add, update, or delete product variants.
- **Pre-conditions:** The actor is authenticated and the target product exists.
- **Basic Flow:** The actor selects a product and manages variant data such as name, price, and quantity (*«include» Add Variant to Product*).
- **Exceptions:** If variant values are invalid, stock quantity is negative, or the product is not found, the system rejects the operation.
- **Post-conditions:** Product variant records are updated and stock information remains consistent.

Use Case 8: Upload Product Image

- **Actor:** Staff, Admin.
- **Description:** Allows authorized personnel to upload product images to Cloudinary and link them to products.
- **Pre-conditions:** The actor is authenticated, the product exists, and an image file is selected.
- **Basic Flow:** The actor uploads an image file, the system sends it to Cloudinary under ecommerce/products, receives the secure URL, and links it to the product (*«extend» Add Image URL*).
- **Exceptions:** If no file is provided, Cloudinary upload fails, or image persistence fails, the system returns an upload or system error.

- **Post-conditions:** The uploaded image URL is stored and associated with the product.

Use Case 9: Manage Categories and Brands

- **Actor:** Admin.
- **Description:** Allows administrators to create and manage product categories and brands.
- **Pre-conditions:** The admin is authenticated and has administrative privileges.
- **Basic Flow:** The admin creates, updates, or deletes category and brand records used by products.
- **Exceptions:** If a category or brand is duplicated, missing required data, or referenced by restricted records, the system rejects the operation.
- **Post-conditions:** Category and brand data is maintained for product classification.

2.1.2.3 Shopping Cart Use Case Diagram

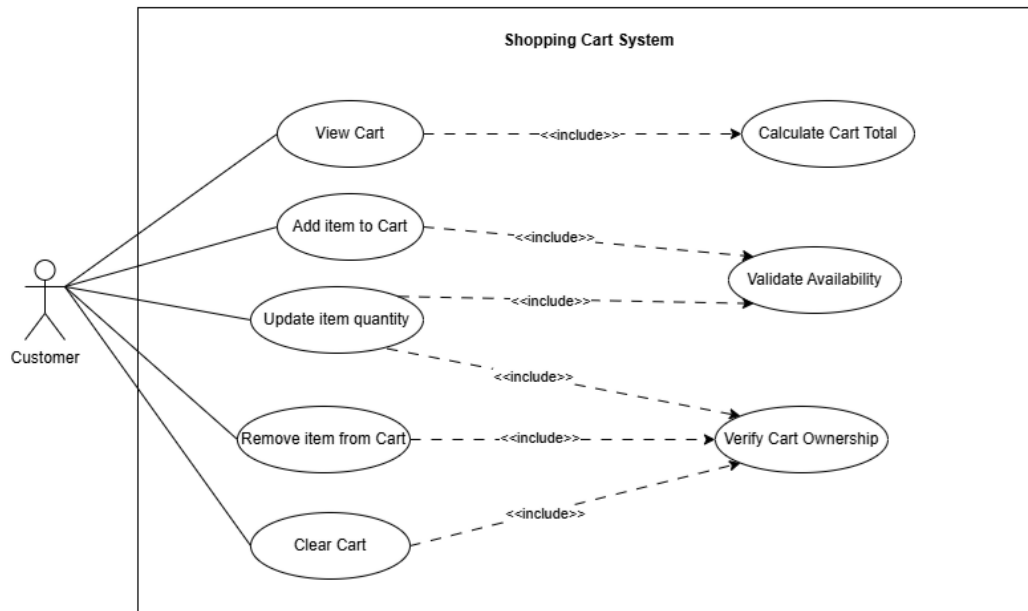


Figure 2.4: Shopping Cart Use Case Diagram

Use Case 1: View Cart

- **Actor:** Customer.
- **Description:** Allows customers to review items currently stored in their shopping cart.
- **Pre-conditions:** The customer is authenticated and has an active cart.
- **Basic Flow:** The customer opens the cart page, and the system displays selected products, variants, quantities, prices, and subtotal after calculating the cart total (*«include» Calculate Cart Total*).
- **Exceptions:** If the cart is empty, the system displays an empty cart message.
- **Post-conditions:** The customer can proceed to update quantities, remove items, or checkout.

Use Case 2: Add Item to Cart

- **Actor:** Customer.
- **Description:** Allows customers to add a selected product variant to the cart.
- **Pre-conditions:** The customer is authenticated and the selected variant exists with available stock.
- **Basic Flow:** The customer selects a product variant and quantity, and the system validates item availability (*«include» Validate Availability*) before adding the item.
- **Exceptions:** If the variant is unavailable, deleted, or stock is insufficient, the system rejects the request.
- **Post-conditions:** The cart contains the selected item with the requested quantity.

Use Case 3: Update Cart Item

- **Actor:** Customer.
- **Description:** Allows customers to change the quantity of an item in the cart.
- **Pre-conditions:** The customer is authenticated and the cart item exists.
- **Basic Flow:** The customer updates the quantity, and the system validates item availability (*«include» Validate Availability*) and verifies cart ownership (*«include» Verify Cart Ownership*) before saving the change.

- **Exceptions:** If the requested quantity exceeds stock, is invalid, or the cart item does not belong to the customer, the system returns an error.
- **Post-conditions:** The cart item quantity and subtotal are updated.

Use Case 4: Remove Cart Item

- **Actor:** Customer.
- **Description:** Allows customers to remove an item from the cart.
- **Pre-conditions:** The customer is authenticated and the selected cart item exists.
- **Basic Flow:** The customer selects a cart item for removal, and the system verifies cart ownership (*«include» Verify Cart Ownership*) before deleting it from the cart.
- **Exceptions:** If the cart item cannot be found or does not belong to the customer, the system returns a not-found or unauthorized response.
- **Post-conditions:** The item is removed and cart totals are recalculated.

Use Case 5: Clear Cart

- **Actor:** Customer.
- **Description:** Allows customers to remove all items from their shopping cart at once.
- **Pre-conditions:** The customer is authenticated and has an active cart containing one or more items.
- **Basic Flow:** The customer selects the clear cart function, and the system verifies cart ownership (*«include» Verify Cart Ownership*) before deleting all cart items.
- **Exceptions:** If the cart is empty, cannot be found, or does not belong to the customer, the system rejects the request or displays an appropriate message.
- **Post-conditions:** The cart contains no items and the cart total is reset.

2.1.2.4 Order Management Use Case Diagram



Figure 2.5: Order Management Use Case Diagram

Use Case 1: Create Order

- **Actor:** Customer.
- **Description:** Allows customers to place an order from selected cart items.
- **Pre-conditions:** The customer is authenticated, has selected cart items, and has a valid shipping address.
- **Basic Flow:** The system retrieves selected cart items (*«include» View Cart*), verifies inventory (*«include» Check Stock Availability*), calculates subtotal, discounts, shipping fee, and final total (*«include» Calculate Order Total*) (*«include» Calculate Shipping Fee*), applies voucher discounts when provided (*«extend» Apply Voucher Discount*), creates the order, decreases inventory (*«include» Decrease Inventory*), and clears purchased cart items (*«include» Clear Cart*).
- **Exceptions:** If stock is insufficient, the address is invalid, voucher validation fails, payment information is invalid, or persistence fails, the system cancels order creation and returns an error.
- **Post-conditions:** A new order is stored, inventory is reduced, and purchased items are removed from the cart.

Use Case 2: View My Orders

- **Actor:** Customer.
- **Description:** Allows customers to view their order history.
- **Pre-conditions:** The customer is authenticated.
- **Basic Flow:** The customer opens the order history page, and the system retrieves order summaries with optional filtering (*«extend» Filter Orders*).
- **Exceptions:** If no orders exist, the system displays an empty order history message.
- **Post-conditions:** The customer can review previous and current orders.

Use Case 3: View Order Detail

- **Actor:** Customer, Admin.
- **Description:** Allows authorized users to inspect complete information for a selected order.

- **Pre-conditions:** The actor is authenticated and has permission to access the selected order.
- **Basic Flow:** The actor selects an order, and the system displays order items (*«include» View Order Items*), shipping address (*«include» View Shipping Address*), voucher information (*«extend» View Voucher Information*), and customer information for administrators (*«extend» View Customer Information*).
- **Exceptions:** If the order does not exist or the actor lacks permission, the system returns a not-found or forbidden response.
- **Post-conditions:** The order detail information is displayed without modifying order data.

Use Case 4: Cancel Order

- **Actor:** Customer.
- **Description:** Allows customers to cancel an order when cancellation rules are satisfied.
- **Pre-conditions:** The customer is authenticated, owns the order, and the order is in a cancellable status.
- **Basic Flow:** The customer requests cancellation, and the system validates the cancellation condition (*«include» Validate Cancel Condition*), restores inventory (*«include» Restore Inventory*), and rolls back voucher usage when applicable (*«extend» Rollback Voucher Usage*).
- **Exceptions:** If the order is already shipped, completed, cancelled, or not owned by the customer, the system rejects the cancellation.
- **Post-conditions:** The order status is changed to cancelled, inventory is restored, and voucher usage is corrected when necessary.

Use Case 5: View Order List

- **Actor:** Admin, Staff.
- **Description:** Allows internal users to monitor customer orders.
- **Pre-conditions:** The actor is authenticated and authorized for order management.
- **Basic Flow:** The actor opens the order management page, and the system displays orders with filtering (*«extend» Filter Orders*) and searching (*«extend» Search Orders*) options.

- **Exceptions:** If no orders match the criteria, the system displays an empty result message.
- **Post-conditions:** The actor can review and select orders for operational processing.

Use Case 6: Update Order Status

- **Actor:** Admin, Staff.
- **Description:** Allows authorized users to manage the order processing lifecycle.
- **Pre-conditions:** The actor is authenticated, authorized, and the target order exists.
- **Basic Flow:** The actor selects a new order status, and the system validates the status transition (*«include» Validate Status Transition*) before updating the order lifecycle (*«include» Update Order Lifecycle*).
- **Exceptions:** If the status transition is invalid, the order is not found, or the actor lacks permission, the system rejects the update.
- **Post-conditions:** The order status is updated according to valid business rules.

Use Case 7: Generate Order Report

- **Actor:** Admin.
- **Description:** Allows administrators to generate order-related business reports.
- **Pre-conditions:** The admin is authenticated and has reporting privileges.
- **Basic Flow:** The admin selects report criteria, and the system generates revenue reports (*«include» Revenue Report*), order statistics (*«include» Order Statistics*), and top customer reports (*«include» Top Customers Report*).
- **Exceptions:** If the selected period has no data or report generation fails, the system displays a report error or empty report message.
- **Post-conditions:** The requested report is generated for administrative analysis.

2.1.2.5 Review Management Use Case Diagram

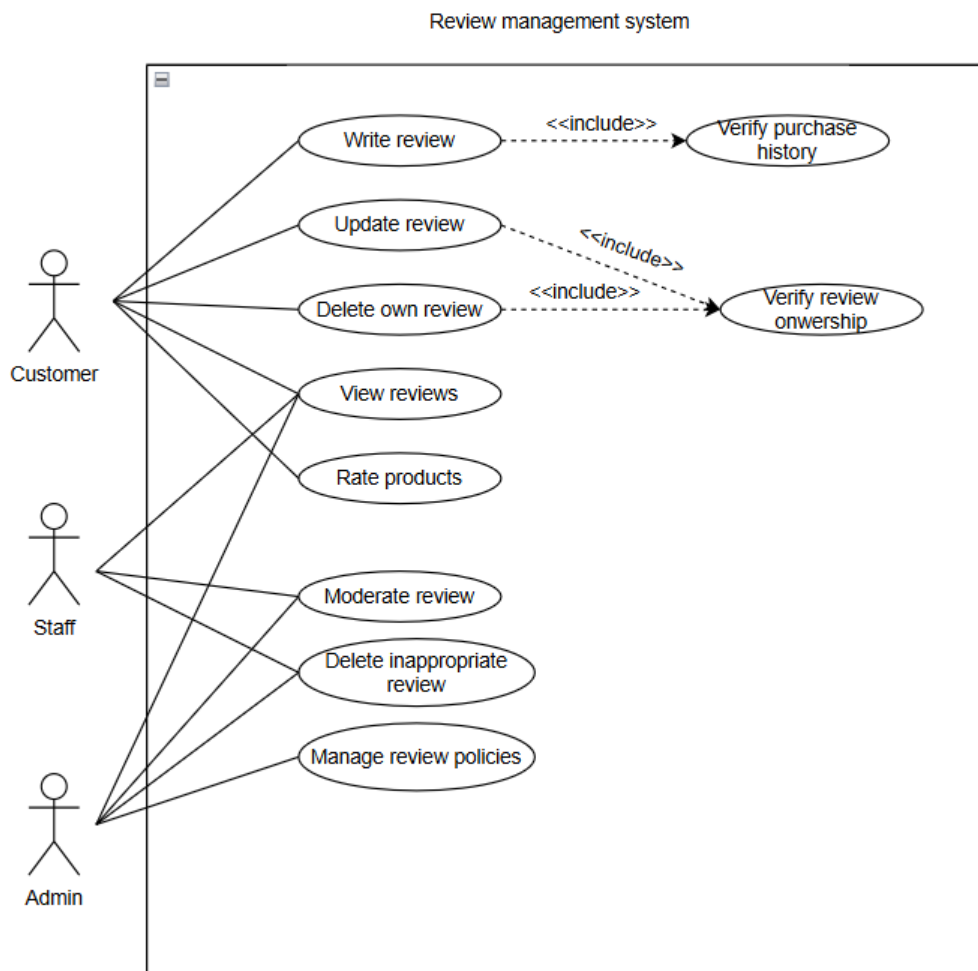


Figure 2.6: Review Management Use Case Diagram

Use Case 1: Write Review

- **Actor:** Customer.
- **Description:** Allows customers to submit ratings and comments for purchased products.
- **Pre-conditions:** The customer is authenticated and has purchased the product.
- **Basic Flow:** The customer enters a rating and comment, and the system validates and saves the review while updating product average rating (*«include» Rating Aggregation*).

- **Exceptions:** If the customer has not purchased the product, submits invalid content, or already reviewed the product where duplicates are not allowed, the system rejects the review.
- **Post-conditions:** The review is stored and product rating information is updated.

Use Case 2: Edit Review

- **Actor:** Customer.
- **Description:** Allows customers to update their own reviews.
- **Pre-conditions:** The customer is authenticated and owns the review.
- **Basic Flow:** The customer edits rating or comment content, and the system validates and saves the changes while recalculating average rating (*«include» Rating Aggregation*).
- **Exceptions:** If the review does not exist, belongs to another user, or contains invalid content, the system rejects the update.
- **Post-conditions:** The review and related product rating are updated.

Use Case 3: Delete Review

- **Actor:** Customer, Admin.
- **Description:** Allows authorized users to remove reviews.
- **Pre-conditions:** The actor is authenticated and has ownership or moderation permission.
- **Basic Flow:** The actor selects a review, and the system deletes or hides it while updating rating aggregation (*«include» Rating Aggregation*).
- **Exceptions:** If the review is not found or the actor lacks permission, the system rejects the action.
- **Post-conditions:** The review is removed from visible review listings.

Use Case 4: View Reviews

- **Actor:** Customer, Guest.
- **Description:** Allows users to read product reviews before making purchasing decisions.

- **Pre-conditions:** The product exists and has visible review data.
- **Basic Flow:** The actor opens the product review section, and the system displays ratings and comments.
- **Exceptions:** If no reviews exist, the system displays an empty review message.
- **Post-conditions:** Review information is displayed without modifying system data.

Use Case 5: Moderate Reviews

- **Actor:** Staff, Admin.
- **Description:** Allows internal users to monitor and remove inappropriate reviews.
- **Pre-conditions:** The actor is authenticated and has review moderation permission.
- **Basic Flow:** The actor reviews submitted comments and removes inappropriate content when necessary (*«extend» Remove Inappropriate Reviews*).
- **Exceptions:** If the review is not found or the actor lacks permission, the system returns an error.
- **Post-conditions:** Review content remains compliant with platform policies.

2.1.2.6 Voucher Management Use Case Diagram

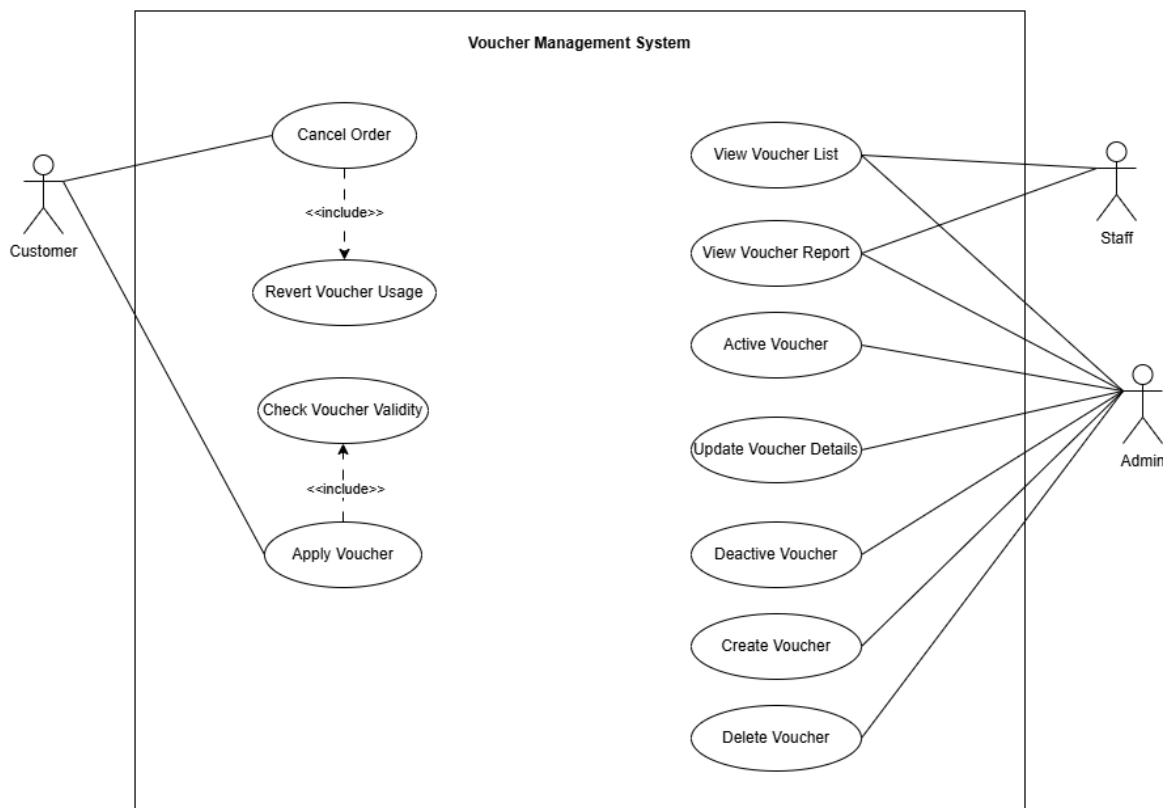


Figure 2.7: Voucher Management Use Case Diagram

Use Case 1: Apply Voucher

- **Actor:** Customer.
- **Description:** Allows customers to apply a voucher code during checkout to reduce the final order amount.
- **Pre-conditions:** The customer is authenticated, has a valid checkout cart, and enters a voucher code.
- **Basic Flow:**
 1. The customer enters a voucher code on the checkout page.
 2. The system checks voucher validity using the included *Check Voucher Validity* use case.
 3. If the voucher is valid, the system calculates the discount and updates the checkout total.

- **Relationship:** This use case includes *Check Voucher Validity*.
- **Exceptions:** If the voucher is invalid, expired, inactive, already used, or does not satisfy order conditions, the system rejects the voucher and keeps the original order total.
- **Post-conditions:** The valid voucher discount is reflected in the checkout summary.

Use Case 2: Check Voucher Validity

- **Actor:** System.
- **Description:** Verifies whether a voucher can be applied to the current checkout request.
- **Pre-conditions:** A voucher code and checkout information are available for validation.
- **Basic Flow:**
 1. The system searches for the voucher by code.
 2. The system checks active status, validity period, usage limit, minimum order value, and customer eligibility.
 3. The system returns the validation result to the voucher application flow.
- **Exceptions:** If any validation rule fails, the system returns a voucher validation error.
- **Post-conditions:** The voucher is either accepted for discount calculation or rejected with a reason.

Use Case 3: Cancel Order

- **Actor:** Customer.
- **Description:** Allows customers to cancel an eligible order that may have used a voucher.
- **Pre-conditions:** The customer is authenticated, owns the order, and the order status allows cancellation.
- **Basic Flow:**
 1. The customer submits an order cancellation request.
 2. The system verifies cancellation eligibility.

3. If the order used a voucher, the system reverts voucher usage through the included *Revert Voucher Usage* use case.

- **Relationship:** This use case includes *Revert Voucher Usage*.
- **Exceptions:** If the order is not found, does not belong to the customer, or cannot be canceled due to its status, the system rejects the request.
- **Post-conditions:** The order is canceled and related voucher usage is restored when applicable.

Use Case 4: Revert Voucher Usage

- **Actor:** System.
- **Description:** Restores voucher usage information when an order using a voucher is canceled.
- **Pre-conditions:** The canceled order contains voucher information and the voucher usage record exists.
- **Basic Flow:** The system identifies the voucher used by the canceled order, decreases or restores the usage count according to business rules, and updates voucher availability.
- **Exceptions:** If the voucher record no longer exists or usage information is inconsistent, the system logs the issue for administrative review.
- **Post-conditions:** Voucher usage data is corrected after order cancellation.

Use Case 5: View Voucher List

- **Actor:** Staff, Admin.
- **Description:** Allows internal users to view available vouchers for monitoring and management.
- **Pre-conditions:** The actor is authenticated and has voucher viewing permission.
- **Basic Flow:** The actor opens the voucher management page, and the system displays voucher records with status, discount value, validity period, and usage information.
- **Exceptions:** If no vouchers exist, the system displays an empty list message.
- **Post-conditions:** Voucher information is displayed without modifying data.

Use Case 6: View Voucher Report

- **Actor:** Staff, Admin.
- **Description:** Allows internal users to review voucher usage and effectiveness reports.
- **Pre-conditions:** The actor is authenticated and has permission to access voucher reports.
- **Basic Flow:** The actor requests a voucher report, and the system summarizes usage count, discount amount, active vouchers, inactive vouchers, and related order statistics.
- **Exceptions:** If report data cannot be loaded, the system returns an error message.
- **Post-conditions:** Voucher performance information is available for business analysis.

Use Case 7: Create Voucher

- **Actor:** Admin.
- **Description:** Allows administrators to create a new promotional voucher.
- **Pre-conditions:** The admin is authenticated and has voucher management permission.
- **Basic Flow:**
 1. The admin enters voucher code, discount value, validity period, usage limit, and conditions.
 2. The system validates the voucher information and saves the new voucher.
- **Exceptions:** If the voucher code is duplicated, required fields are missing, date ranges are invalid, or discount rules are inconsistent, the system rejects the request.
- **Post-conditions:** A new voucher is stored and becomes available according to its active status and validity period.

Use Case 8: Update Voucher Details

- **Actor:** Admin.
- **Description:** Allows administrators to modify voucher details such as discount value, conditions, validity period, and usage limits.

- **Pre-conditions:** The admin is authenticated, authorized, and the voucher exists.
- **Basic Flow:** The admin edits voucher information, and the system validates and saves the updated details.
- **Exceptions:** If the voucher is not found, update data is invalid, or the change conflicts with business rules, the system rejects the update.
- **Post-conditions:** Voucher details are updated for future validation and checkout usage.

Use Case 9: Active Voucher

- **Actor:** Admin.
- **Description:** Allows administrators to activate a voucher so that it can be used during checkout.
- **Pre-conditions:** The admin is authenticated, and the voucher exists in an inactive or unpublished state.
- **Basic Flow:** The admin selects the activation action, and the system marks the voucher as active if its configuration is valid.
- **Exceptions:** If the voucher is expired, missing required conditions, or cannot be activated, the system rejects the action.
- **Post-conditions:** The voucher becomes available for eligible checkout requests.

Use Case 10: Deactive Voucher

- **Actor:** Admin.
- **Description:** Allows administrators to deactivate a voucher without deleting its historical data.
- **Pre-conditions:** The admin is authenticated, and the voucher exists in an active state.
- **Basic Flow:** The admin selects the deactivation action, and the system marks the voucher as inactive.
- **Exceptions:** If the voucher is not found or the actor lacks permission, the system rejects the action.
- **Post-conditions:** The voucher is no longer available for new checkout requests, while historical order data remains unchanged.

Use Case 11: Delete Voucher

- **Actor:** Admin.
- **Description:** Allows administrators to remove a voucher record when deletion is permitted by business rules.
- **Pre-conditions:** The admin is authenticated, authorized, and the voucher exists.
- **Basic Flow:** The admin selects a voucher for deletion, and the system removes it or performs a safe deletion strategy according to order history constraints.
- **Exceptions:** If the voucher is not found or is referenced by historical orders, the system rejects physical deletion and recommends deactivation instead.
- **Post-conditions:** The voucher is removed from management lists or remains inactive if historical references must be preserved.

2.1.2.7 Support Request Use Case Diagram

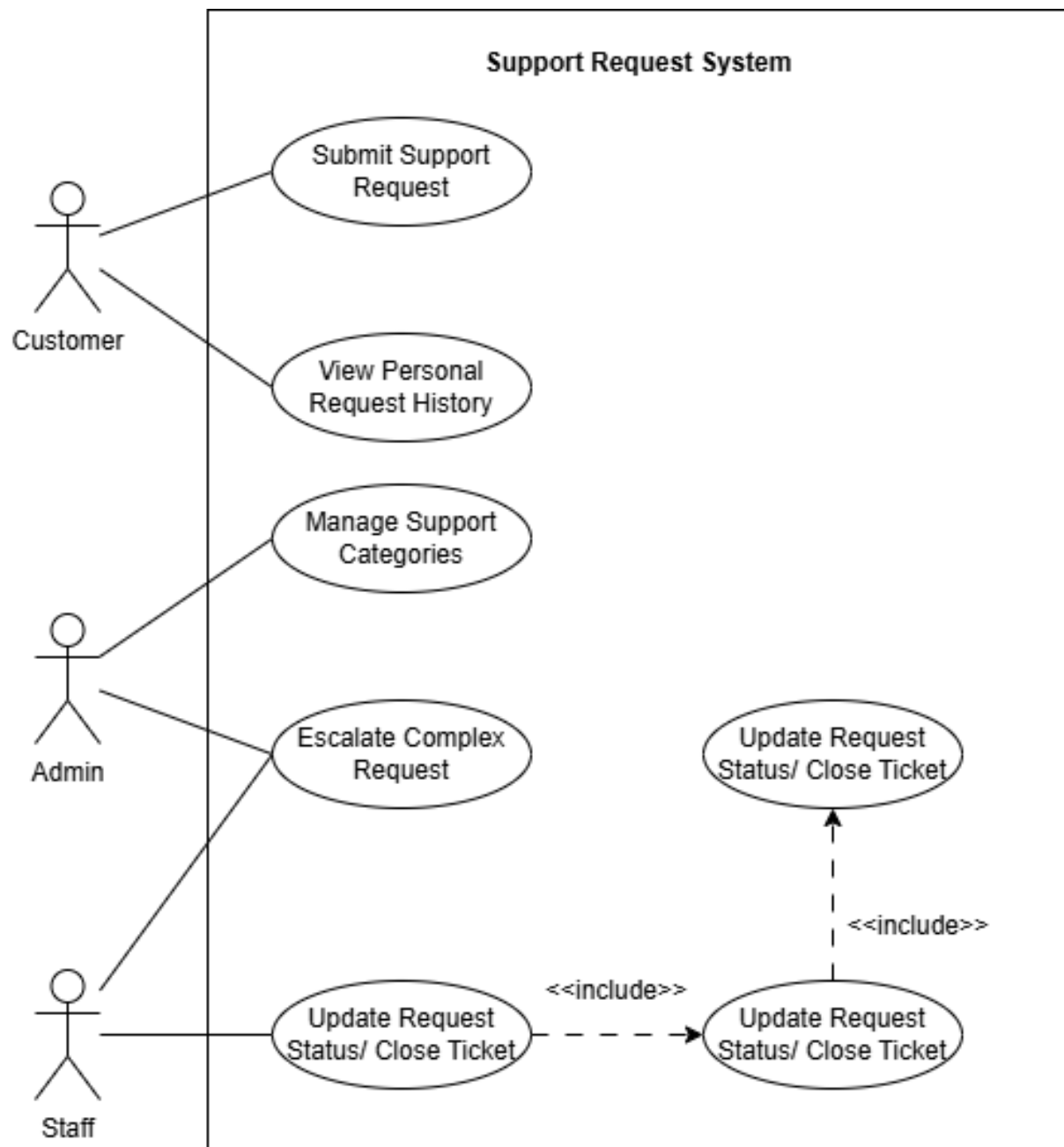


Figure 2.8: Support Request Use Case Diagram

Use Case 1: Submit Support Request

- **Actor:** Customer.
- **Description:** Allows customers to send support questions or issue reports to the system.

- **Pre-conditions:** The customer is authenticated and provides required support request information.
- **Basic Flow:**
 1. The customer enters request subject, description, and optional order reference.
 2. The system validates and stores the request.
- **Exceptions:** If required fields are missing or the referenced order is invalid, the system rejects the request.
- **Post-conditions:** A new support request is created and awaits staff handling.

Use Case 2: Process Support Request

- **Actor:** Staff, Admin.
- **Description:** Allows internal users to review and respond to customer support requests.
- **Pre-conditions:** The actor is authenticated and has support management permission.
- **Basic Flow:** The actor opens the support request list, reviews a request, updates its status, and provides a response.
- **Exceptions:** If the request is not found or the status transition is invalid, the system rejects the update.
- **Post-conditions:** The support request status and response information are updated.

2.1.2.8 Administration & System Management Use Case Diagram

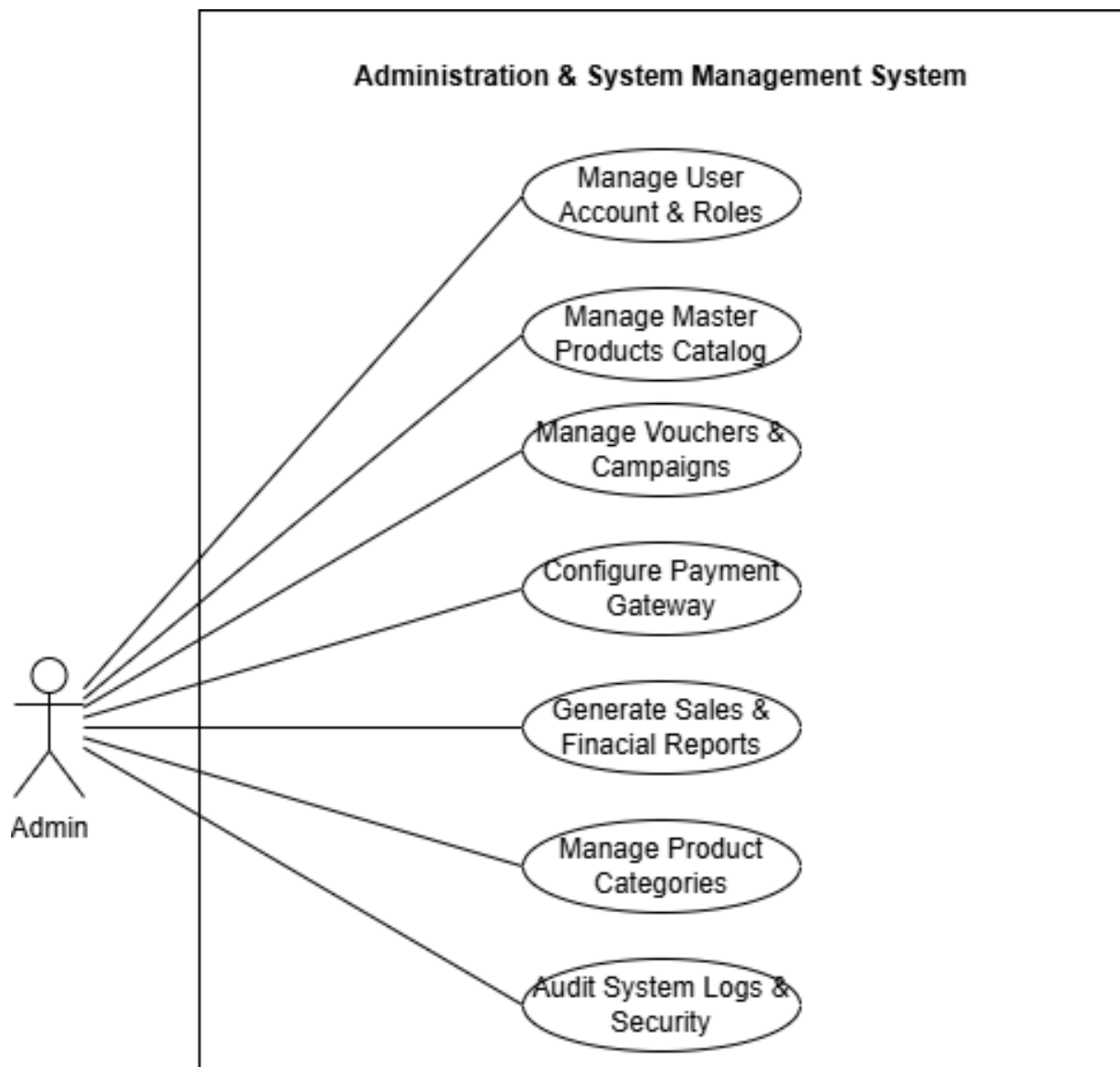


Figure 2.9: Admin System Management Use Case Diagram

Use Case 1: Manage Users

- **Actor:** Admin.
- **Description:** Allows administrators to view, update, activate, or deactivate user accounts.
- **Pre-conditions:** The admin is authenticated and has system management permission.

- **Basic Flow:** The admin searches for a user, reviews account information, and updates status or role as needed.
- **Exceptions:** If the user is not found, the role assignment is invalid, or the admin attempts a restricted operation, the system rejects the change.
- **Post-conditions:** User account information is updated according to administrative rules.

Use Case 2: Manage System Configuration

- **Actor:** Admin.
- **Description:** Allows administrators to maintain system-level settings required for platform operation.
- **Pre-conditions:** The admin is authenticated and authorized for system configuration.
- **Basic Flow:** The admin opens configuration management, updates permitted settings, and the system validates and saves the changes.
- **Exceptions:** If a setting value is invalid or persistence fails, the system rejects the update and keeps the previous configuration.
- **Post-conditions:** Valid system configuration changes are stored and applied according to system rules.

2.1.3 Activity Diagrams

2.1.3.1 User Registration Activity Diagram

This activity diagram describes the main control flow, decision points, and system responses for user registration activity diagram in the E-Commerce Management System.

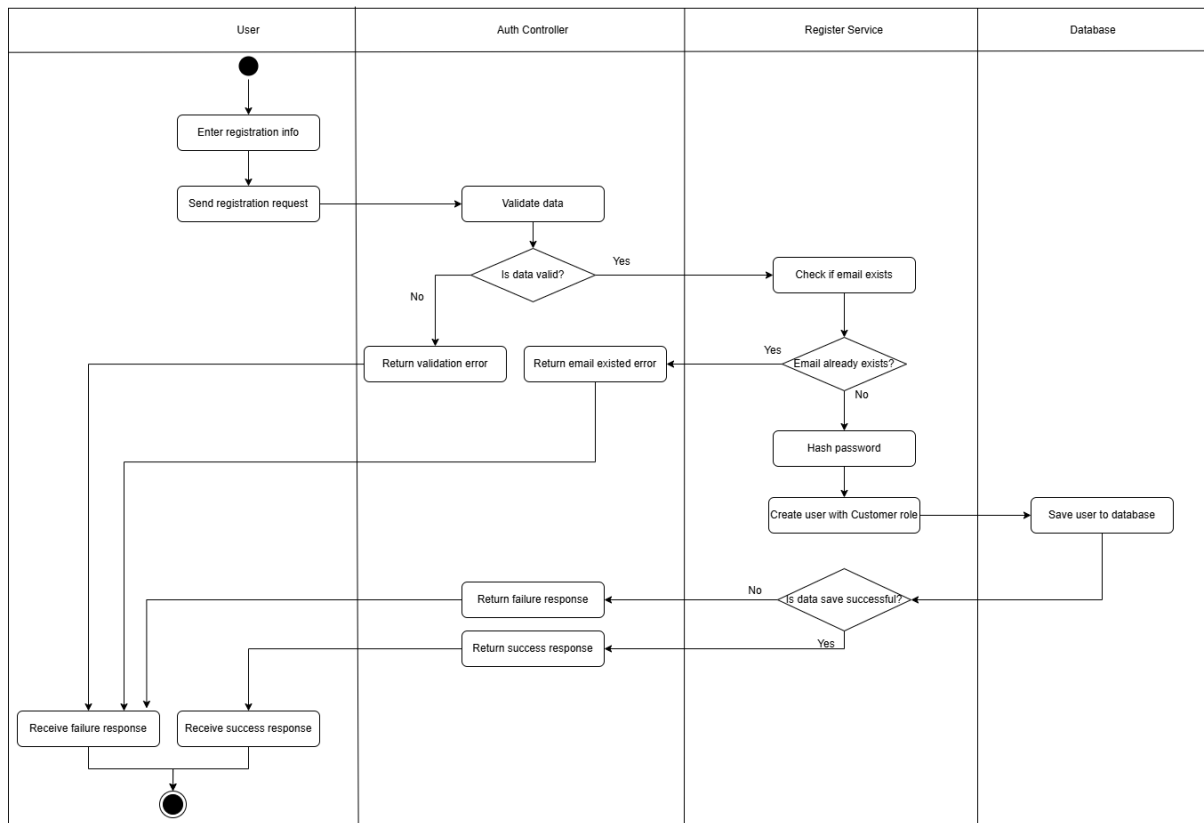


Figure 2.10: User Registration Activity Diagram

Workflow Description

1. Enter Registration Information

The process starts in the User swimlane when the user enters registration information such as username, email, and password. After completing the form, the user sends a registration request to the system.

2. Validate Registration Data

The request is received by the Auth Controller, where the system validates the input data.

The validation process checks:

- Required fields
- Email format
- Password constraints
- Invalid or missing information

Invalid Data Flow

If the input data is invalid:

- The system returns a validation error response.
- The user receives the failure response.
- The process terminates.

This step helps ensure data integrity and prevents invalid information from entering the system.

3. Check Existing Email

If the validation process is successful, the request is forwarded to the Register Service.

The Register Service checks whether the submitted email already exists in the system.

Email Already Exists Flow

If the email already exists:

- The service returns an email existed error.
- The controller sends a failure response back to the user.
- The process ends.

This prevents duplicate accounts from being created using the same email address.

4. Hash Password

If the email does not exist, the system hashes the user password before storing it.

Password hashing is an important security mechanism because:

- Plain text passwords are never stored in the database.
- User credentials are protected even if the database is compromised.

5. Create User Account

After hashing the password, the Register Service creates a new user account with the default Customer role.

At this stage, the user object is prepared with all necessary information before being saved into the database.

6. Save User Data to Database

The system sends the user data to the Database layer for persistence.

The database attempts to save the new account information.

Save Failure Flow

If the save operation fails:

- The system returns a failure response.
- The user receives the failure message.
- The process terminates.

Save Success Flow

If the save operation succeeds:

- The system returns a success response.
- The user receives confirmation that the registration was completed successfully.
- The process ends.

2.1.3.2 User Login Activity Diagram

This activity diagram describes the main control flow, decision points, and system responses for user login activity diagram in the E-Commerce Management System.

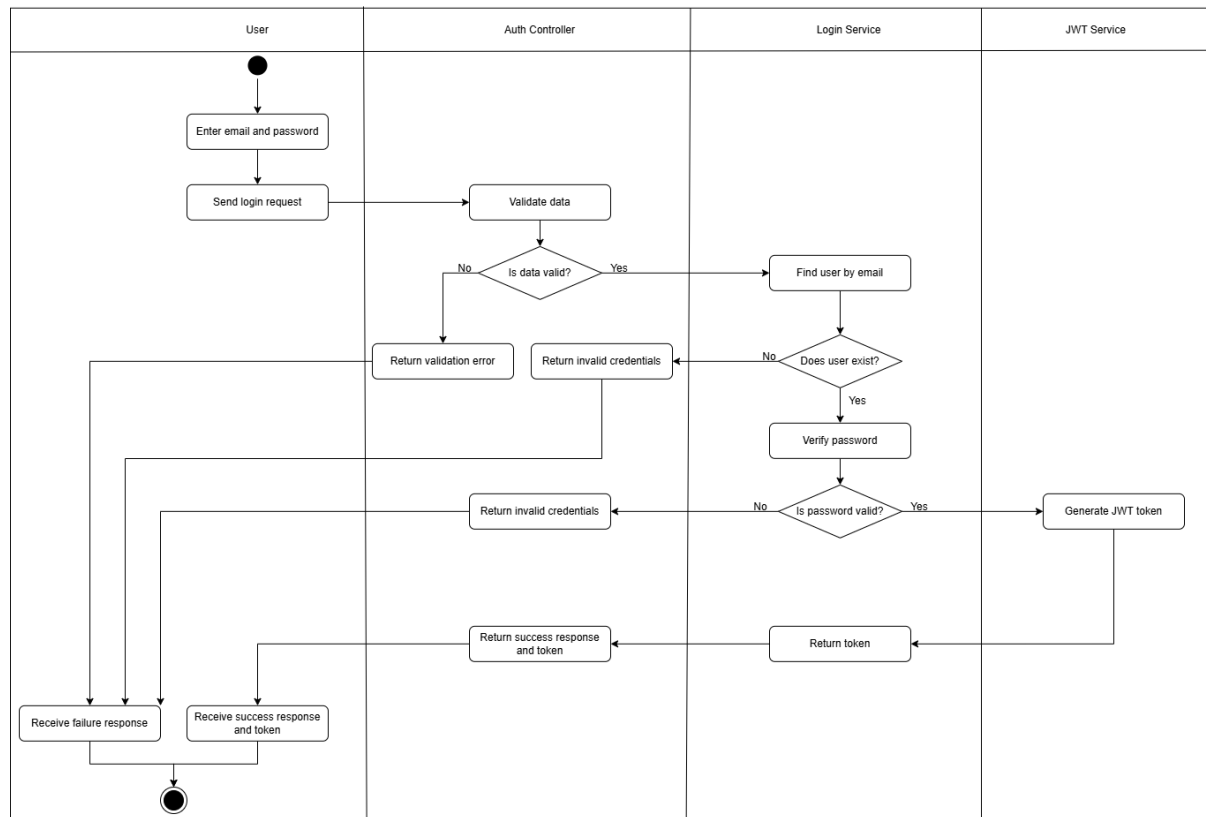


Figure 2.11: User Login Activity Diagram

Workflow Description

1. Enter Login Credentials

The process starts in the User swimlane when the user enters an email address and password into the login form.

After entering the required information, the user sends a login request to the system.

2. Validate Login Data

The login request is received by the Auth Controller, where the system validates the submitted data.

The validation process checks:

- Required input fields
- Correct email format
- Empty or invalid values

Invalid Data Flow

If the submitted data is invalid:

- The system returns a validation error response.
- The user receives a failure response.
- The process terminates.

This validation step ensures that invalid requests are rejected before authentication processing begins.

3. Find User by Email

If the input data is valid, the request is forwarded to the Login Service.

The Login Service searches for a user account using the provided email address.

User Not Found Flow

If no user account matches the email:

- The system returns a failure response.
- The user receives the failure message.
- The login process ends.

This step prevents unauthorized access using non-existing accounts.

4. Verify Password

If the user account exists, the Login Service verifies the entered password against the encrypted password stored in the system.

Password verification is performed securely using hashing comparison techniques rather than comparing plain text passwords.

Incorrect Password Flow

If the password is incorrect:

- The system returns an invalid password response.
- The user receives a failure response.

- The process terminates.

This mechanism ensures secure authentication and protects user accounts from unauthorized access.

5. Generate JWT Token

If the password is verified successfully, the request is sent to the JWT Service.

The JWT Service generates a JSON Web Token (JWT) containing authenticated user information and authorization data.

The generated token is used for:

- Stateless authentication
- Secure API access
- User session management
- Role-based authorization

6. Return Success Response

After the JWT token is generated:

- The system returns a success response along with the token.
- The user receives the authentication token successfully.
- The login process ends.

The token can later be attached to protected API requests to verify the user's identity and permissions.

2.1.3.3 Role-based Access Control Flow

This activity diagram describes the main control flow, decision points, and system responses for role-based access control flow in the E-Commerce Management System.

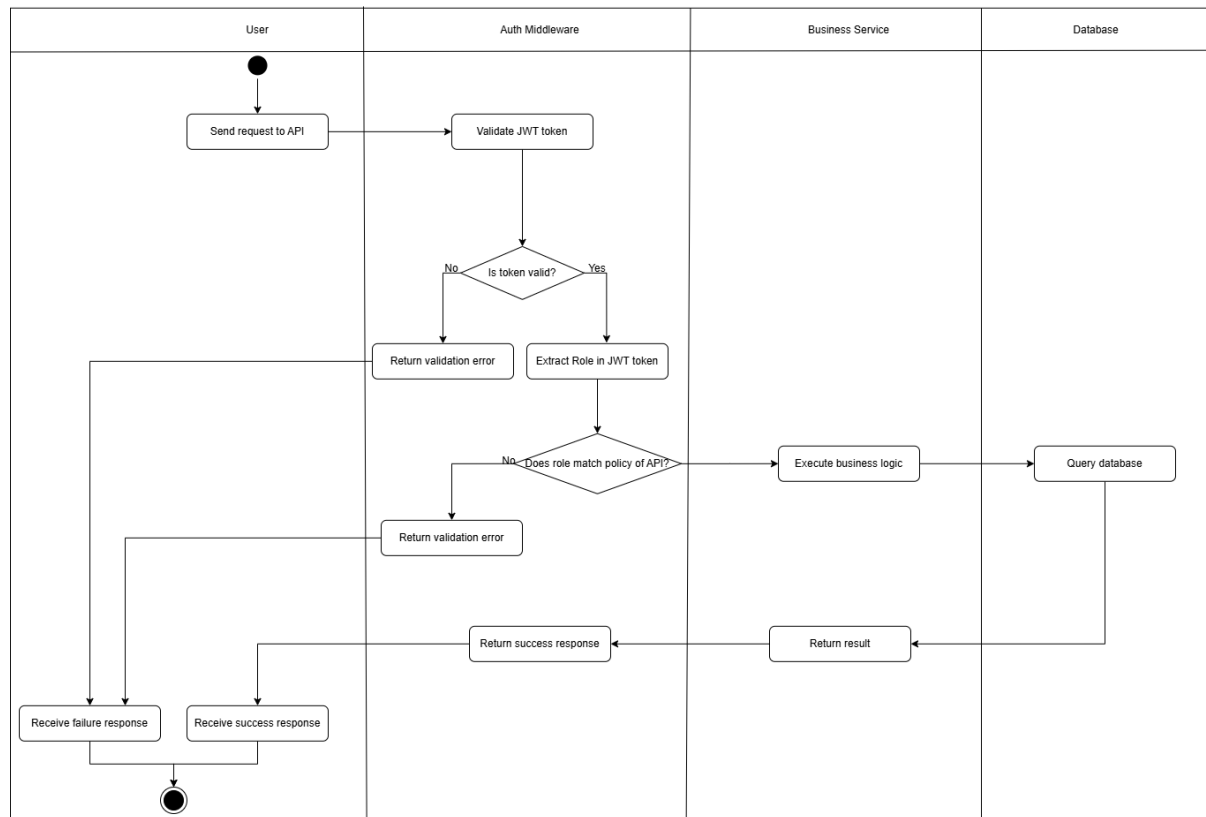


Figure 2.12: Role Based Access Control Activity Diagram

Workflow Description

1. Send Request to Protected API

The process begins when a User sends a request to a protected API endpoint.

The request contains a JWT token, typically included in the Authorization header using the Bearer Token scheme.

The request is then intercepted by the Auth Middleware before reaching the business layer.

2. Validate JWT Token

The Auth Middleware validates the received JWT token.

The validation process includes:

- Verifying the token signature
- Checking token expiration

- Confirming token integrity
- Ensuring the token has not been tampered with

Invalid Token Flow

If the token is invalid:

- The middleware returns a validation error response.
- The user receives a failure response.
- The process terminates.

This prevents unauthorized or expired tokens from accessing protected resources.

3. Extract User Role

If the JWT token is valid, the middleware extracts the user's role information from the token claims.

Examples of roles may include:

- Customer
- Staff
- Manager
- Administrator

The extracted role is then compared against the authorization policy defined for the requested API endpoint.

4. Verify Role Authorization

The Auth Middleware checks whether the user's role satisfies the required permissions.

Authorized Flow

If the user's role matches the API access policy:

- The request is forwarded to the Business Service.
- The requested business operation is executed.

- Necessary data is retrieved from the database.
- The result is returned to the middleware.
- A success response is sent back to the user.

This ensures that only users with appropriate privileges can perform protected actions.

Unauthorized Flow

If the user's role does not satisfy the authorization policy:

- The middleware returns an authorization error response.
- The user receives a failure response.
- The process terminates.

This prevents users from accessing resources beyond their assigned permissions.

5. Execute Business Logic

For authorized users, the request reaches the Business Service layer.

The Business Service performs the requested operation, such as:

- Viewing protected information
- Creating new records
- Updating existing data
- Deleting resources

If required, the service interacts with the database to retrieve or modify data.

6. Query Database

The Business Service sends database requests to perform the necessary data operations.

The Database processes the query and returns the requested data or operation result.

The Business Service then prepares the final response.

7. Return Response

After successful execution of the business logic:

- The Business Service returns the result.
- The Auth Middleware forwards a success response.
- The User receives the requested data or confirmation message.
- The process ends.

XSS Prevention

The system applies basic Cross-Site Scripting (XSS) prevention using a layered approach. On the frontend, React automatically escapes JSX content before rendering API data to the DOM, and the project does not use `dangerouslySetInnerHTML`. On the backend, `SecurityHeadersMiddleware.cs` adds security headers to API responses, including content type protection, frame blocking, referrer policy, and Content Security Policy. Since the backend returns JSON instead of server-rendered HTML, the current architecture does not directly inject user input into HTML responses.

Security Considerations

This workflow implements a secure authorization mechanism through JWT-based authentication and role-based access control.

Key security benefits include:

- Preventing unauthorized access to protected APIs
- Restricting functionality based on user roles
- Supporting stateless authentication using JWT
- Reducing database lookups by storing role information inside token claims
- Improving scalability and performance for distributed systems

2.1.3.4 Product Management Activity Diagram

This activity diagram describes the main control flow, decision points, and system responses for product management activity diagram in the E-Commerce Management System.

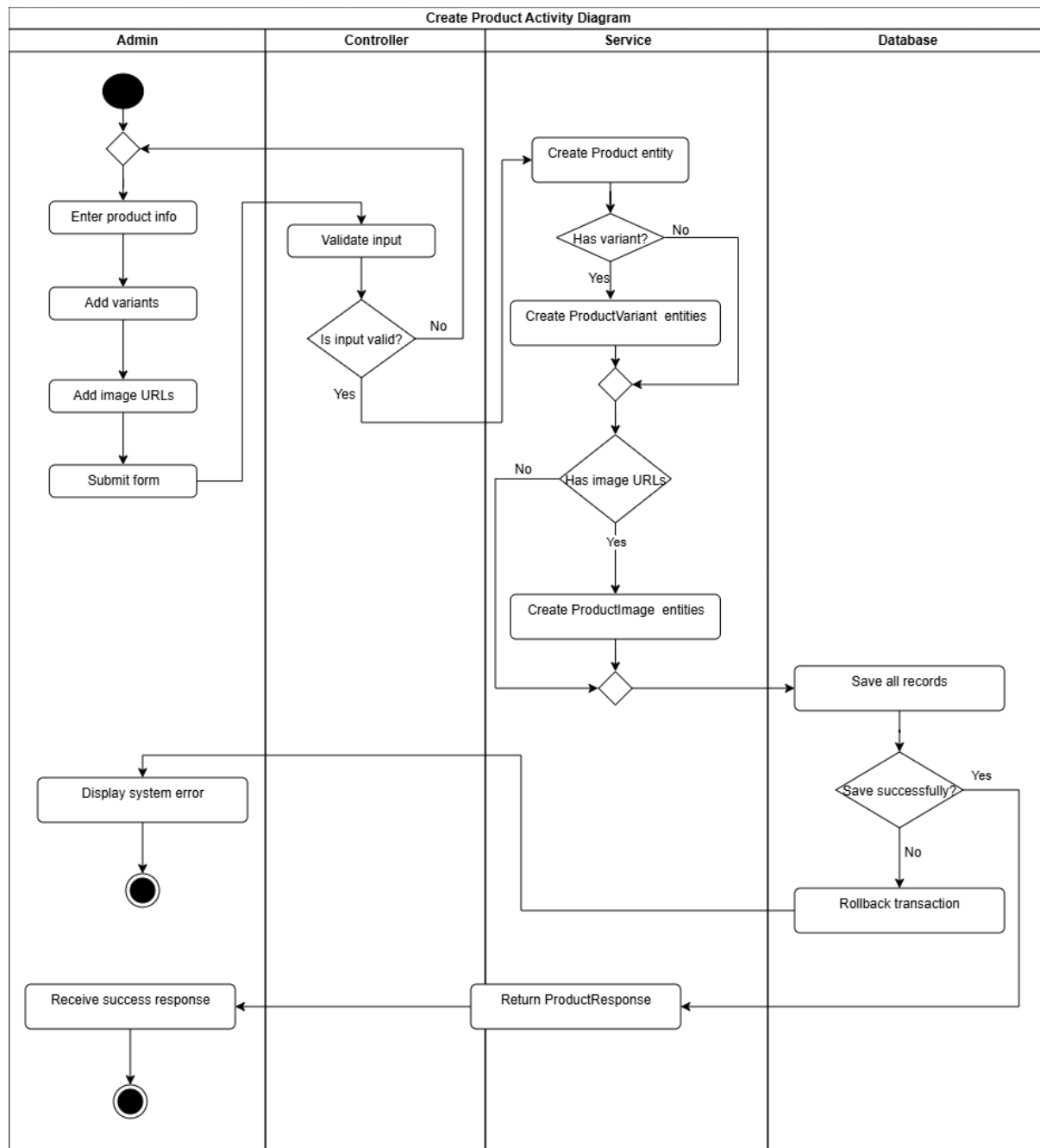


Figure 2.13: Product Management Activity Diagram

Workflow Description

1. Enter Product Information

The process begins in the Admin swimlane. The administrator fills in the required product details, including the product name, description, category, and brand. The admin then optionally adds one or more product variants, including name, price, and quantity, and image URLs before submitting the form to the system.

2. Validate Input

The submitted request is received by the Controller, which performs input validation before forwarding the data to the Service layer.

The validation process checks:

- Required fields, including name, category, and brand.
- Data type and length constraints.
- At least one valid variant must be provided.

Invalid Input Flow

If the input data is invalid:

- The Controller returns a validation error response to the Admin swimlane.
- The system displays the error message to the administrator.
- The process terminates.

This step ensures that only well-formed product data enters the business logic layer.

3. Create Product Entity

If validation is successful, the request is forwarded to the Service layer. The Service constructs a new **Product** entity populated with the submitted information, including name, description, price, category, and brand references.

4. Create Product Variant Entities

The Service checks whether the request includes product variants.

Has Variants Flow

If one or more variants are provided:

- The Service creates a **ProductVariant** entity for each entry, storing the variant name, price, and stock quantity.
- Each variant is associated with the newly created Product entity.

If no variants are provided, this step is skipped and the process continues directly to image handling. This conditional step allows products to be created with or without predefined variants.

5. Create Product Image Entities

The Service then checks whether the request includes image URLs.

Has Image URLs Flow

If image URLs are provided:

- The Service creates a **ProductImage** entity for each URL, linking it to the corresponding Product entity.
- The URLs are expected to be Cloudinary secure URLs obtained from a prior image upload operation.

If no image URLs are provided, this step is skipped. Separating image handling from core product creation allows images to be attached independently after upload.

6. Save All Records to Database

Once all entities have been constructed, the Service passes the complete object graph—Product, ProductVariants, and ProductImages—to the Database layer for persistence in a single operation.

Save Failure Flow

If the save operation fails:

- The Database layer rolls back the transaction, ensuring no partial data is committed.
- A failure response is propagated back through the Service and Controller to the Admin swimlane.
- The system displays a system error message to the administrator.
- The process terminates.

The transactional rollback guarantees data integrity by preventing incomplete product records from being stored in the database.

7. Return Product Response

If the save operation is successful, the Database layer confirms the commit. The Service maps the persisted entities into a **ProductResponse** object and returns it through the Controller to the Admin swimlane. The administrator receives a success response confirming that the product has been created and is now available in the system.

2.1.3.5 Product Variant Management Activity Diagram

This activity diagram describes the main control flow, decision points, and system responses for product variant management activity diagram in the E-Commerce Management System.

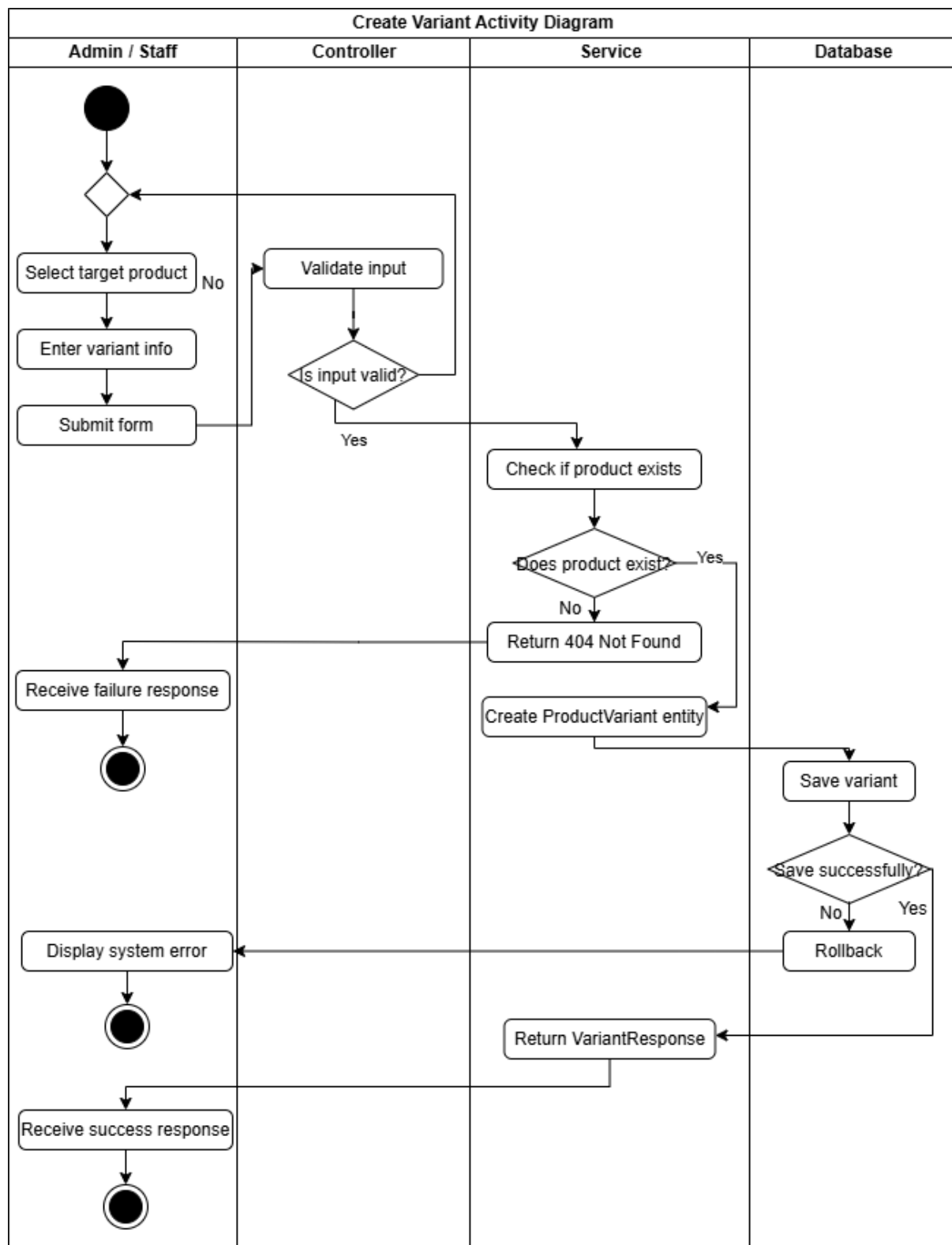


Figure 2.14: Product Variant Management Activity Diagram

Workflow Description

1. Select Target Product and Enter Variant Information

The process begins in the Admin / Staff swimlane. The administrator or staff member selects an existing product to which the new variant will be added. They then enter the variant details, including the variant name, such as color or size combination, price, and stock quantity, before submitting the form to the system.

2. Validate Input

The submitted request is received by the Controller, which validates the input data before passing it to the Service layer.

The validation process checks:

- Required fields, including variant name, price, and quantity.
- Data type and value constraints, such as price and quantity must be positive.

Invalid Input Flow

If the input data is invalid:

- The Controller returns a validation error response to the Admin / Staff swimlane.
- The actor receives a failure response.
- The process terminates.

This step prevents malformed variant data from reaching the business logic layer.

3. Check If Product Exists

If validation passes, the request is forwarded to the Service layer. The Service queries the database to verify that the target product exists and has not been soft-deleted from the system.

Product Not Found Flow

If the product does not exist:

- The Service returns a 404 Not Found error response.
- The response is propagated back to the Admin / Staff swimlane.
- The actor receives a failure response.

- The process terminates.

This check ensures that variants are never created for non-existent or deleted products, preserving referential integrity in the database.

4. Create Product Variant Entity

If the product exists, the Service constructs a new **ProductVariant** entity containing the submitted name, price, and quantity values. The entity is linked to the target product via its foreign key reference.

5. Save Variant to Database

The Service passes the newly created **ProductVariant** entity to the Database layer for persistence.

Save Failure Flow

If the save operation fails:

- The Database layer rolls back the transaction to prevent any partial data from being committed.
- A system error response is propagated back to the Admin / Staff swimlane.
- The system displays a system error message to the actor.
- The process terminates.

The rollback mechanism guarantees that no incomplete variant records are retained in the database in the event of a persistence failure.

6. Return Variant Response

If the save operation is successful, the Database layer confirms the commit. The Service maps the persisted entity into a **VariantResponse** object and returns it through the Controller to the Admin / Staff swimlane. The actor receives a success response confirming that the variant has been created and is now associated with the target product.

2.1.3.6 Product Image Management Activity Diagram

This activity diagram describes the main control flow, decision points, and system responses for product image management activity diagram in the E-Commerce Management System.

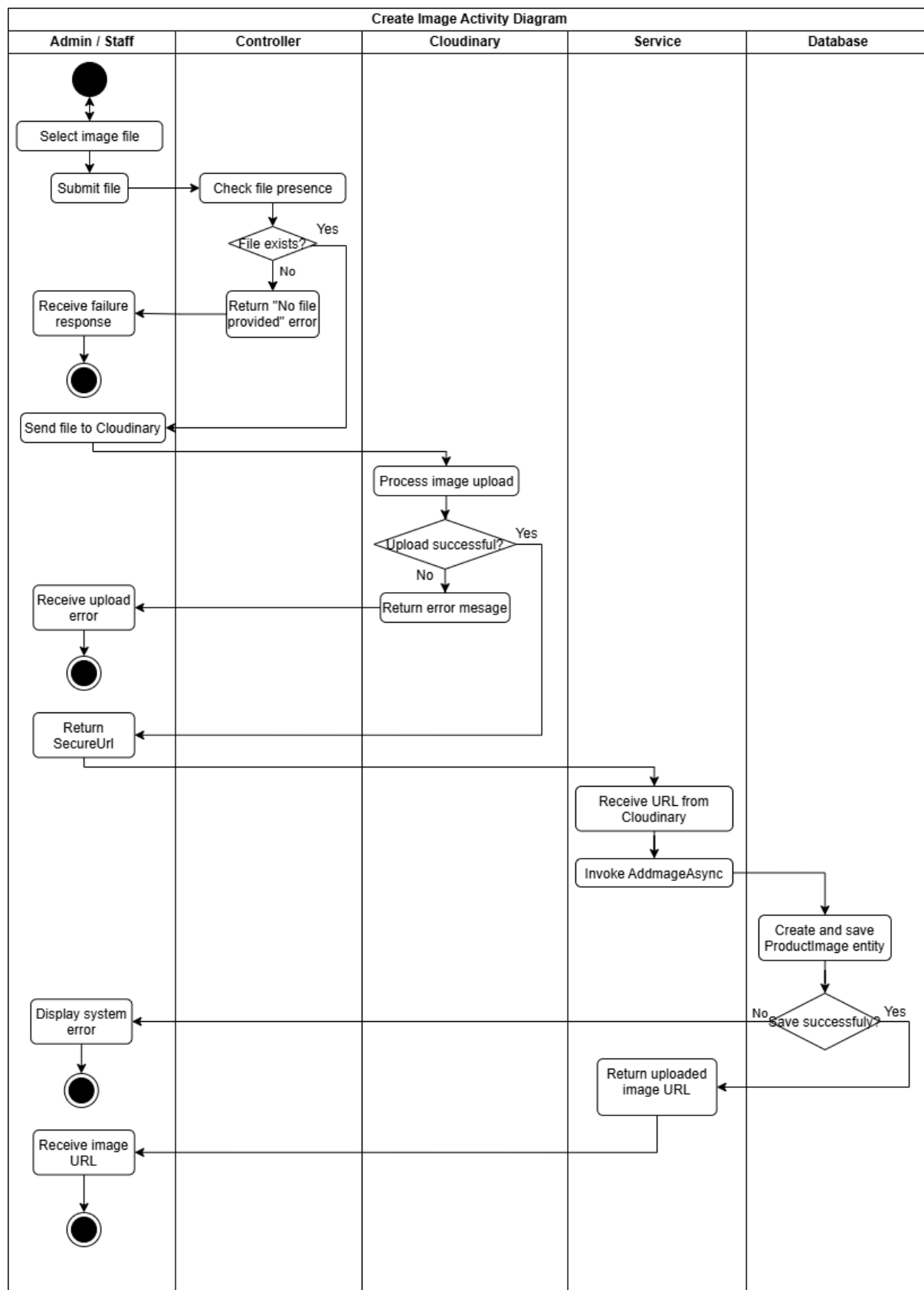


Figure 2.15: Product Image Management Activity Diagram

Workflow Description

1. Select and Submit Image File

The process begins in the Admin / Staff swimlane. The administrator or staff member selects an image file from their local system and submits it to the system for upload.

2. Check File Presence

The submitted request is received by the Controller, which checks whether a file is actually attached to the request before proceeding further.

No File Provided Flow

If no file is detected in the request:

- The Controller returns a ‘No file provided’ error response.
- The Admin / Staff swimlane receives a failure response.
- The process terminates.

This check prevents empty upload requests from reaching the external Cloudinary service, reducing unnecessary external API calls.

3. Send File to Cloudinary

If a file is present, the Controller forwards the file to the Cloudinary swimlane for cloud storage processing. Cloudinary receives the raw image file and begins the upload operation under the designated folder `ecommerce/products`.

Upload Failure Flow

If the Cloudinary upload operation fails:

- Cloudinary returns an error message to the Controller.
- The Controller propagates the error back to the Admin / Staff swimlane.
- The actor receives an upload error notification.
- The process terminates.

Handling Cloudinary failures separately ensures that database records are never created for images that do not physically exist in cloud storage.

4. Return Secure URL

If the upload is successful, Cloudinary returns a `SecureUrl`—the permanent HTTPS address of the stored image. This URL is passed back through the Controller to the Admin / Staff swimlane, which then forwards it to the Service layer to complete the linking process.

5. Receive URL and Invoke `AddImageAsync`

The Service layer receives the `SecureUrl` from Cloudinary. It then invokes the `AddImageAsync` method, which prepares a new `ProductImage` entity containing the URL and the corresponding product identifier, ready for persistence.

6. Create and Save `ProductImage` Entity

The Service passes the `ProductImage` entity to the Database layer for persistence.

Save Failure Flow

If the save operation fails:

- The Database layer returns a failure signal to the Service.
- The Service propagates a system error response back to the Admin / Staff swimlane.
- The system displays a system error message to the actor.
- The process terminates.

This ensures that even when the image is successfully stored in Cloudinary, no broken or unlinked image reference is retained in the database if the persistence step fails.

7. Return Uploaded Image URL

If the save operation is successful, the Database layer confirms the commit. The Service returns the uploaded image URL as the final response through the Controller to the Admin / Staff swimlane. The actor receives the image URL, confirming that the image has been successfully uploaded to Cloudinary and permanently linked to the target product in the system.

2.1.3.7 Shopping Cart Activity Diagram

This activity diagram describes the main control flow, decision points, and system responses for shopping cart activity diagram in the E-Commerce Management System.

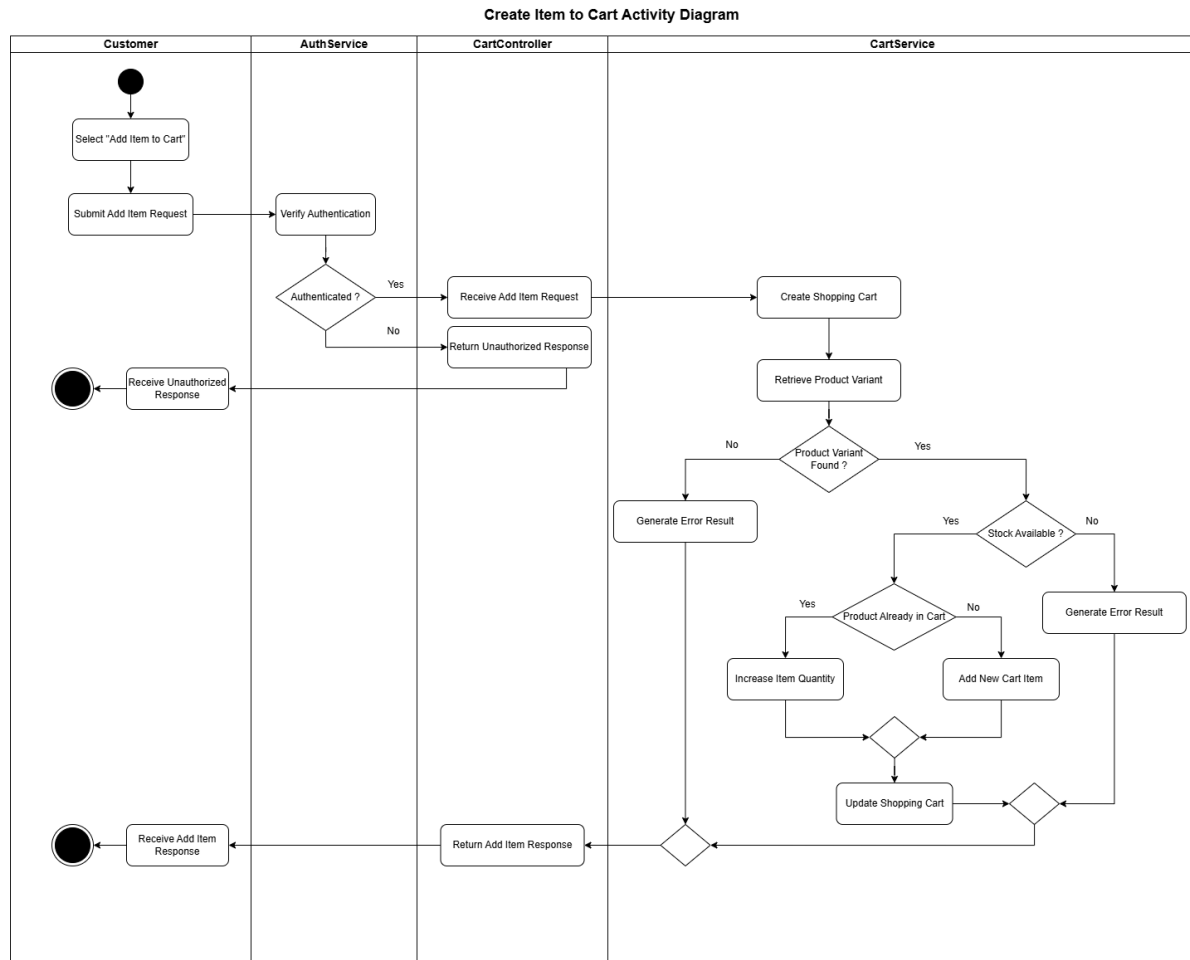


Figure 2.16: Shopping Cart Activity Diagram

Workflow Description

1. Select Add Item to Cart

The process begins in the Customer swimlane when the customer selects the *Add Item to Cart* action from a product page or product listing. After selecting the product variant and desired quantity, the customer submits an add-item request to the system.

2. Verify Authentication

The request is first handled by the Auth Service, where the system verifies whether the customer is authenticated.

Unauthenticated Flow

If the customer is not authenticated:

- The Cart Controller returns an unauthorized response.
- The customer receives the unauthorized response.
- The process terminates without changing the shopping cart.

This validation ensures that cart data is associated with a valid customer account and prevents unauthorized cart manipulation.

3. Receive Add Item Request

If authentication is successful, the Cart Controller receives the add-item request and forwards it to the Cart Service for business processing.

4. Create or Retrieve Shopping Cart

The Cart Service creates a shopping cart for the customer if one does not already exist. If the customer already has an active cart, the service uses the existing cart to continue the operation.

5. Retrieve Product Variant

After identifying the customer's cart, the Cart Service retrieves the selected product variant from the product data source.

Product Variant Not Found Flow

If the requested product variant cannot be found:

- The Cart Service generates an error result.
- The Cart Controller returns the add-item response containing the error information.
- The customer receives the add-item response.

This step prevents invalid or deleted product variants from being added to the cart.

6. Check Stock Availability

If the product variant exists, the Cart Service checks whether enough stock is available for the requested quantity.

Out-of-Stock Flow

If stock is not available:

- The Cart Service generates an error result.

- The Cart Controller returns the add-item response with the stock validation result.
- The customer receives the add-item response.

This validation ensures that customers cannot add unavailable quantities to the shopping cart.

7. Check Existing Cart Item

When the product variant exists and stock is available, the Cart Service checks whether the same product variant is already present in the customer's cart.

Existing Item Flow

If the product is already in the cart, the system increases the quantity of the existing cart item instead of creating a duplicate row.

New Item Flow

If the product is not yet in the cart, the system creates a new cart item with the selected product variant and quantity.

8. Update Shopping Cart

After either increasing the existing item quantity or adding a new cart item, the Cart Service updates the shopping cart. The updated cart state includes the latest item list, quantities, and cart summary information.

9. Return Add Item Response

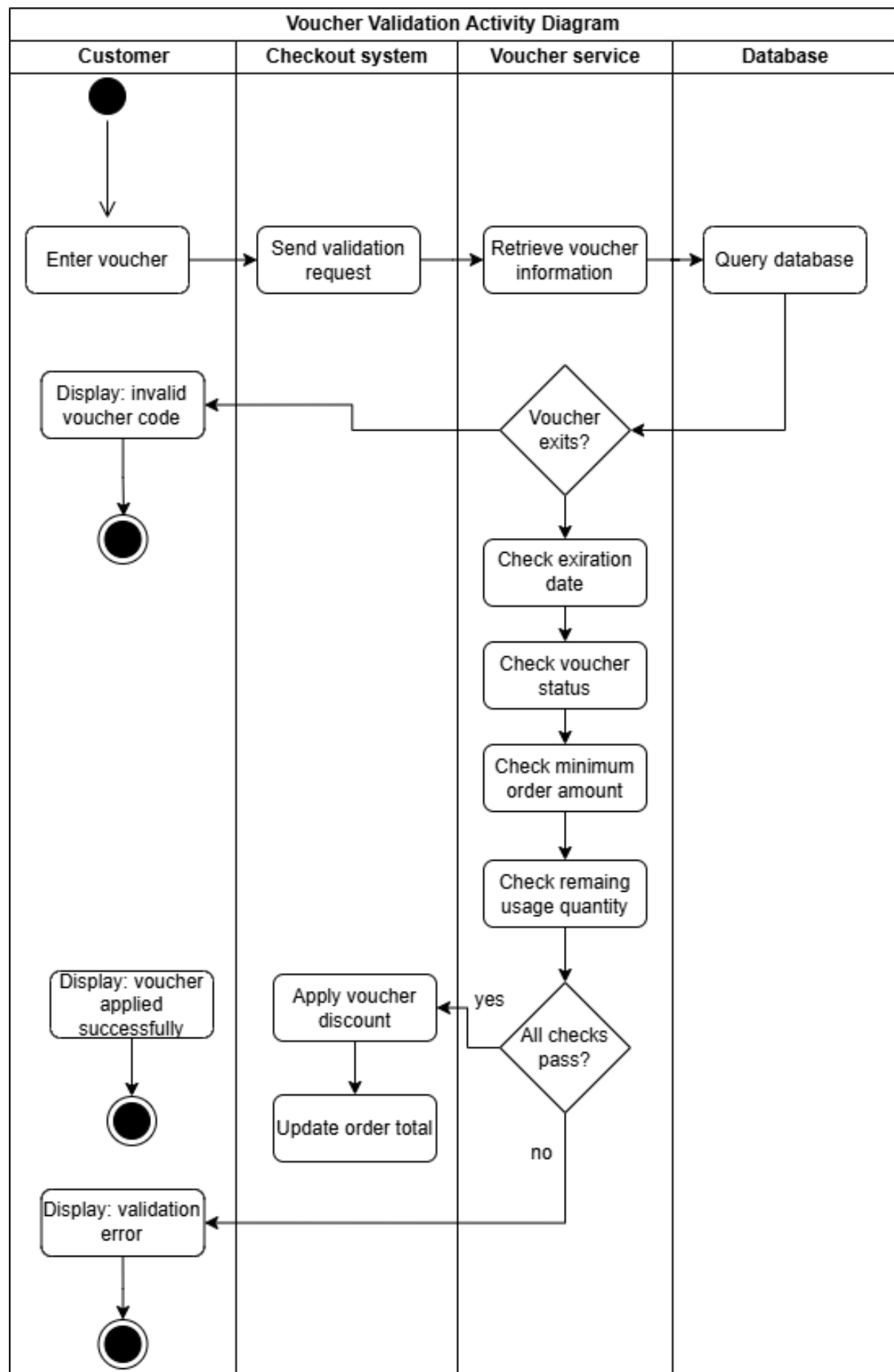
The Cart Service returns the processing result to the Cart Controller. The controller then sends the add-item response back to the customer.

10. End

The process ends after the customer receives the add-item response. A successful response means the shopping cart has been updated, while an error response explains why the requested item could not be added.

2.1.3.8 Voucher Validation Activity Diagram

This activity diagram describes the main control flow, validation logic, decision points, and system responses for voucher validation activity in the E-Commerce Management System.



Workflow Description

1. Enter Voucher Code

The process begins when the customer enters a voucher code during the checkout process. The customer inputs a voucher code into the checkout form.

2. Send Validation Request

The checkout system sends the voucher code to the voucher service for validation.

3. Check Voucher Existence

The voucher service verifies whether the voucher exists in the database. If the voucher exists, the validation process continues.

Invalid Voucher Flow

If the voucher does not exist:

- The system displays an invalid voucher message.
- The customer receives the invalid voucher response.
- The process ends.

4. Check Voucher Expiration

The system checks whether the voucher is still valid based on its expiration date. If the voucher has not expired, the process continues.

Expired Voucher Flow

If the voucher has expired:

- The system displays an expiration warning.
- The customer receives the expiration response.
- The process ends.

5. Check Voucher Conditions

The system verifies business conditions such as:

- Minimum order amount

- Voucher usage limit
- Customer eligibility
- Product/category restrictions

Not Eligible Flow

If any condition fails:

- The system displays a voucher condition failure message.
- The customer receives the condition failure response.
- The process ends.

6. Calculate Discount

If all conditions are satisfied, the voucher service calculates the discount amount based on voucher configuration.

7. Apply Voucher and Update Order Summary

The checkout system applies the discount to the current order total. The order system updates:

- Discount amount
- Final payable amount
- Voucher information

8. Display Updated Checkout Information

The customer receives the updated checkout summary with the applied voucher discount.

9. End

The process ends after the voucher has been successfully validated and applied.

2.1.3.9 Order Creation Activity Diagram

This activity diagram describes the main control flow, business validation, payment handling, and system responses for order creation activity in the E-Commerce Management System.

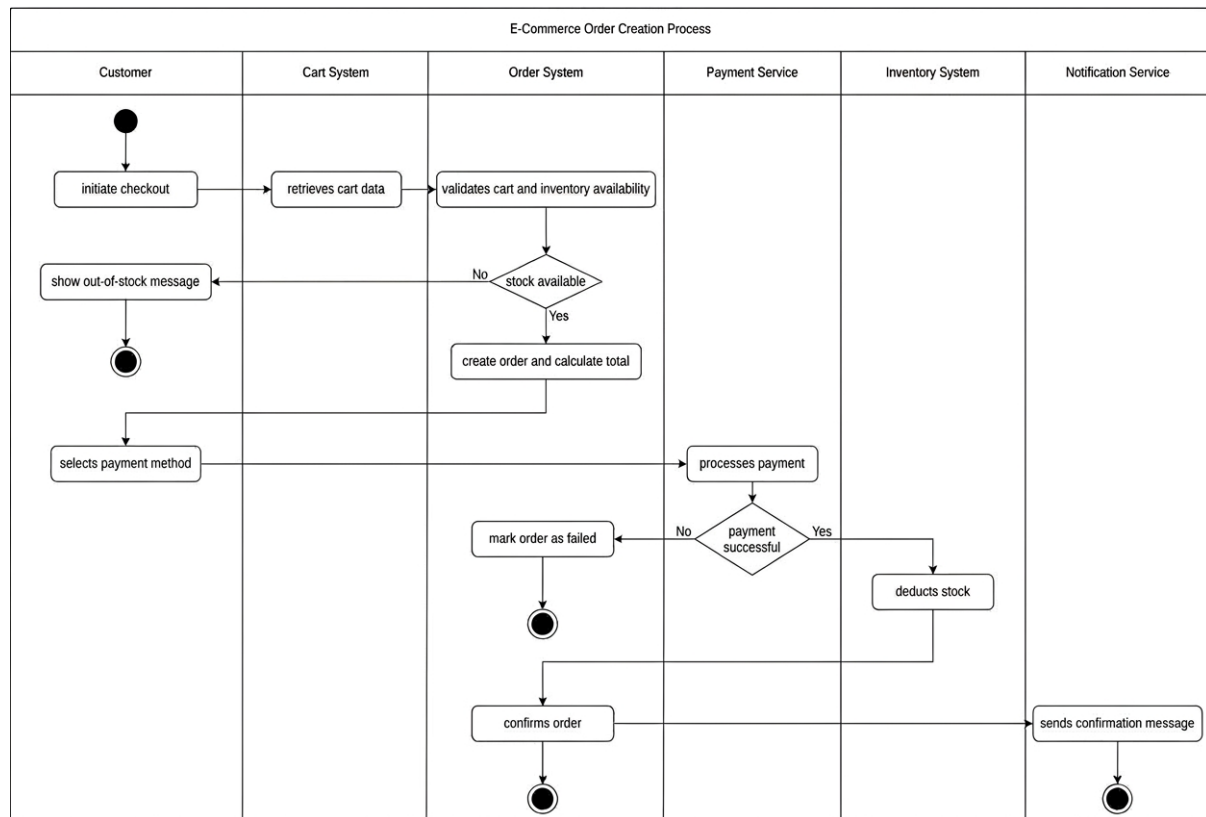


Figure 2.18: Order Creation Activity Diagram

Workflow Description

1. Open Checkout Page

The process begins when the customer proceeds to checkout from the shopping cart. The customer reviews selected cart items and shipping information.

2. Validate Shopping Cart

The cart system verifies:

- Product availability
- Quantity validity
- Active cart status

Invalid Cart Flow

If issues exist:

- The system displays an error message.

- The customer receives the error response.
- The process ends.

3. Check Inventory

If the cart is valid, the inventory system checks stock availability for all product variants. If stock is sufficient, inventory is reserved temporarily.

Stock Unavailable Flow

If stock is insufficient:

- The system notifies the customer.
- The customer receives the stock unavailable message.
- The process ends.

4. Select Payment Method

The customer selects a payment method such as Cash, Momo, or VNPay.

5. Process Payment

The payment gateway processes the transaction request. If payment succeeds, the order creation process continues.

Payment Failed Flow

If payment fails:

- The system displays a payment failure message.
- The customer receives the failure response.
- The process ends.

6. Create Order

The order system creates:

- Order record
- Order details

- Payment information

7. Update Inventory and Clear Shopping Cart

The inventory system deducts product quantities after successful order creation. The cart system removes purchased items from the customer's cart.

8. Send Order Confirmation and Update Order Status

The notification service sends order confirmation details to the customer. The system updates the order status to **Pending** or **Processing** depending on payment result.

9. End

The process ends after the order has been successfully created and confirmed.

2.1.3.10 Payment Processing Activity Diagram

This activity diagram describes the main control flow, validation logic, payment handling, and system responses for payment processing activity in the E-Commerce Management System.

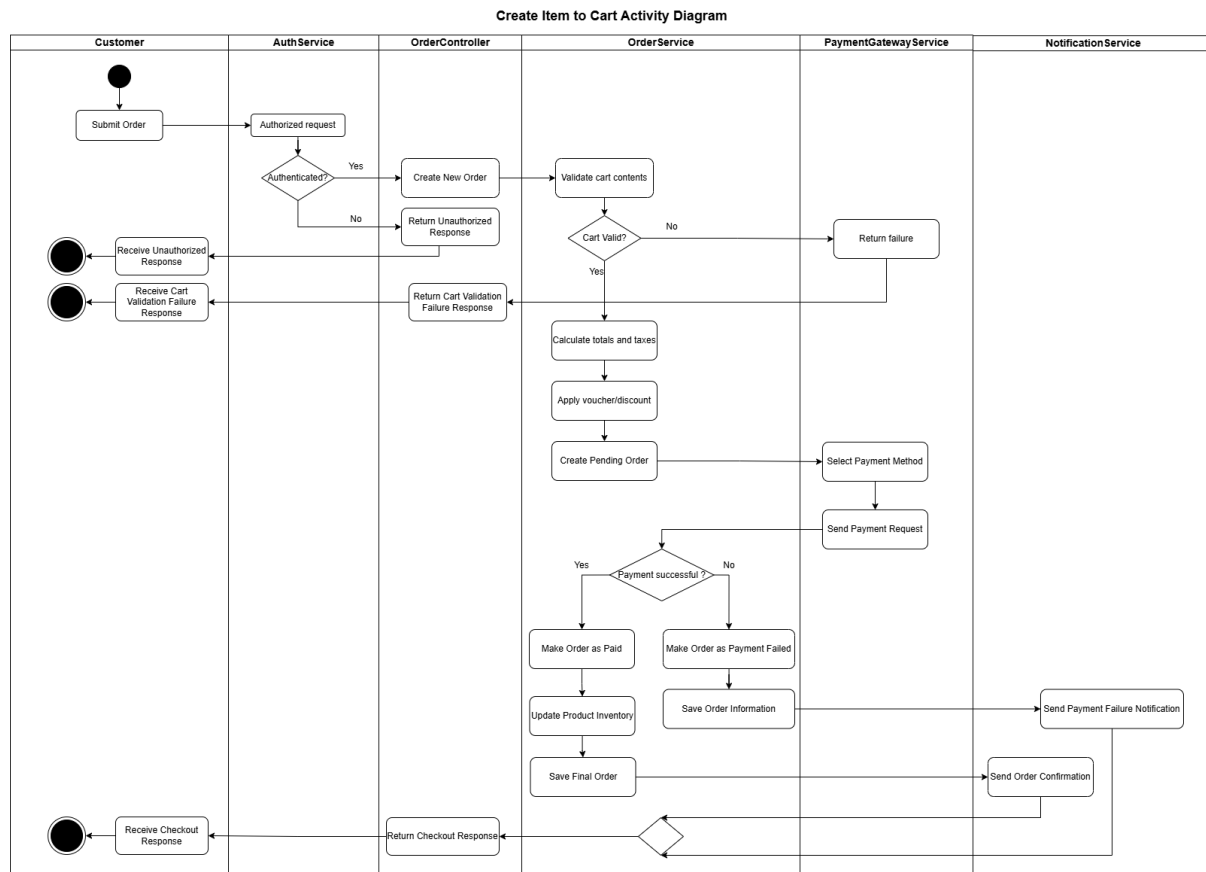


Figure 2.19: Payment Processing Activity Diagram

Workflow Description

1. Submit Order

The process begins in the Customer swimlane when the customer submits an order from the checkout page. The request contains the selected cart items, shipping information, voucher information if provided, and the selected payment option.

2. Authorize Request

The request is first handled by the Auth Service. The system checks whether the customer is authenticated before allowing the checkout process to continue.

Unauthenticated Flow

If the customer is not authenticated:

- The Order Controller returns an unauthorized response.
- The customer receives the unauthorized response.

- The process terminates without creating an order.

This step prevents unauthorized users from creating orders or accessing private cart information.

3. Create New Order Request

If authentication is successful, the Order Controller receives the checkout request and starts the order creation process. The controller forwards the request to the Order Service for business validation and payment preparation.

4. Validate Cart Contents

The Order Service validates the customer's cart contents before creating the order. The validation checks whether the selected cart items exist, belong to the current customer, have valid quantities, and are still available for purchase.

Cart Validation Failure Flow

If the cart is invalid:

- The Order Service returns a failure result.
- The Order Controller returns a cart validation failure response.
- The customer receives the cart validation failure response.

This validation ensures that invalid, empty, or outdated cart data cannot be used to create an order.

5. Calculate Totals and Taxes

If the cart is valid, the Order Service calculates the order subtotal, taxes, shipping cost, and final payable amount. These calculations prepare the order summary before payment is requested.

6. Apply Voucher or Discount

The Order Service applies voucher or discount information when the customer provides a valid promotion. The discount amount is deducted from the order total before the pending order is created.

7. Create Pending Order

After calculating the final amount, the Order Service creates a pending order. At this stage, the order is stored with a temporary status while the payment result is being processed.

8. Select Payment Method and Send Payment Request

The Payment Gateway Service receives the payment information, selects the required payment method, and sends the payment request to the payment provider. This step handles online payment processing before the order is finalized.

9. Check Payment Result

The system checks whether the payment transaction is successful.

Successful Payment Flow

If the payment is successful:

- The Order Service marks the order as paid.
- The system updates product inventory based on the purchased quantities.
- The final order information is saved.
- The Notification Service sends an order confirmation to the customer.

This flow confirms that the payment has been completed and that the order can move forward to fulfillment.

Payment Failure Flow

If the payment fails:

- The Order Service marks the order as payment failed.
- The system saves the order information with the failed payment status.
- The Notification Service sends a payment failure notification to the customer.

This flow preserves payment failure information and informs the customer that the check-out process was not completed successfully.

10. Return Checkout Response

After either the successful payment flow or the payment failure flow is handled, the Order Controller returns the checkout response to the customer.

11. End

The process ends when the customer receives the checkout response. A successful response indicates that the order has been paid, inventory has been updated, and confirmation has

been sent. A failure response explains whether the issue was caused by authentication, cart validation, or payment processing.

2.1.3.11 VNPay Activity Diagram

This activity diagram describes the secure VNPay payment flow, including payment URL generation, customer redirection, asynchronous IPN processing, callback verification, order status updates, and frontend payment result display in the E-Commerce Management System.

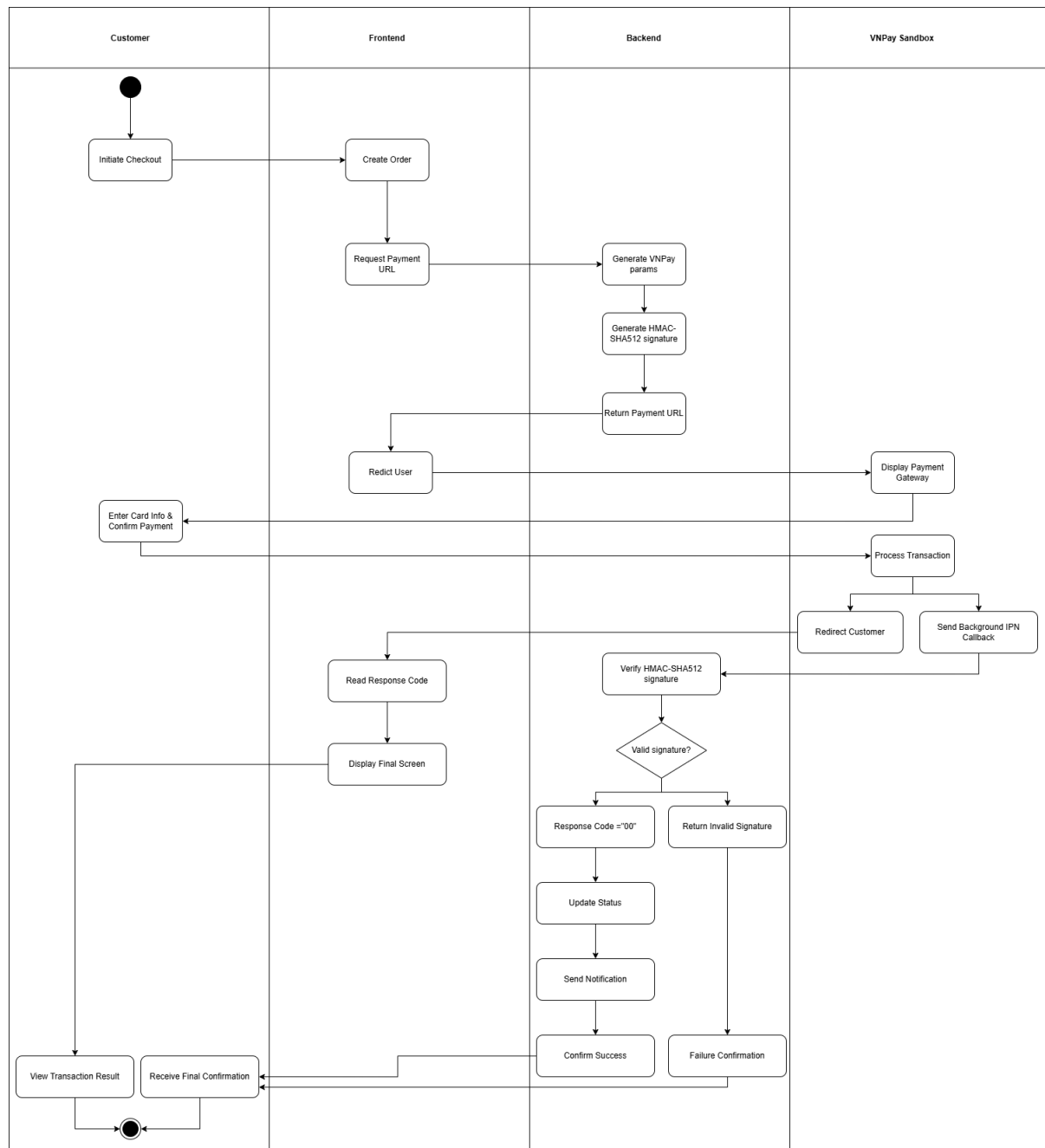


Figure 2.20: VNPay Activity Diagram

Workflow Description

1. Initiate Payment Request

The process begins in the Customer swimlane when the customer confirms checkout. The request is forwarded to the Frontend swimlane, which first calls the `POST /api/orders` endpoint to create the order record. After the order is created, the frontend calls the `POST /api/payment/vnpay/create` endpoint with the generated `orderId` to request a

secure payment URL.

Business Justification: This step initiates the secure handoff between the internal order management system and the external payment provider.

2. Generate Payment URL and Signature

The request reaches the Backend swimlane. The backend constructs the required VNPAY parameters, maps the `orderId` to `vnp_TxnRef`, sets expiration dates, and multiplies the `TotalAmount` by 100 to satisfy VNPAY's base unit requirement. The backend then sorts the parameters alphabetically, URL-encodes them, and generates a secure cryptographic signature using the HMAC-SHA512 algorithm together with the merchant's `HashSecret`. Finally, the backend returns the fully constructed payment URL to the frontend.

Business Justification: Generating the HMAC-SHA512 signature on the backend ensures that payment request data cannot be tampered with or intercepted during network transit.

3. Redirect to Payment Gateway and Authenticate

After receiving the payment URL, the Frontend swimlane redirects the customer's browser to the VNPAY Sandbox swimlane. VNPAY displays its secure payment portal, where the customer enters card information and confirms the transaction through OTP verification.

Business Justification: Delegating sensitive card data entry directly to VNPAY helps the system maintain PCI DSS compliance because raw card details never pass through the application's servers.

4. Process IPN Webhook (Asynchronous Branch)

After the transaction is processed, the workflow splits into two parallel operations. In the asynchronous backend branch, the VNPAY Sandbox swimlane sends an Internet Payment Notification (IPN) request to `GET /api/payment/vnpay/ipn`. The backend extracts the URL parameters and verifies the HMAC-SHA512 signature before accepting any payment result.

Invalid Signature Flow

If the HMAC-SHA512 signature does not match the backend calculation:

- The backend rejects the IPN data and returns `RspCode "97"` to VNPAY.
- The customer receives no success notification because the system ignores the invalid request.
- The webhook process terminates.

Business Justification: Strict signature validation prevents malicious actors from spoofing successful payment callbacks to falsely authorize orders or reserve inventory.

Successful Payment Flow

If the signature is valid and `vnp_ResponseCode` equals "00":

- The backend updates the `OrderStatus` to `Processing`.
- The system sends an in-app order confirmation notification to the customer.
- The backend returns `RspCode` "00" to VNPay.
- The backend process completes successfully.

Business Justification: This ensures that fulfillment and logistics processing only begin after the banking provider legitimately confirms the payment.

Failed Payment Flow

If the signature is valid but `vnp_ResponseCode` does not equal "00":

- The backend updates the `OrderStatus` to `Cancelled`.
- The system sends a `PAYMENT_FAILED` in-app notification to the customer.
- The backend returns `RspCode` "00" to VNPay to acknowledge receipt.
- The backend process completes the cancellation flow.

Business Justification: This automated status update immediately frees locked inventory when a customer's payment is declined.

5. Process Return URL and Display Result (Synchronous Branch)

In parallel with the IPN branch, the VNPay Sandbox swimlane redirects the user back to the Frontend swimlane at the `/payment/result` route. The frontend extracts `vnp_ResponseCode` from the URL parameters to determine the displayed payment result.

Successful Display Flow

If `vnp_ResponseCode` equals "00":

- The frontend renders the payment success interface.

- The customer views confirmation that the transaction was processed successfully.
- The process ends.

Failed Display Flow

If `vnp_ResponseCode` does not equal "00":

- The frontend renders the payment failure interface.
- The customer views an error message explaining that the transaction was declined.
- The process ends.

2.1.3.12 Order Cancellation Activity Diagram

This activity diagram describes the main control flow, validation logic, refund handling, and system responses for order cancellation activity in the E-Commerce Management System.

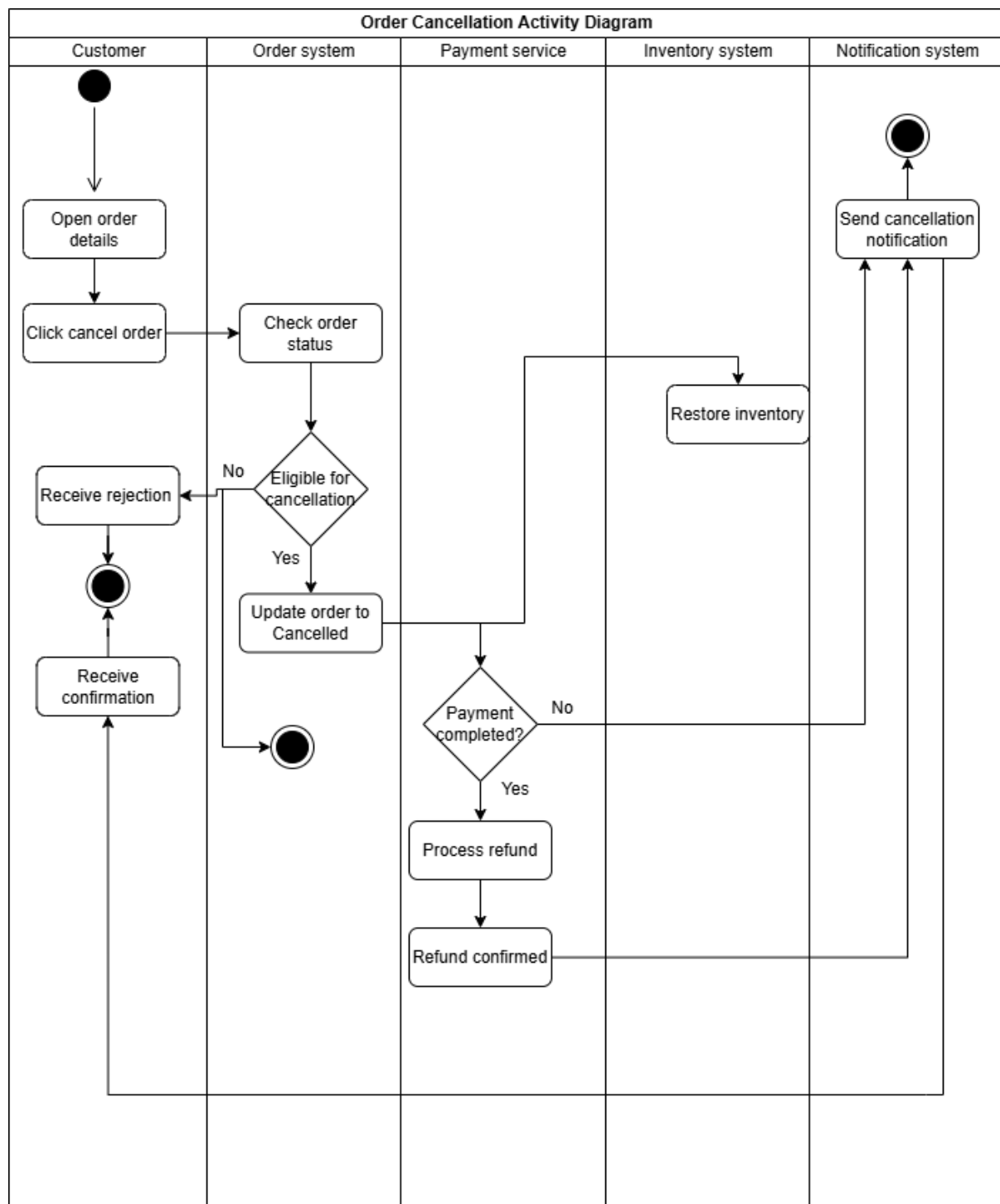


Figure 2.21: Order Cancellation Activity Diagram

Workflow Description

1. Select Order and Send Cancellation Request

The process begins when the customer requests order cancellation. The customer selects

a previously created order from the order history page, and the order system receives the cancellation request.

2. Check Order Status

The system verifies whether the order is eligible for cancellation. Orders in Pending or Processing status can continue cancellation.

Not Eligible Flow

If delivered or shipped orders cannot be canceled:

- The system displays a rejection message.
- The customer receives the rejection response.
- The process ends.

3. Update Order Status

The order system updates the order status to **Cancelled**.

4. Restore Inventory

The inventory system restores product quantities back to stock.

5. Check Payment Status

The system checks whether the order payment has already been completed. If the order is unpaid, refund processing is skipped.

Paid Order Flow

If the order payment has already been completed:

- Refund processing begins.
- The payment gateway processes the refund transaction.
- The process continues after the refund result is handled.

Refund Failed Flow

If the refund transaction fails:

- The system flags the refund issue for manual handling.

- The customer receives the cancellation handling response.
- The process continues to notification handling.

6. Send Cancellation Notification

The notification service sends cancellation confirmation to the customer.

7. End

The process ends after the order cancellation workflow is completed.

2.1.3.13 Review Management Activity Diagram

This activity diagram describes the main control flow, validation logic, moderation process, and system responses for review management activity in the E-Commerce Management System.

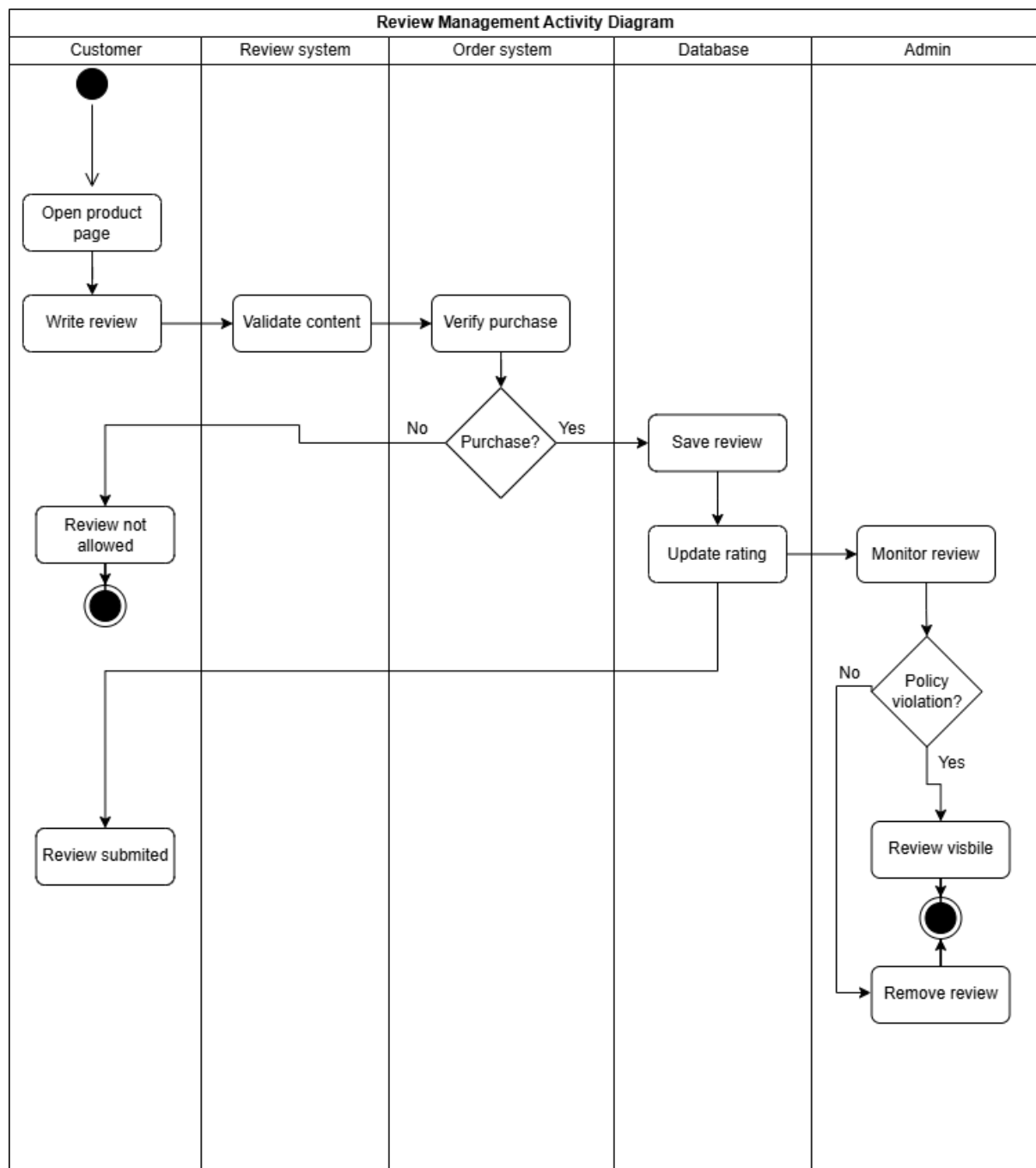


Figure 2.22: Review Management Activity Diagram

Workflow Description

1. Select Purchased Product

The process begins when the customer submits a product review. The customer selects a product from the completed order history.

2. Enter and Submit Review Information

The customer enters:

- Rating score
- Review comment

The review system receives the review request.

3. Verify Purchase History

The order system checks whether the customer has purchased the selected product. If the customer has purchased the product, the review validation process continues.

Not Purchased Flow

If the customer has never purchased the product:

- The system rejects the review request.
- The customer receives the rejection response.
- The process ends.

4. Validate Review Content

The review system validates:

- Rating range
- Empty content
- Inappropriate language

Invalid Review Flow

If validation fails:

- The system displays an error message.
- The customer receives the validation failure response.
- The process ends.

5. Save Review

If the review content is valid, the review system stores the review information in the database.

6. Update Product Rating

The product system recalculates the product average rating based on all reviews.

7. Moderate Review

Staff or Admin may review submitted content for policy violations. If approved, the review remains publicly visible.

Review Removed Flow

If the review violates policy:

- Staff or Admin hides or deletes the review from the system.
- The review is removed from public visibility.
- The process continues to completion.

8. Display Review

The approved review becomes visible on the product detail page.

9. End

The process ends after the review workflow is completed successfully.

2.1.3.14 Support Request Activity Diagram

This activity diagram describes the main control flow, decision points, and system responses for support request activity diagram in the E-Commerce Management System.

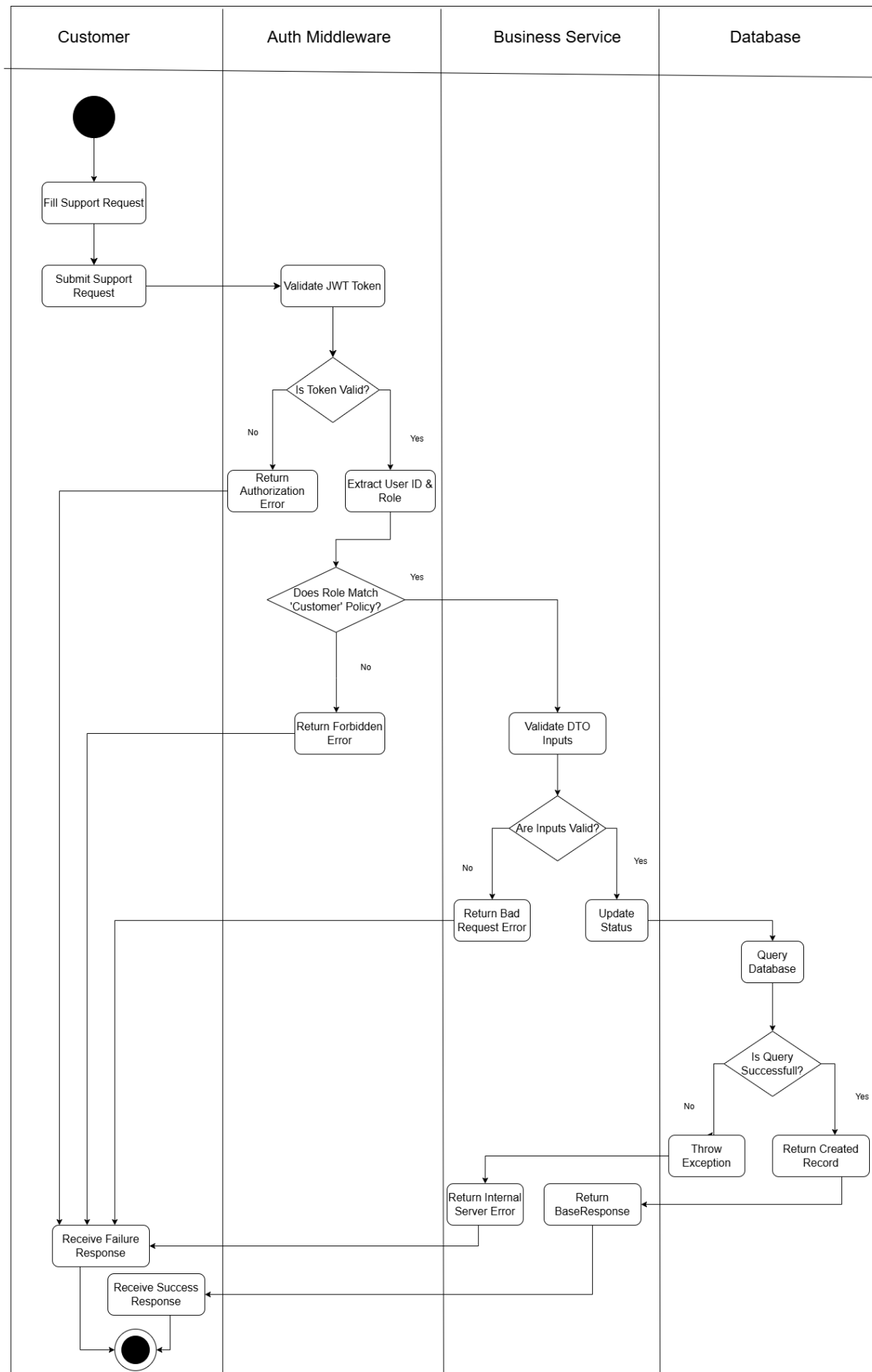


Figure 2.23: Support Request Activity Diagram

Workflow Description

1. Submit Support Request

The process initiates in the Client application when the user fills out the support ticket form containing the subject, category, and message details. After completing the form, the client application sends a POST request to the system API.

2. Authenticate and Authorize Request

The request is first intercepted by the Auth Middleware, which validates the user's identity and permissions before allowing access to the business logic.

The validation process checks:

- The presence and validity of the JWT Token.
- The authorization role (verifying the user has 'Customer' privileges).

Authentication Failure Flow

If the authentication or authorization fails:

- The middleware immediately returns a 401 Unauthorized or 403 Forbidden exception.
- The user receives the failure response.
- The process terminates.

This step ensures that only logged-in customers can submit support tickets and protects the endpoint from unauthorized access.

3. Validate Input Data

If authenticated successfully, the request is forwarded to the Business Service layer (Support Controller), where the system validates the data payload (DTO).

Invalid Data Flow

If the input data is missing or invalid:

- The system returns a 400 Bad Request error.
- The user receives a validation failure message.

- The process ends.

4. Process Business Logic

Upon successful data validation, the service maps the incoming data to a system entity. During this mapping phase, the system automatically assigns a default status of "Pending" to the new request, ensuring it is properly queued for the support staff.

5. Save Record to Database

The system sends the processed entity to the Database layer for persistence. The database attempts to execute the insert query to save the new support ticket.

Save Failure Flow

If the database operation fails (e.g., connection drop or constraint violation):

- The system triggers a 500 Internal Server Error.
- The user receives a system failure notification.
- The process terminates.

Save Success Flow

If the database operation succeeds:

- The system maps the saved record back into a response object.
- The user receives a 200 OK confirmation that the ticket was created successfully.
- The process completes.

2.1.3.15 Staff Order Processing Activity Diagram

This activity diagram describes the main control flow, decision points, and system responses for staff order processing activity diagram in the E-Commerce Management System.

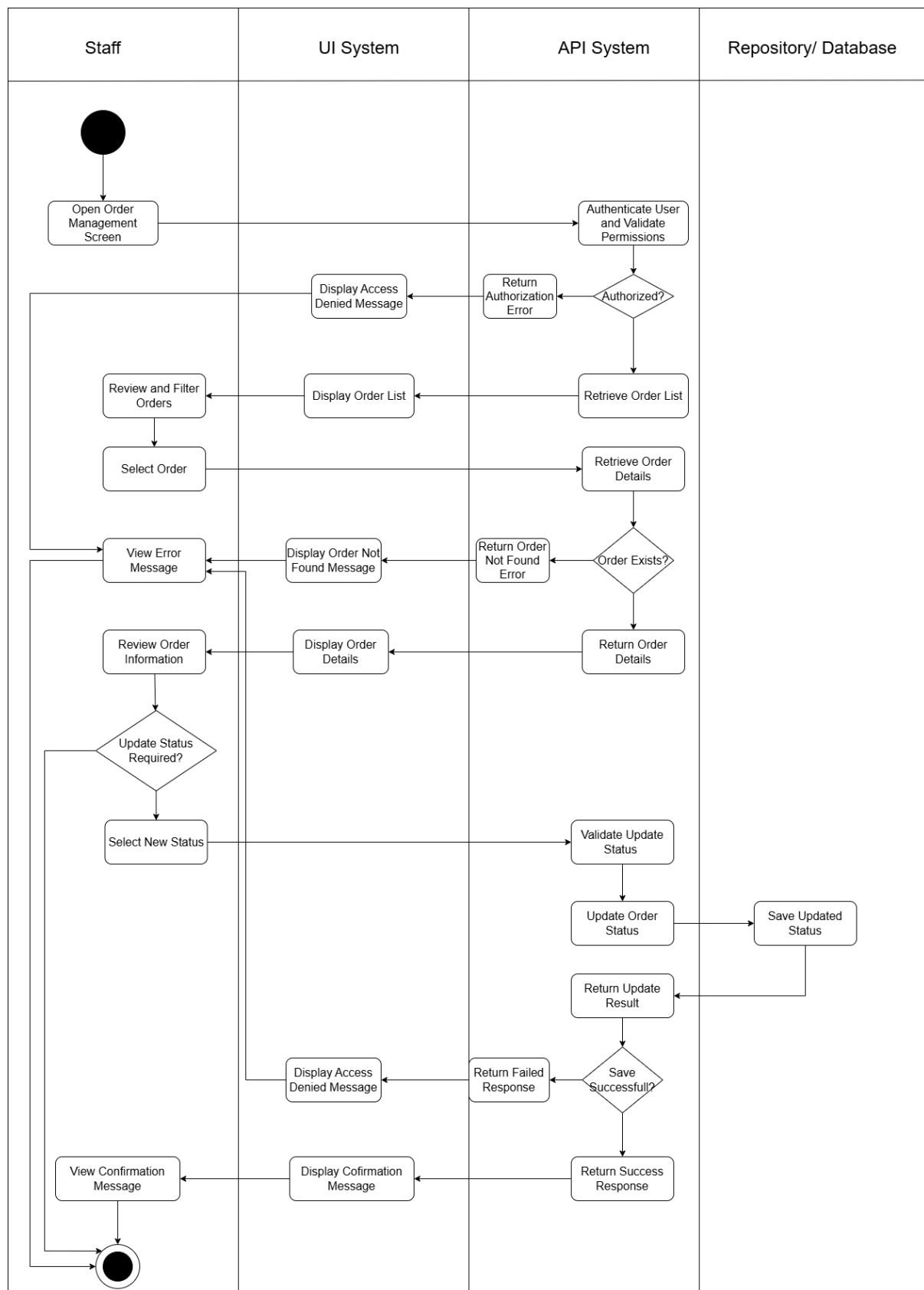


Figure 2.24: Staff Order Processing Activity Diagram

Workflow Description

1. Open Order Management Screen

The process begins when a staff member opens the Order Management module. The staff accesses the order management screen through the UI System.

2. Authenticate User and Validate Permissions

The system authenticates the user and verifies access permissions. The system determines whether the user is authorized to access order management functions.

Not Authorized Flow

If the user is not authorized:

- The system returns an authorization error.
- The UI displays an access denied message.
- The process ends.

3. Retrieve and Display Order List

If the user is authorized, the system proceeds to retrieve the order list from the database. The UI displays the available orders.

4. Review, Filter, and Select Order

The staff reviews and filters the order list to locate a specific order. The staff selects an order for review.

5. Retrieve Order Details

The system retrieves detailed information about the selected order, including:

- Order information
- Ordered products
- Shipping address
- Current status
- Customer information

Order Not Found Flow

If the selected order does not exist:

- The system returns an order not found error.
- The UI displays an order not found message.
- The staff reviews the error message, and the process returns to the order list.

6. Return and Display Order Details

If the order exists, the system returns the order details. The UI presents the order information, and the staff reviews the order details.

7. Determine Whether a Status Update Is Required

The system determines whether the staff intends to update the order status. If no status update is required, the staff only reviews the information and the process ends.

8. Select New Status

If a status update is required, the staff chooses the target status for the order. Examples include:

- Pending
- Processing
- Shipped
- Delivered
- Cancelled

9. Validate Status Update

The system validates the requested status transition. Validation includes:

- Whether the target status is valid.
- Whether the transition is allowed.
- Whether the user has permission to perform the update.

Invalid Status Update Flow

If the status update is not valid:

- The system returns an update failure response.
- The UI displays an error message.
- The process returns to the order details screen.

10. Update Order Status and Save Updated Status

If the status update is valid, the system updates the order status. The updated status is persisted in the database.

Save Failure Flow

If the update operation is not completed successfully:

- The system returns a failure response.
- The UI displays an error message.
- The staff reviews the error message, and the process returns to the order details screen.

11. Return Success Response

If the save operation is successful, the API returns a successful update result.

12. Display Confirmation Message

The UI displays a confirmation message. The staff reviews the confirmation message.

13. End

The process ends when the order status has been successfully updated and confirmed, or when an error occurs and no further action is taken.

2.2 Database

2.2.1 Users Table

The Users Table stores account information for all users in the E-Commerce Management System, including authentication data, contact details, spending information, role assignment, and soft delete status.

Field Name	Data Type	Constraint	Description / Business Meaning
Id	Guid	Primary Key, NOT NULL	Primary key storing a unique user identifier as a GUID.
Name	Nvarchar(100)	NOT NULL	Login name or short name. Required, maximum 100 characters.
Password	Nvarchar(100)	NOT NULL	Password, hashed before storage. Required.
Email	Nvarchar(100)	NOT NULL, Usually Unique	User's email address. Used for login or password recovery.
FullName	Nvarchar(100)	NULL	Full name of the user. Optional.
PhoneNumber	Varchar(10)	NULL	User's contact phone number, maximum 10 digits.
Address	Nvarchar(max)	NULL	Default residential address of the user.
CreatedDate	Datetime	NOT NULL, DEFAULT GETDATE()	Account creation time. Defaults to current system time.
totalSpend	Decimal(18,2)	NOT NULL, DEFAULT 0, CHECK (≥ 0)	Total amount spent by the user on the E-Commerce system. Used for membership tier, voucher, or discount evaluation.
RoleId	Guid	NOT NULL, Foreign Key	Foreign key pointing to the Role table, used for authorization such as Admin, Staff, Customer, etc.
IsDeleted	Boolean	NOT NULL, DEFAULT 0 (False)	Soft delete status flag. If 1, the account is locked or hidden from the system.

Table 2.1: Structure of Users Table

2.2.2 Roles Table

The Roles Table stores authorization role information used to control access permissions in the E-Commerce Management System.

Field Name	Data Type	Constraint	Description / Business Meaning
Id	Guid	Primary Key, NOT NULL, DEFAULT NEWID()	Primary key storing the role identifier as a GUID. The system automatically generates a new GUID upon initialization.
Name	Nvarchar(50)	NOT NULL, Usually Unique	Name of the system authorization role, such as Admin, Customer, or Staff. Required, maximum 50 characters.

Table 2.2: Structure of Roles Table

2.2.3 Products Table

The **Products** table stores core product catalog information used for searching, filtering, displaying, and managing products in the E-Commerce Management System.

Field Name	Data Type	Constraints	Description / Business Meaning
Id	Guid	PRIMARY KEY, NOT NULL, ValueGeneratedOnAdd (EF)	Unique identifier for the product. Uses GUID to avoid distributed data conflicts.
Name	Nvarchar(100)	NOT NULL	Product name displayed to the user and used for searching/filtering.
Description	Nvarchar(200)	NULL	Short description for the product. Can be left blank.

Price	Decimal(18,2)	NOT NULL	Base price of the product, used in various display/calculation flows, even if variants have their own prices.
AverageRating	Float	NOT NULL	Average rating of the product. Review business logic usually keeps this value in the range of 0–5 at the application level.
CategoryId	Guid	NOT NULL, FOREIGN KEY, INDEX	The category the product belongs to. Mandatory for classification, filtering, and catalog management.
BrandId	Guid	NOT NULL, FOREIGN KEY, INDEX	The brand of the product. Mandatory for filtering by brand and product management.
IsDeleted	Boolean	NOT NULL, DEFAULT (false)	Soft delete flag. Deleted products will not appear in the main business queries, such as GetProductsQuery filters using !IsDeleted.

Table 2.3: Structure of Products Table

2.2.4 Product Variants Table

The `ProductVariants` table stores different sellable versions of a product, such as color, size, storage capacity, or other product-specific options.

Field Name	Data Type	Constraint	Description / Business Meaning
------------	-----------	------------	--------------------------------

Id	Guid	Primary Key, NOT NULL, not AUTO INCREMENT, value generated at the application layer	Unique identifier for a product variant
ProductId	Guid	Foreign Key, NOT NULL	Determines which product this variant belongs to
Name	Nvarchar(max)	NOT NULL	Variant name, e.g., “Red”, “Size M”, “128GB”
Price	Decimal(18,2)	NOT NULL	Selling price of the variant
Quantity	int	NOT NULL	Inventory quantity of the variant

Table 2.4: Structure of Product Variants Table

2.2.5 Product Images Table

The **ProductImages** table stores image records associated with products, including the image URL used for frontend product display.

Field Name	Data Type	Constraint	Description / Business Meaning
Id	Guid	Primary Key, NOT NULL, non-auto-increment, generated at application layer	Unique identifier for each product image. Primary key generated using <code>Guid.NewGuid()</code> .
ProductId	Guid	Foreign Key, NOT NULL	Identifies which product this image belongs to. Mandatory field.
ImageUrl	Nvarchar(max)	NOT NULL	Path/URL of the uploaded image, such as Cloudinary. Main data for frontend display.

Table 2.5: Structure of Product Images Table

2.2.6 Brands Table

The **Brands** table stores brand information used for product catalog display, product management, and brand-based filtering in the E-Commerce Management System.

Field Name	Data Type	Constraints	Description / Business Meaning
Id	Uniqueidentifier / Guid	PRIMARY KEY, NOT NULL, application default (Guid.NewGuid())	Unique identifier for the brand record. This field is referenced by Products.BrandId in a one-to-many relationship.
Name	Nvarchar(150)	NOT NULL, recommended UNIQUE	Human-readable brand name used in product catalog display, product management, and brand lookup operations.
Description	Nvarchar(1000)	NULL	Optional longer description or marketing text for the brand. This field can be left blank when only the brand name is required.
IsActive	Bit / Boolean	NOT NULL, DEFAULT (1)	Indicates whether the brand is active and published. Inactive brands can be hidden from customer-facing catalog filters while still preserving historical product data.
CreatedAt	Datetimeoffset	NOT NULL, DEFAULT (SYSUTCDATETIME())	Timestamp showing when the brand record was created. This supports audit tracking and administrative review.

UpdatedAt	Datetimeoffset	NULL	Timestamp showing when the brand record was last modified. The value is updated when brand information changes.
-----------	----------------	------	---

Table 2.6: Structure of Brands Table

The **Brands** table supports a one-to-many relationship with the **Products** table. Each brand can be assigned to many products through **Products.BrandId**, while each product belongs to one brand. The **Name** field should be unique to avoid duplicate brand entries, and a UNIQUE index can be enforced when strict brand normalization is required. An index on **Name** can also improve brand lookup and filtering performance in the product catalog.

2.2.7 Categories Table

The **Categories** table stores product category information used for organizing and classifying products in the E-Commerce Management System.

Field Name	Data Type	Constraints	Description / Business Meaning
Id	Guid	PRIMARY KEY, NOT NULL, ValueGeneratedOnAdd (EF)	Unique identifier for each category. Uses GUID to support distributed database operations safely.
Name	Nvarchar(100)	NOT NULL	Name of the category displayed in product filtering, searching, and catalog organization features.

Table 2.7: Structure of Categories Table

The **Categories** table is responsible for maintaining the product classification structure of the system. Each category can contain multiple products through a one-to-many relationship between **Categories** and **Products**. This table supports product organization, category filtering, navigation menus, and catalog management operations.

2.2.8 Shopping Cart Table

The **Carts** table stores temporary shopping cart information associated with customers before the checkout and order creation process.

Field Name	Data Type	Constraints	Description / Business Meaning
Id	Guid	PRIMARY KEY, NOT NULL, ValueGeneratedOnAdd (EF)	Unique identifier for each shopping cart.
UserId	Guid	NOT NULL, FOREIGN KEY (Users.Id), INDEX	References the customer who owns the shopping cart.
CreatedAt	Datetime	NOT NULL	Stores the date and time when the shopping cart was created.

Table 2.8: Structure of Shopping Cart Table

The **Carts** table manages customer shopping sessions before checkout. Each customer can own an active cart containing multiple cart items, which are stored in the **CartItem**s table.

2.2.9 Cart Items Table

The **CartItem**s table stores cart ownership information and creation metadata used to connect a shopping cart with a specific user account.

Field Name	Data Type	Constraint	Description / Business Meaning
Id	Guid	Primary Key, NOT NULL, Value Generated On Add	Unique identifier for the cart. Each cart has its own code.
UserId	Guid	Foreign Key, NOT NULL, UNIQUE index	Links the cart to a specific user. The system is designed so that each user has only one cart.

CreatedAt	Datetime	NOT NULL	Time when the cart was first created. Used to track the initialization history of the cart.
-----------	----------	----------	---

Table 2.9: Structure of Cart Items Table

2.2.10 Orders Table

The **Orders** table stores checkout and order summary data for each completed customer order. It keeps payment, discount, shipping, and user ownership information as a snapshot at the time of order creation.

Field Name	Data Type	Constraint	Description / Business Meaning
Id	Guid	Primary Key, NOT NULL, ValueGeneratedOnAdd	Unique identifier of the order. The system generates a GUID for each order.
OrderDate	Datetime	NOT NULL	The date and time when the order is created. Currently assigned using UTC time in the service layer.
SubTotal	Decimal(18,2)	NOT NULL	Total product amount before discount. Calculated from quantity $\times$ unit price of all cart items.
DiscountAmount	Decimal(18,2)	NOT NULL	Discount amount applied from a voucher. Equals 0 if no voucher is used.
TotalAmount	Decimal(18,2)	NOT NULL	Final payable amount after discount. Current logic: SubTotal - DiscountAmount.

Status	Int	NOT NULL, default at application layer = Pending	Order status mapped from OrderStatus enum: Pending, Processing, Shipped, Delivered, Cancelled.
UserId	Guid	NOT NULL, Foreign Key	User who created the order. Each order belongs to an authenticated user.
ReceiverName	Nvarchar(100)	NOT NULL	Receiver's full name. Stored as a snapshot at checkout time.
ReceiverPhone	Nvarchar(15)	NOT NULL	Receiver's phone number. Snapshot copied from shipping information.
ShippingAddress	Nvarchar(500)	NOT NULL	Full shipping address stored as plain text snapshot.
PaymentMethodId	Guid	NOT NULL, Foreign Key	Selected payment method for the order.
VoucherId	Guid	NULL, Foreign Key	Applied voucher for the order. Nullable if no voucher is used.

Table 2.10: Structure of Orders Table

2.2.11 Order Details Table

The `OrderDetails` table stores each product variant line that belongs to an order. It records quantity and unit price information to preserve the exact product price at the time of purchase.

Field Name	Data Type	Constraints	Business Description / Meaning
Id	Guid	PRIMARY KEY, NOT NULL	Unique identifier for an order detail line.
OrderId	Guid	NOT NULL, FOREIGN KEY	Which order this belongs to.
ProductVariantId	Guid	NOT NULL, FOREIGN KEY	The product variant being purchased.

OrderQuantity	Int	NOT NULL, CHECK (OrderQuantity $\geq$ 1)	The quantity ordered for that variant.
UnitPrice	Decimal(18,2)	NOT NULL	Unit price at the time of order, used for price snapshotting.

Table 2.11: Structure of Order Details Table

2.2.12 Vouchers Table

The Vouchers Table is designed to store detailed information regarding discount codes and promotional campaigns applied during the checkout process within the E-Commerce Management System.

Field Name	Data Type	Constraint	Description
VoucherID	Int / Guid	Primary Key, Identity	Unique identifier for each voucher.
Code	Varchar(50)	Unique, Not Null	The alphanumeric code entered by users (e.g., SUMMER2026).
DiscountType	Int / Enum	Not Null	Type of discount: 1 = Percentage, 2 = Fixed Amount.
DiscountValue	Decimal(18,2)	Not Null	Value of the discount (e.g., 15.00 for 15% or 50000.00 VND).
MinOrderValue	Decimal(18,2)	Nullable	Minimum cart value required to activate the voucher.
MaxDiscountAmount	Decimal(18,2)	Nullable	Cap on the max discount value if using percentage type.
UsageLimit	Int	Not Null	Total number of times this voucher can be claimed globally.
UsedCount	Int	Not Null, Default 0	Current number of times the voucher has been applied.

StartDate	Datetime	Not Null	Date and time when the voucher becomes active.
EndDate	Datetime	Not Null	Expiration date and time of the voucher.
IsActive	Boolean	Default True	Quick flag to enable or disable the voucher manually.
CreatedAt	Datetime	Default GET-DATETIME()	Log timestamp of record creation.
UpdatedAt	Datetime	Nullable	Log timestamp of the last modification.

Table 2.12: Structure of Vouchers Table

2.2.13 Reviews Table

The Reviews Table systematically records customer feedback, ratings, and moderation statuses for purchased products within the system.

Field Name	Data Type	Constraint	Description
ReviewID	Int / Guid	Primary Key, Identity	Unique identifier for the product review.
ProductID	Int / Guid	Foreign Key, Not Null	References Products(ProductID).
CustomerID	Int / Guid	Foreign Key, Not Null	References Users(UserID) (Role: Customer).
OrderID	Int / Guid	Foreign Key, Nullable	References Orders(OrderID) to verify authentic purchases.
Rating	Int	Not Null (Check 1-5)	Rating value ranging strictly from 1 star to 5 stars.
Comment	Nvarchar(max)	Nullable	Text content containing the details of customer feedback.
IsApproved	Boolean	Default True / False	Moderation flag to control review visibility on product page.

CreatedAt	Datetime	Default GET-DATE()	Timestamp when the review was written.
UpdatedAt	Datetime	Nullable	Timestamp if the comment was later edited.

Table 2.13: Structure of Reviews Table

2.2.14 Support Requests Table

The Support Requests Table manages the lifecycle of customer service tickets, including categorization, priority assignments, and resolution statuses.

Field Name	Data Type	Constraint	Description
RequestID	Int / Guid	Primary Key, Identity	Unique identifier for the support ticket.
CustomerID	Int / Guid	Foreign Key, Not Null	References Users(UserID) indicating the submitter.
AssignedStaffID	Int / Guid	Foreign Key, Nullable	References Users(UserID) indicating the handler.
Subject	Nvarchar(255)	Not Null	Title summary of the complaint or question.
Category	Nvarchar(100)	Not Null	Categorization metadata (e.g., Payment, Technical, Logistics).
Description	Nvarchar(max)	Not Null	Complete structural breakdown of the problem reported.
Priority	Int / Enum	Default 1	Importance score: 0 = Low, 1 = Medium, 2 = High.
Status	Int / Enum	Default 0	Lifecycle state: 0=Pending, 1=In Progress, 2=Resolved, 3=Closed.
CreatedAt	Datetime	Default GET-DATE()	Ticket submission date.
UpdatedAt	Datetime	Nullable	Last update timestamp (status switch or new response).

Table 2.14: Structure of Support Requests Table

2.2.15 Notifications Table

The Notifications Table stores user-facing system messages, including the recipient, notification content, read status, creation timestamp, and notification type. This table supports account alerts, order updates, support updates, and other event-driven messages displayed to users in the application.

Field Name	Data Type	Constraint	Description / Business Meaning
Id	Guid	Primary Key, NOT NULL, DEFAULT NEWID()	Primary key of the notification, stored in GUID format. The system initializes a unique code each time a new notification is created.
UserId	Guid	Foreign Key, NOT NULL	ID of the user receiving the notification, such as Customer or Admin. This foreign key identifies the recipient of the notification.
Title	Nvarchar(150)	NOT NULL	Brief title or summary of the notification, such as “Order delivered” or “New voucher”. It is useful for summary display on the user interface.
Message	Nvarchar(500)	NOT NULL	Detailed content of the notification, storing up to 500 characters.

IsRead	Boolean	NOT NULL, DEFAULT 0 (False)	Flag used to track the user's notification view status. It supports displaying a new-notification indicator on the interface, where 0 means Unread and 1 means Read.
CreatedAt	Datetime	NOT NULL, DEFAULT GETUTCTIME()	Time when the notification was created and sent, stored in Coordinated Universal Time (UTC). It is used for sorting the latest notifications first.
Type	Nvarchar(50)	NULL or Default Empty	Notification category, such as "ORDER", "SYSTEM", "PROMOTION", or "SUPPORT". This supports frontend icon display or specific navigation logic when the user clicks the notification.

Table 2.15: Structure of Notifications Table

The table has a many-to-one relationship with the Users table because one user can receive multiple notifications, while each notification belongs to one specific user.

2.2.16 Payment Methods Table

The `PaymentMethods` table stores available payment options used during checkout and order creation in the E-Commerce Management System.

Field Name	Data Type	Constraints	Description / Business Meaning
------------	-----------	-------------	--------------------------------

Id	Uniqueidentifier / Guid	PRIMARY KEY, NOT NULL, application default (Guid.NewGuid())	Unique identifier for the payment method. This field is referenced by <code>Orders.PaymentMethodId</code> in a one-to-many relationship.
Name	Nvarchar(100)	NOT NULL, recommended UNIQUE	Display name for the payment method, such as <i>VNPay</i> . It is used during checkout and order display.
Description	Nvarchar(255)	NULL	Short description or instruction text for the payment method. This field can explain payment steps or provider-specific notes.
IsActive	Bit / Boolean	NOT NULL, DEFAULT (1)	Indicates whether the payment method is currently enabled for checkout. Only active payment methods should be shown to customers.
ConfigJson	Nvarchar(MAX)	NULL	Optional JSON configuration for gateway parameters, merchant identifiers, provider flags, or other external payment integration settings.
CreatedAt	Datetimeoffset	NOT NULL, DEFAULT (SYSUTCDATETIME())	Timestamp showing when the payment method record was created. This supports auditability and administrative tracking.

UpdatedAt	Datetimeoffset	NULL	Timestamp showing when the payment method record was last modified. The value is updated when payment method configuration changes.
-----------	----------------	------	---

Table 2.16: Structure of Payment Methods Table

The `PaymentMethods` table supports a one-to-many relationship with the `Orders` table through `Orders.PaymentMethodId`. Each payment method can be used by many orders, while each order records one selected payment method depending on the checkout flow. The `IsActive` field controls whether a method appears during checkout, allowing administrators to disable unavailable payment channels without deleting historical references. The `Name` field should be unique when duplicate payment method labels are not allowed, and a `UNIQUE` index can be added for stronger data consistency.

Recommended database constraints include a foreign key from `Orders.PaymentMethodId` to `PaymentMethods.Id`, a `UNIQUE` index on `Name` when required by business rules, and audit fields such as `CreatedAt` and `UpdatedAt` for payment configuration tracking.

2.2.17 Entity Relationships Overview (ERD)

This subsection provides an overview of the main entity relationships in the database design.

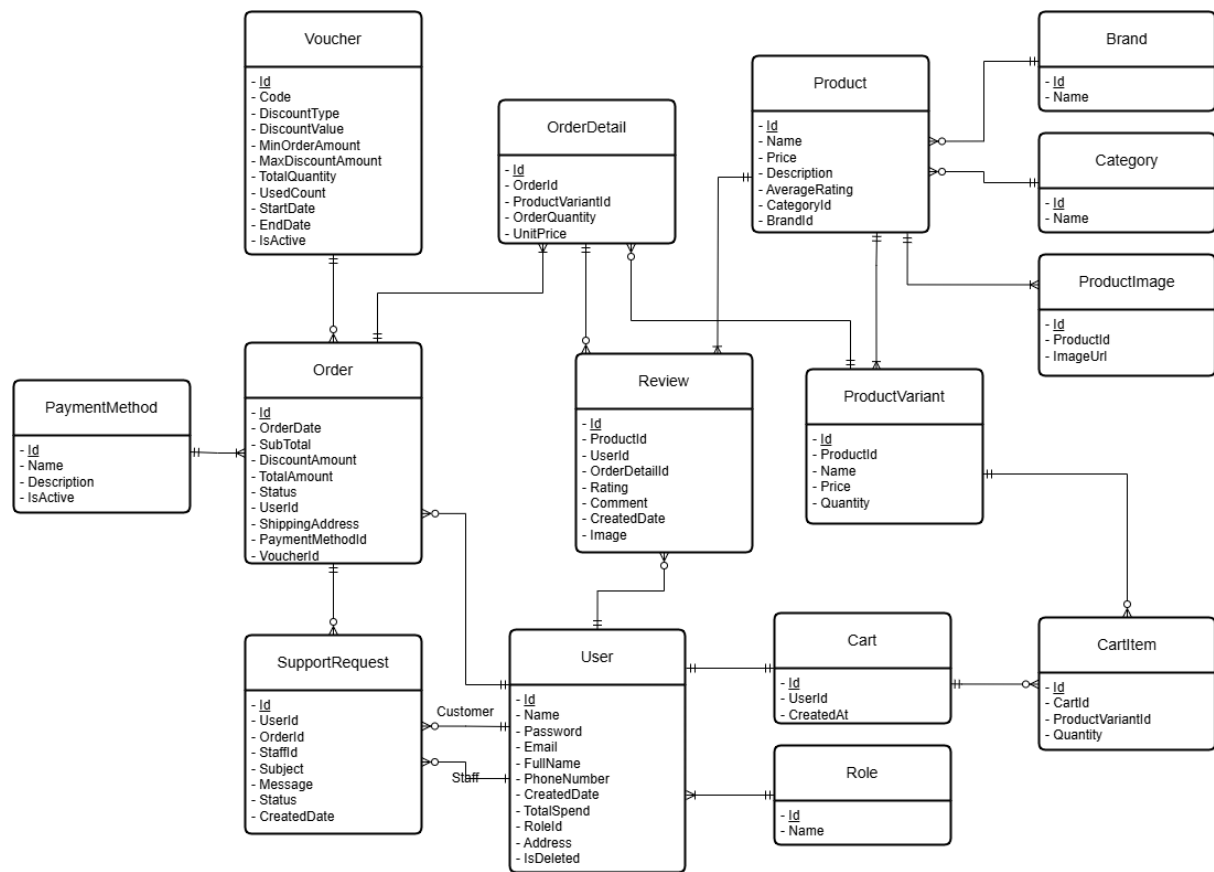


Figure 2.25: Entity Relationship Diagram

Chapter 3

Application Functions

3.1 Authentication & User Account

3.1.1 Login Screen

This screen allows users (Customers, Staff, and Administrators) to securely authenticate using their email and password to retrieve a JWT session token.

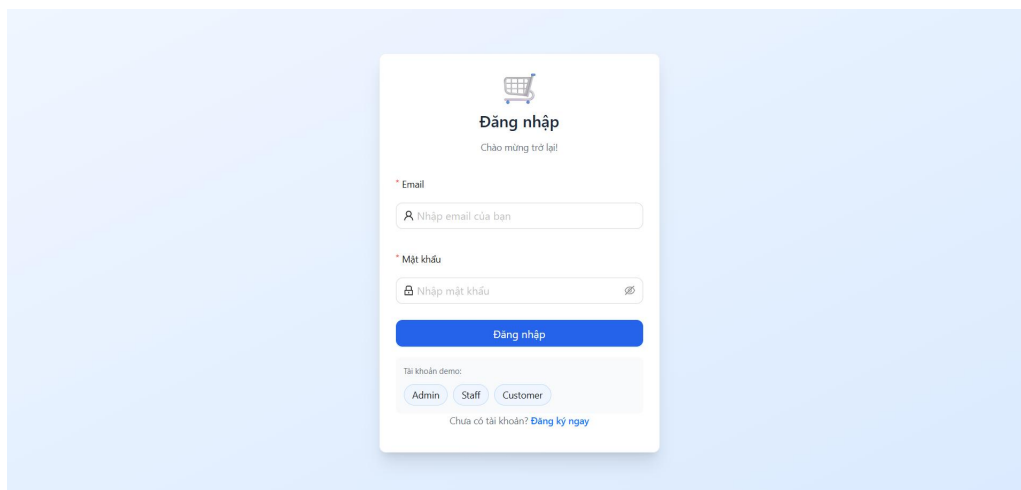


Figure 3.1: Login Screen

3.1.2 Register Screen

Enables new customers to create an account by providing basic personal information and setting up a secure, encrypted password.

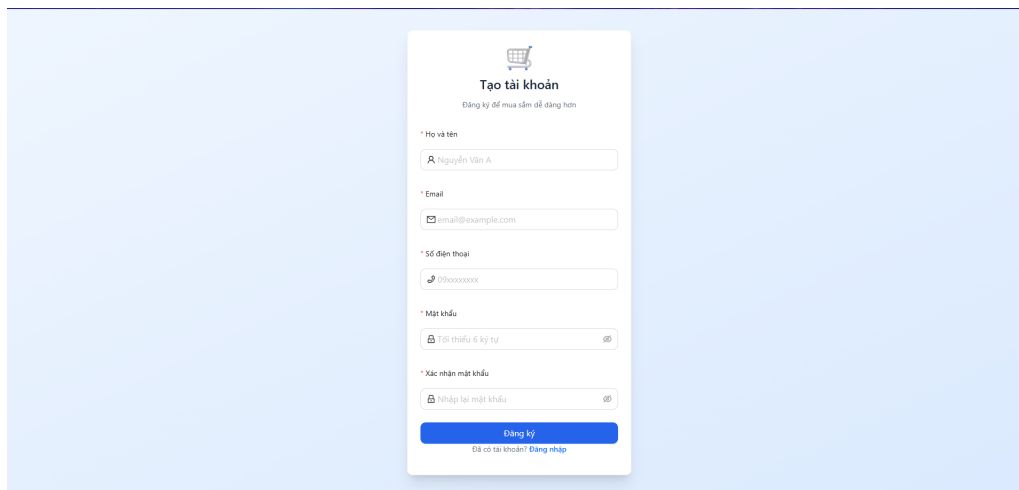
The image shows a mobile app registration screen titled "Tạo tài khoản" (Create account) with a shopping cart icon. Below the title is a subtitle "Đăng ký để mua sắm dễ dàng hơn" (Register to shop more easily). The form contains five input fields: "Họ và tên" (Name) with the value "Nguyễn Văn A", "Email" with "email@example.com", "Số điện thoại" (Phone number) with "0900000000", "Mật khẩu" (Password) with a strength indicator "Yếu thiểu 6 ký tự" (Weak, at least 6 characters), and "Xác nhận mật khẩu" (Confirm password) with a prompt "Nhập lại mật khẩu" (Re-enter password). A blue "Đăng ký" (Register) button is at the bottom, with a link "Đã có tài khoản? Đăng nhập" (Already have an account? Log in) below it.

Figure 3.2: Register Screen

3.1.3 My Account Screen

Displays the user's personal profile information, including total accumulated spending, and allows them to modify basic contact details.

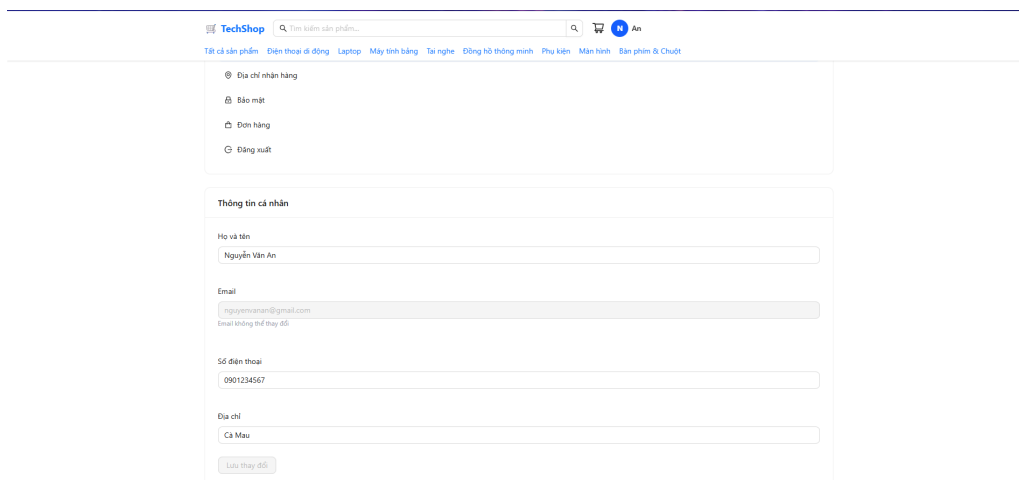
The image shows a web application's "My Account" page for "TechShop". The header includes a search bar, a shopping cart icon, and a user profile icon labeled "An". Below the header is a navigation menu with links: "Tất cả sản phẩm", "Điện thoại di động", "Laptop", "Máy tính bảng", "Tai nghe", "Đồng hồ thông minh", "Phụ kiện", "Màn hình", "Sàn phim & Chuột". The main content area has a sidebar with links: "Địa chỉ nhận hàng", "Bảo mật", "Đơn hàng", and "Đăng xuất". The "Thông tin cá nhân" (Personal Information) section contains form fields for "Họ và tên" (filled with "Nguyễn Văn An"), "Email" (filled with "nguyenvan@gmail.com" and a "Email không thể thay đổi" note), "Số điện thoại" (filled with "0901234567"), and "Địa chỉ" (filled with "Cà Mau"). A "Lưu thay đổi" (Save changes) button is at the bottom.

Figure 3.3: My Account Screen

3.1.4 Manage Shipping Addresses

Allows customers to add, edit, delete, and set default delivery addresses, including Province, District, Ward, and Street, for a faster checkout process.

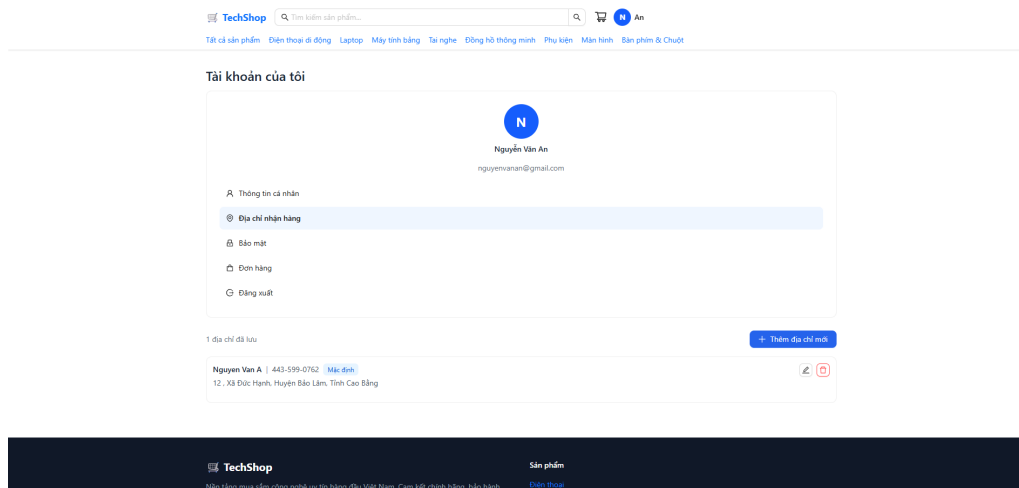


Figure 3.4: Manage Shipping Addresses

3.1.5 Change Password Function

Provides a secure interface for authenticated users to update their account credentials.

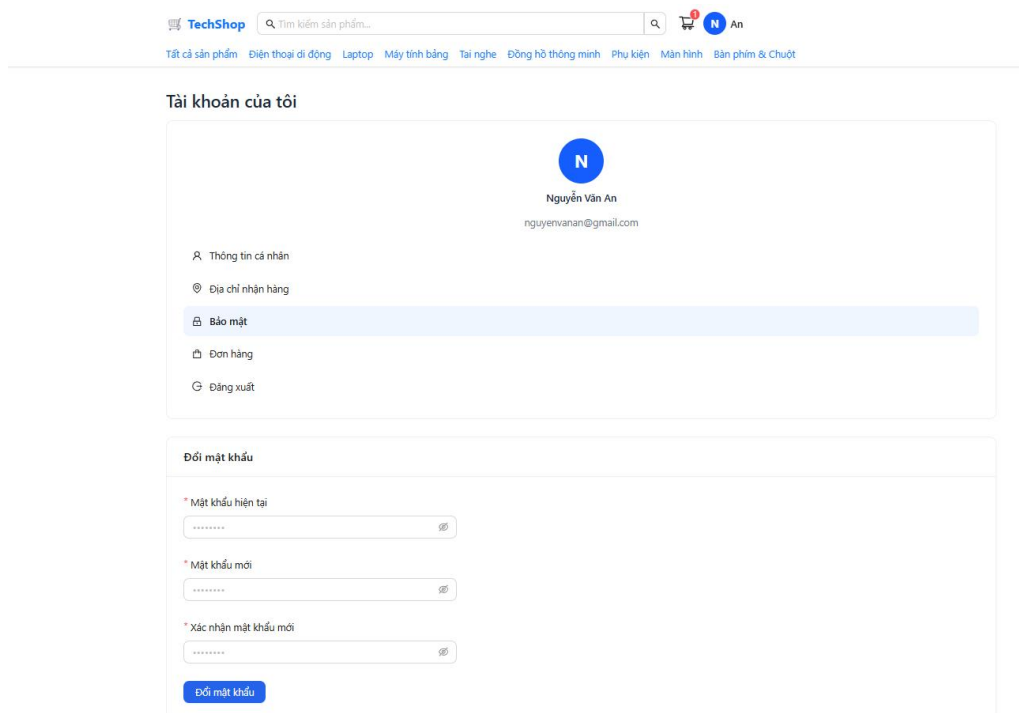


Figure 3.5: Change Password Function

3.2 Product Catalog

3.2.1 Home Screen

The main landing page displaying highlighted product categories, featured brands, and promotional banners to guide customer navigation.

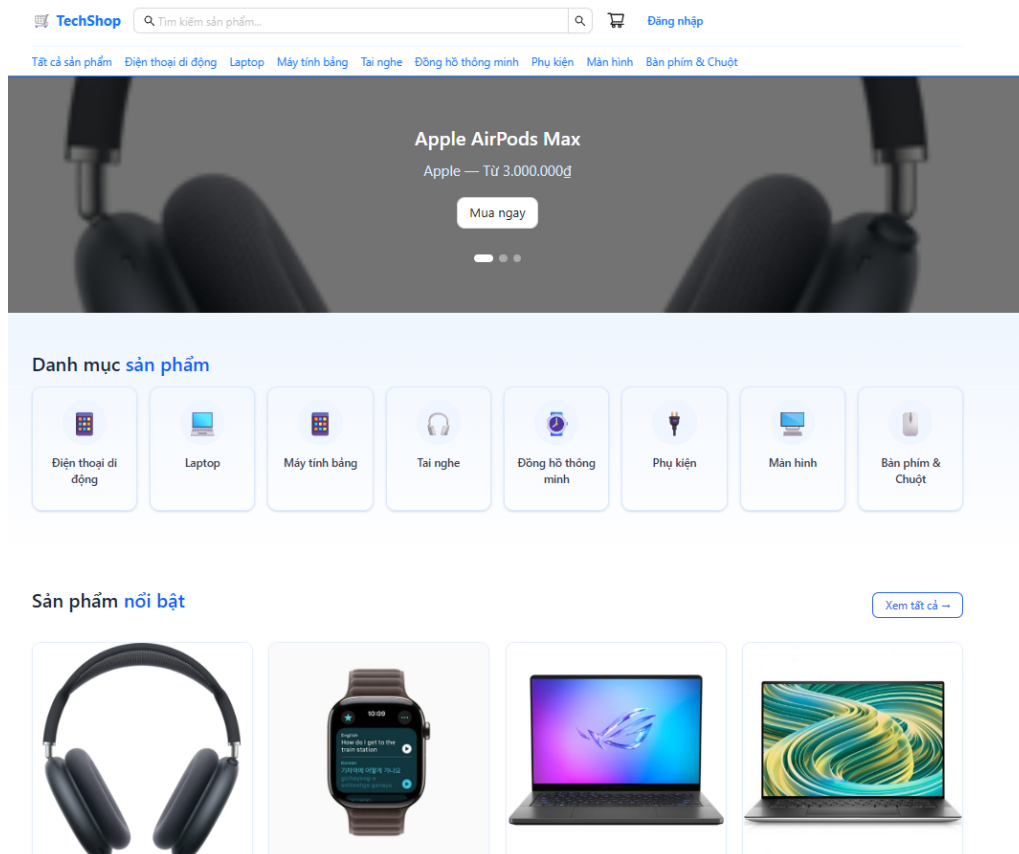


Figure 3.6: Home Screen

3.2.2 Product Listing Screen

Displays a paginated catalog of available products, allowing users to search by keywords and filter items by category or price.

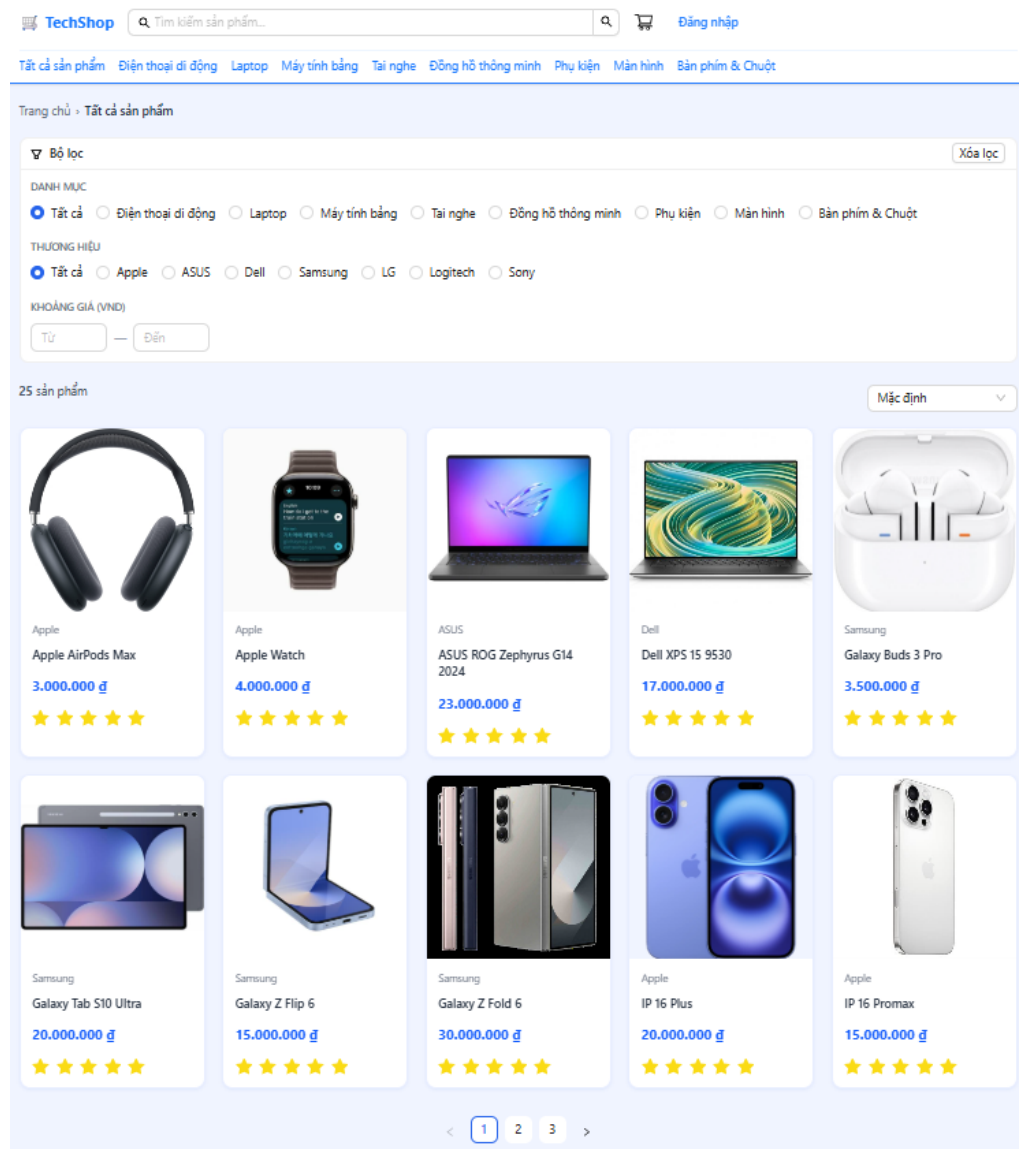


Figure 3.7: Product Listing Screen

3.2.3 Product Detail Screen

Shows comprehensive product information, including descriptions, image galleries hosted on Cloudinary, average ratings, and selectable variants such as color or RAM.

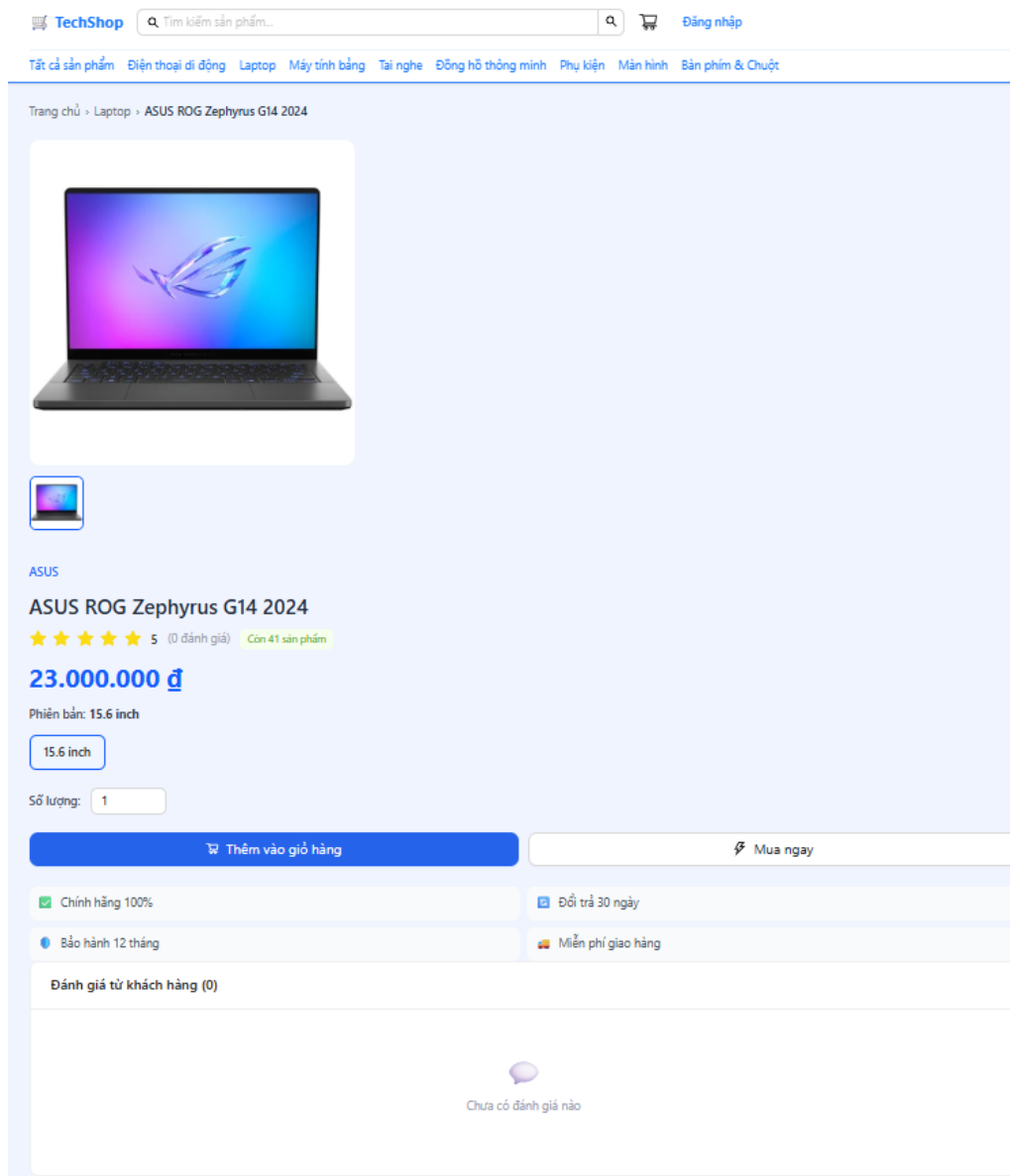


Figure 3.8: Product Detail Screen

3.3 Shopping Cart & Checkout

3.3.1 Shopping Cart Screen

Allows customers to review selected product variants, adjust quantities, dynamically calculate subtotals, and remove items before proceeding to checkout.

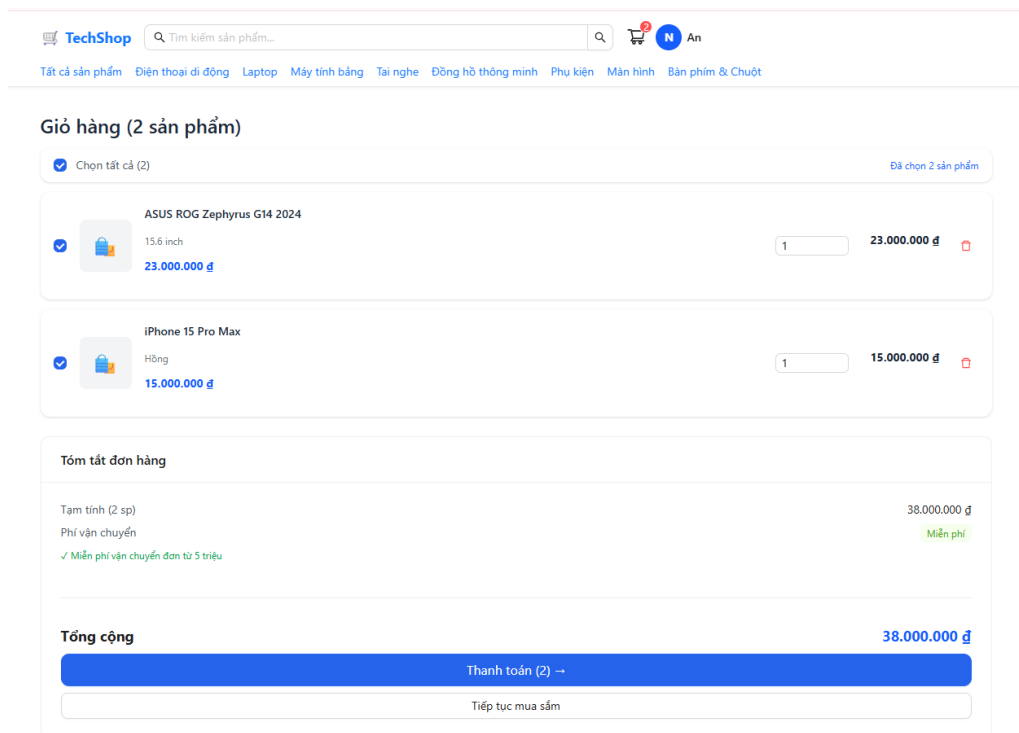


Figure 3.9: Shopping Cart Screen

3.3.2 Checkout Screen

The final confirmation step where customers verify their shipping address, apply valid promotional vouchers, and review the final calculated order total.

The screenshot displays the checkout interface of the TechShop e-commerce system. At the top, there is a navigation bar with the TechShop logo, a search bar, and icons for a shopping cart, notifications, and a user profile. Below the navigation bar, a horizontal menu lists various product categories. The main content area is titled 'Thanh toán' (Checkout) and features a progress bar with three steps: 'Thông tin giao hàng' (Shipping Information), 'Thanh toán' (Payment), and 'Xác nhận' (Confirmation). The 'Xác nhận' step is currently active, indicated by a blue circle with the number 3. The checkout details are organized into several sections: 'Xác nhận đơn hàng' (Confirm order), which includes shipping information (Nhật Linh, 0123456789, NOASXH Định Hòa A32, Phường Hiệp Thành, Thành phố Thủ Dầu Một, Tỉnh Bình Dương) and the payment method (VNPay); 'Sản phẩm (3)' (Products (3)), which lists the items being purchased: Apple Watch (Đen, 1 unit, 4,000,000 VND), Apple AirPods Max (Đen, 1 unit, 3,000,000 VND), and ASUS ROG Zephyrus G14 2024 (15.6 inch, 1 unit, 23,000,000 VND); 'Đơn hàng (3)' (Orders (3)), which provides a detailed view of each item with its image, name, specifications, and price; a 'Mã giảm giá' (Discount code) field with an 'Áp dụng' (Apply) button; a 'Tạm tính' (Subtotal) of 30,000,000 VND and a 'Vận chuyển' (Shipping) fee of 'Miễn phí' (Free); and a 'Tổng cộng' (Total) of 30,000,000 VND. At the bottom, there are two buttons: 'Quay lại' (Go back) and 'Đặt hàng' (Place order).

Sản phẩm (3)	
Apple Watch — Đen × 1	4.000.000 đ
Apple AirPods Max — Đen × 1	3.000.000 đ
ASUS ROG Zephyrus G14 2024 — 15.6 inch × 1	23.000.000 đ

Đơn hàng (3)	
Apple Watch Đen × 1	4.000.000 đ
Apple AirPods Max Đen × 1	3.000.000 đ
ASUS ROG Zephyrus G14 2024 15.6 inch × 1	23.000.000 đ

Tổng cộng	
Tạm tính	30.000.000 đ
Vận chuyển	Miễn phí
Tổng cộng	30.000.000 đ

Figure 3.10: Checkout Screen

3.3.3 Payment Selection

Presents available payment options, such as VNPay and Cash on Delivery, fetched dynamically from the system's active payment method configurations.

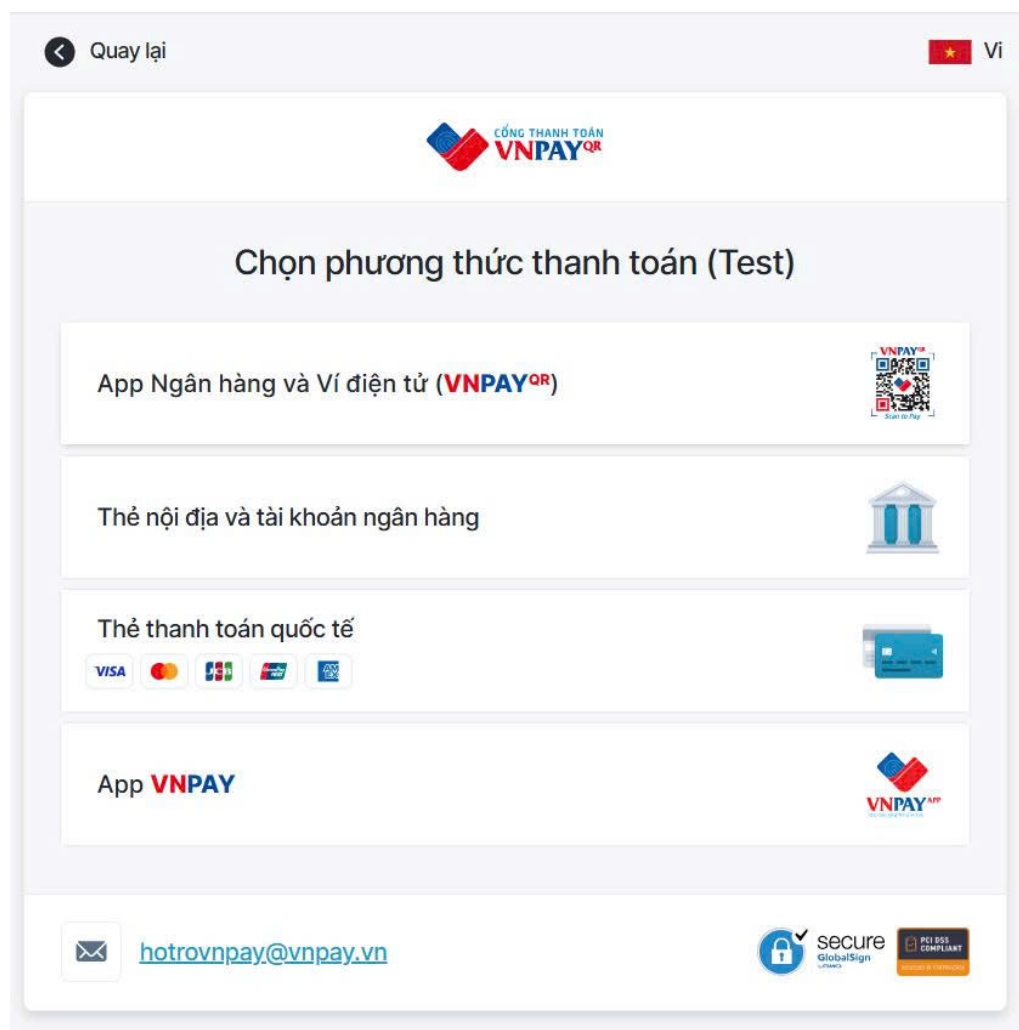


Figure 3.11: Payment Selection

3.3.4 Payment Gate

The external VNPay Sandbox interface where users securely enter their card details and OTP to process the financial transaction.

The screenshot shows the VNPay payment gateway interface for NCB bank. The page is titled "Thanh toán qua Ngân hàng NCB" (Payment via NCB Bank). On the left, under "Thông tin đơn hàng (Test)" (Test Order Information), the following details are listed: Số tiền thanh toán (Payment amount) 3.970.000 VND, Giá trị đơn hàng (Order value) 3.970.000 VND, Phí giao dịch (Transaction fee) 0 VND, Mã đơn hàng (Order code) 71412c7e-fb4e-45b7-b128-5f175a25d576, and Nhà cung cấp (Provider) https://vnshop.vn/. On the right, under "Thẻ nội địa" (Local Card), the card details are entered: Số thẻ (Card number) *****2198, Tên chủ thẻ (Cardholder name) NGUYEN VAN A, and Ngày phát hành (Issuance date) 07/15. There is a field for Mã khuyến mại (Promotional code) with a button "Chọn hoặc nhập mã" (Select or enter code). Below this is a link "Điều kiện sử dụng dịch vụ" (Terms of service). At the bottom, there are two buttons: "Hủy thanh toán" (Cancel payment) and "Tiếp tục" (Continue). The footer includes the email hotrovnpay@vnpay.vn, security logos for Secure Gateway and NCB, and the text "Phát triển bởi VNPAY © 2026".

Figure 3.12: Payment Gate

3.3.5 Payment Success Notification

Displays the definitive transaction result based on the `vnp_ResponseCode` returned from the payment gateway.

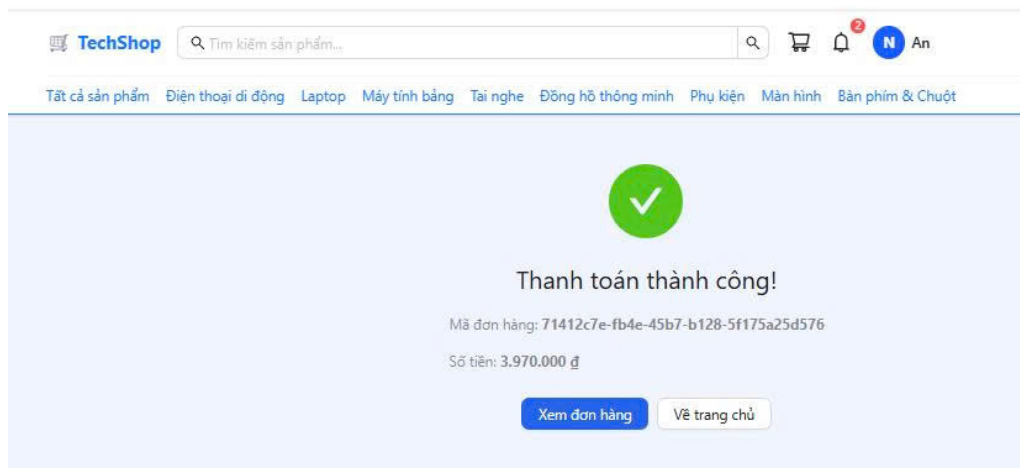


Figure 3.13: Payment Success Notification

3.4 Order Management

3.4.1 My Orders Screen

Allows customers to track their current and past orders, view order details, and monitor the processing status such as Pending, Shipped, or Delivered.

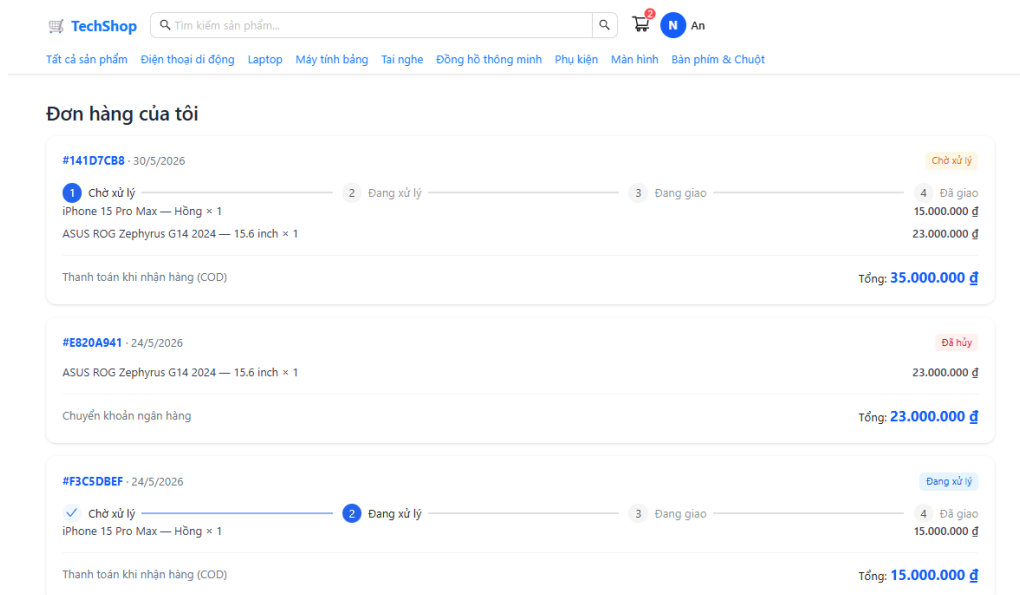


Figure 3.14: My Orders Screen

3.4.2 Admin Order Management Screen

Provides administrators with a comprehensive overview of all system orders, enabling them to monitor overall platform sales and fulfillment statuses.

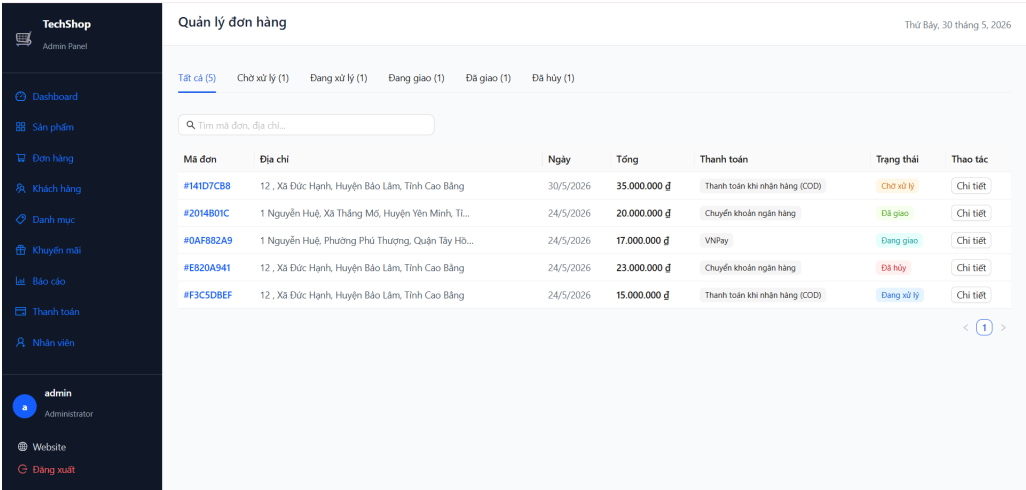


Figure 3.15: Admin Order Management Screen

3.4.3 Staff Order Processing Screen

The operational interface for staff members to review incoming orders, verify payment states, and progressively update the order lifecycle.

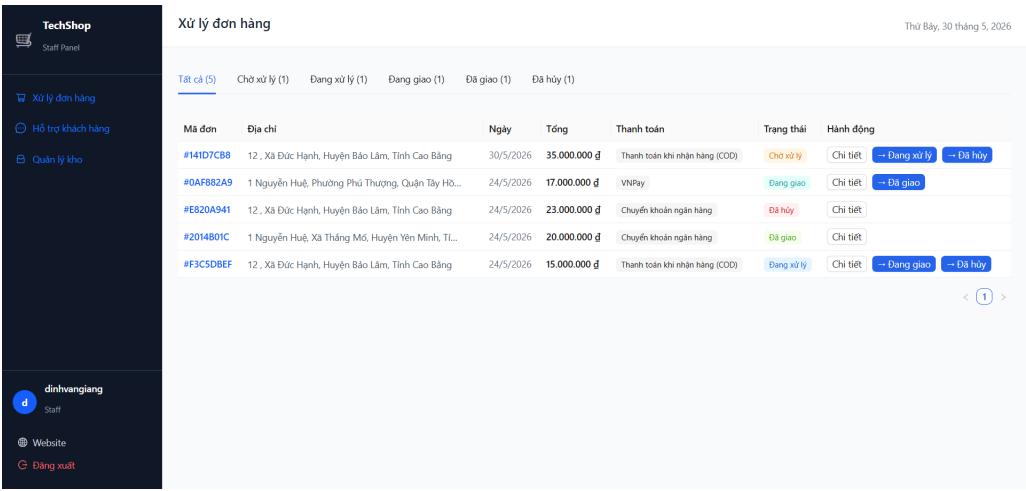


Figure 3.16: Staff Order Processing Screen

3.5 Customer Support

3.5.1 Support Request Screen

Enables customers to submit inquiry tickets or report issues by providing a subject, selecting a category, and writing a detailed message.

TechShop

Tìm kiếm sản phẩm...

Tất cả sản phẩm Điện thoại di động Laptop Máy tính bảng Tai nghe Đồng hồ thông minh Phụ kiện Màn hình Sản phẩm & Chuột

Hỗ trợ khách hàng

Gửi yêu cầu mới Lịch sử hỗ trợ Câu hỏi thường gặp

* Chủ đề *

Mô tả ngắn vấn đề của bạn

* Nội dung chi tiết *

Mô tả chi tiết vấn đề bạn đang gặp phải...

Gửi yêu cầu

TechShop

Nền tảng mua sắm công nghệ uy tín hàng đầu Việt Nam, Cam kết chính hãng, bảo hành đầy đủ.

Sản phẩm

Điện thoại Laptop Tai nghe Đồng hồ thông minh Phụ kiện Màn hình Sản phẩm & Chuột

Hỗ trợ

Trung tâm hỗ trợ

Liên hệ

123 Nguyễn Huệ, Q1, TP.HCM

Figure 3.17: Support Request Screen

3.5.2 View Support Tickets Screen

Allows customers to view the history and monitor the current status, such as Pending, In Progress, or Resolved, of their submitted support requests.

TechShop

Tìm kiếm sản phẩm...

Tất cả sản phẩm Điện thoại di động Laptop Máy tính bảng Tai nghe Đồng hồ thông minh Phụ kiện Màn hình Sản phẩm & Chuột

Hỗ trợ khách hàng

Gửi yêu cầu mới **Lịch sử hỗ trợ** Câu hỏi thường gặp

Vấn Đề

30/5/2026

Nội Dung Của Vấn Đề

Chờ xử lý

TechShop

Nền tảng mua sắm công nghệ uy tín hàng đầu Việt Nam, Cam kết chính hãng, bảo hành đầy đủ.

Sản phẩm

Điện thoại Laptop Tai nghe Đồng hồ thông minh Phụ kiện Màn hình Sản phẩm & Chuột

Hỗ trợ

Trung tâm hỗ trợ

Liên hệ

123 Nguyễn Huệ, Q1, TP.HCM

1800 1234 (miễn phí)

support@techshop.vn

8:00 - 22:00 hàng ngày

© 2024 TechShop. All rights reserved.

Figure 3.18: View Support Tickets Screen

3.5.3 Staff Support Management Screen

The administrative interface for staff to read customer complaints, assign responsibilities, and update the resolution status of tickets.

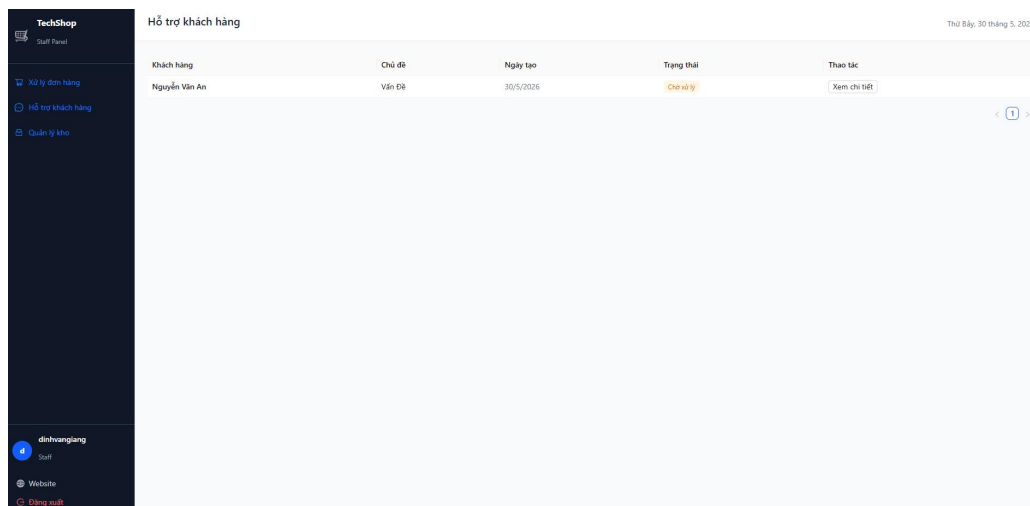


Figure 3.19: Staff Support Management Screen

3.6 Administration

3.6.1 Admin Dashboard Screen

A high-level analytics interface providing a quick summary of total revenue, recent order counts, and top-spending customers.

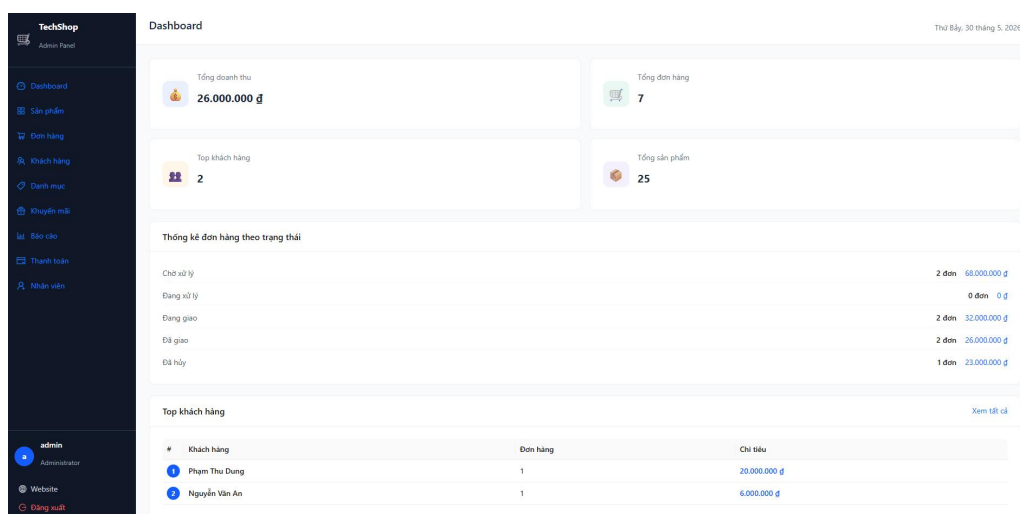


Figure 3.20: Admin Dashboard Screen

3.6.2 Product Management Screen

Allows administrators to perform CRUD operations on the product catalog, manage variants, and upload product images directly to Cloudinary.

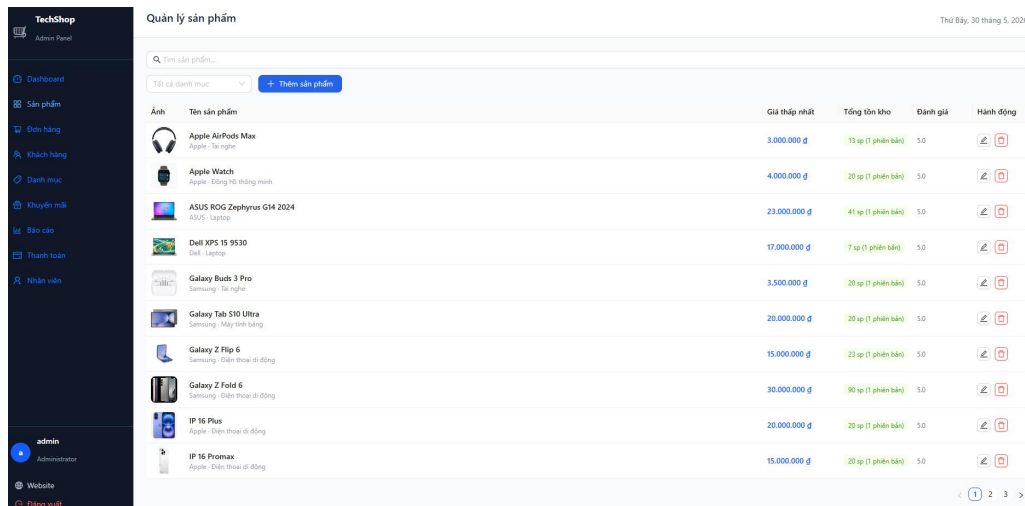


Figure 3.21: Product Management Screen

3.6.3 Customer Management Screen

Displays a list of registered customer accounts, allowing administrators to view contact details and oversee user activity.

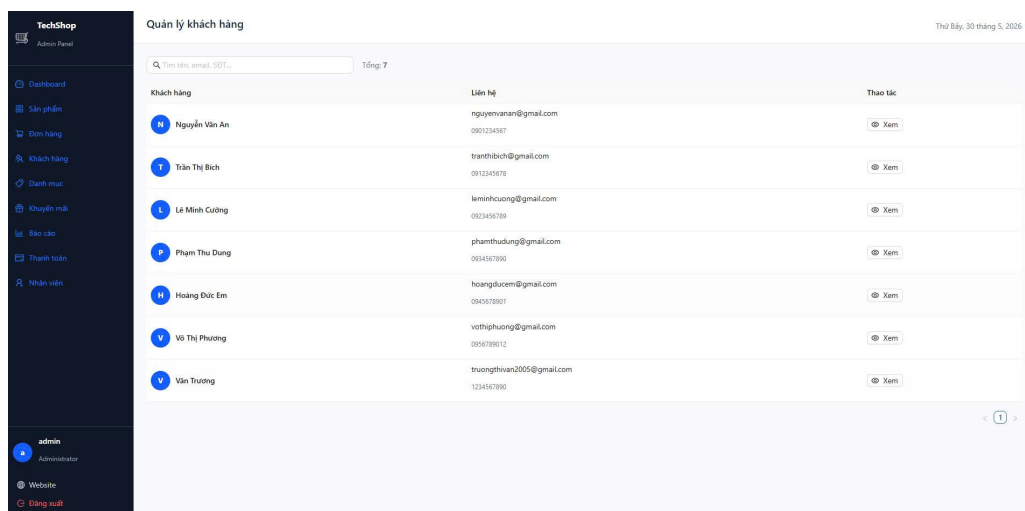


Figure 3.22: Customer Management Screen

3.6.4 Category Management Screen

An interface for creating, updating, and soft-deleting product categories to maintain a logically organized catalog hierarchy.

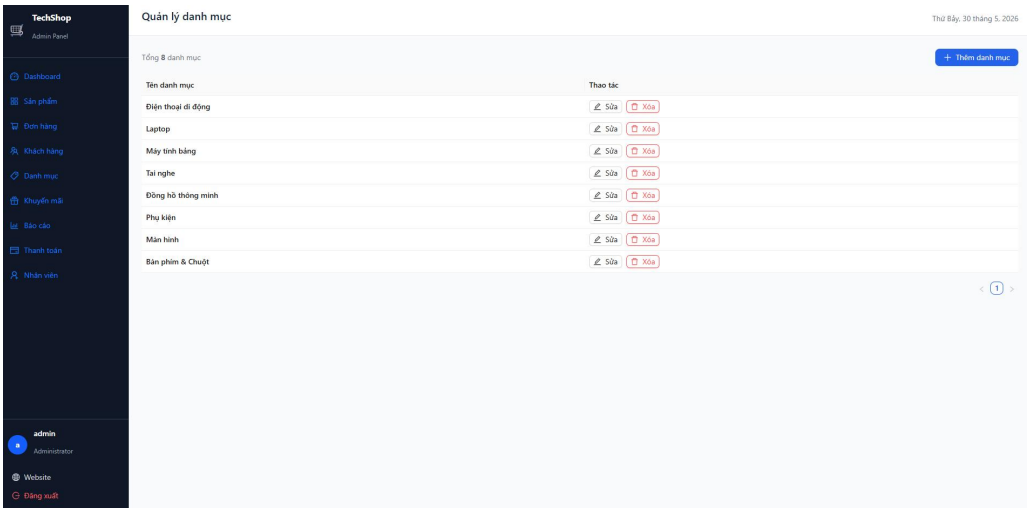


Figure 3.23: Category Management Screen

3.6.5 Promotion Management Screen

Allows administrators to configure discount vouchers, set validity dates, specify minimum order amounts, and control global usage limits.

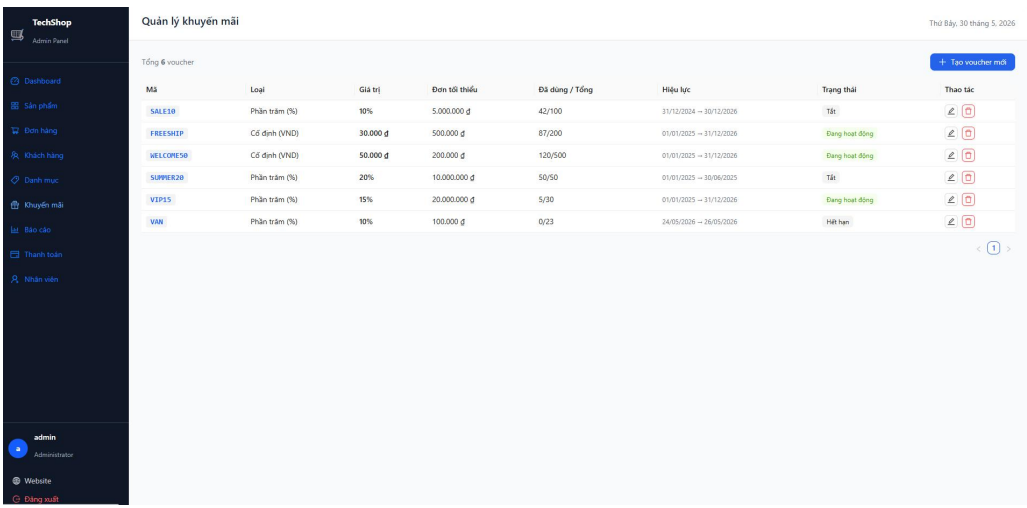


Figure 3.24: Promotion Management Screen

3.6.6 Payment Method Management Screen

Provides tools to activate, deactivate, or configure integration settings for available payment gateways like VNPay.

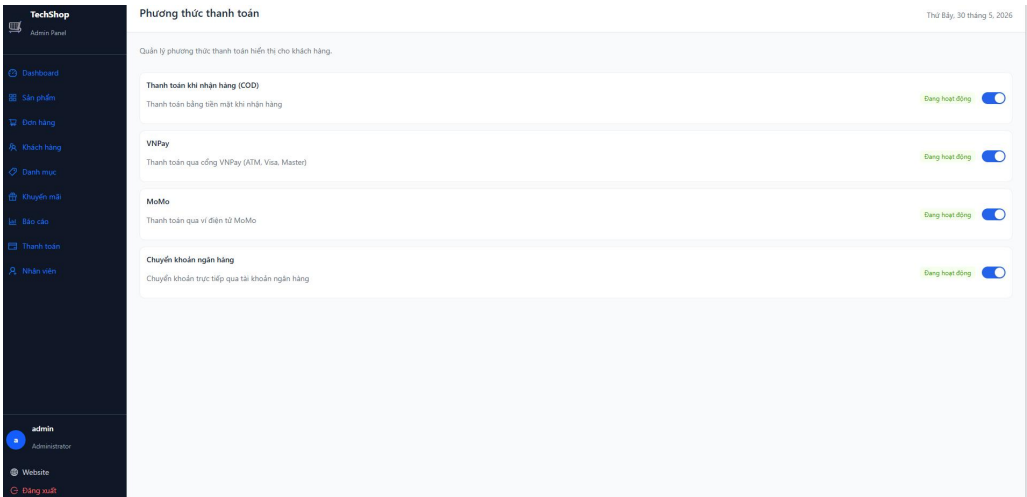


Figure 3.25: Payment Method Management Screen

3.6.7 Staff Management Screen

Allows super-administrators to securely provision and manage internal staff accounts for operational delegation.

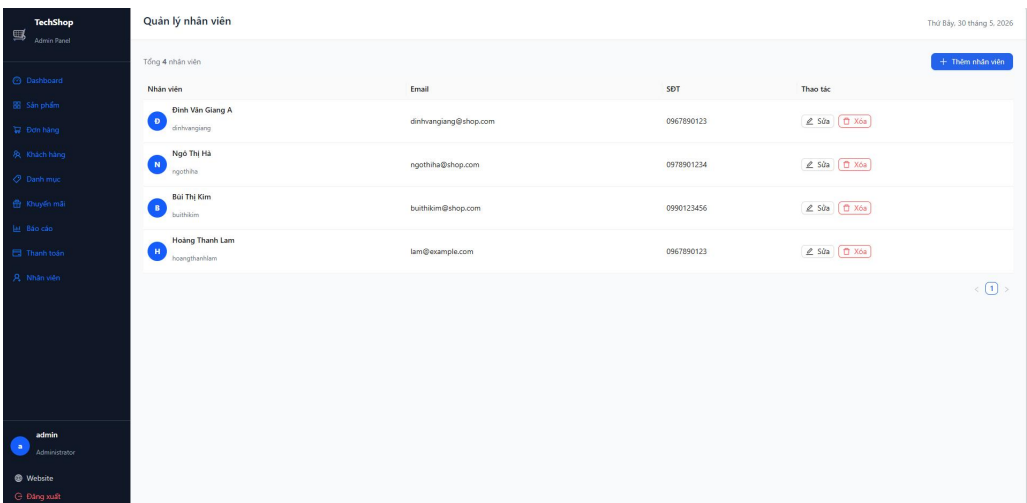


Figure 3.26: Staff Management Screen

3.7 Inventory Management

3.7.1 Inventory Management Screen

Provides a centralized view of product variant stock quantities, helping staff monitor availability and plan restocks effectively.

Ảnh	Tên sản phẩm	Giá thấp nhất	Tổng tồn kho	Đánh giá	Hành động
	Apple AirPods Max Apple - Tai nghe	3.000.000 đ	13 sp (1 phiên bản)	5,0	
	Apple Watch Apple - Đồng hồ thông minh	4.000.000 đ	20 sp (1 phiên bản)	5,0	
	ASUS ROG Zephyrus G14 2024 ASUS - Laptop	23.000.000 đ	41 sp (1 phiên bản)	5,0	
	Dell XPS 15 9530 Dell - Laptop	17.000.000 đ	11 sp (1 phiên bản)	5,0	
	Galaxy Buds 3 Pro Samsung - Tai nghe	3.500.000 đ	20 sp (1 phiên bản)	5,0	
	Galaxy Tab S10 Ultra Samsung - Máy tính bảng	20.000.000 đ	20 sp (1 phiên bản)	5,0	
	Galaxy Z Flip 6 Samsung - Điện thoại di động	15.000.000 đ	23 sp (1 phiên bản)	5,0	
	Galaxy Z Fold 6 Samsung - Điện thoại di động	30.000.000 đ	90 sp (1 phiên bản)	5,0	
	IP 16 Plus Apple - Điện thoại di động	20.000.000 đ	20 sp (1 phiên bản)	5,0	
	IP 16 Promax Apple - Điện thoại di động	15.000.000 đ	20 sp (1 phiên bản)	5,0	

Figure 3.27: Inventory Management Screen

3.8 Reports & Analytics

3.8.1 Revenue Report Screen

Generates financial summaries within specific date ranges to help administrators track total sales and business growth over time.

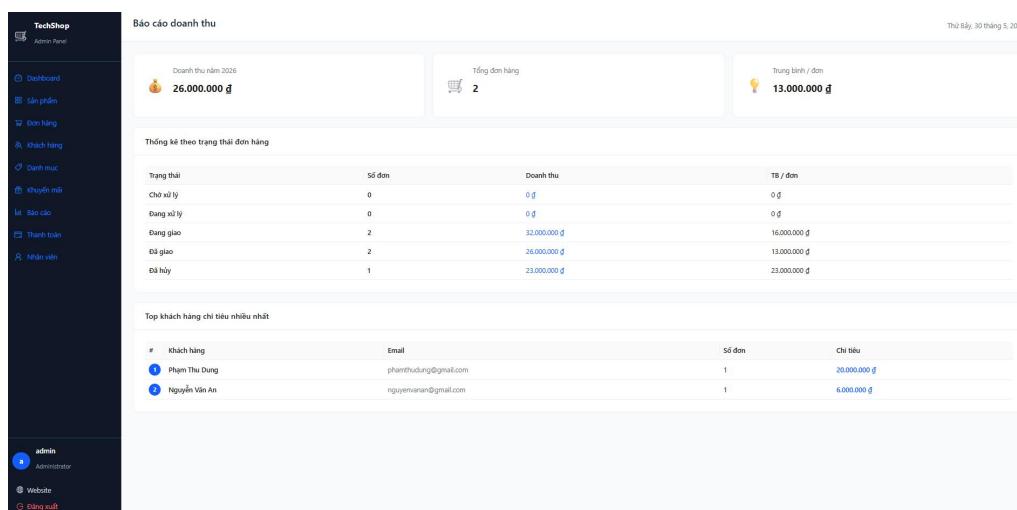


Figure 3.28: Revenue Report Screen

3.8.2 Order Statistics Screen

Displays analytical data on order volumes categorized by their current lifecycle status, such as delivered versus cancelled orders.

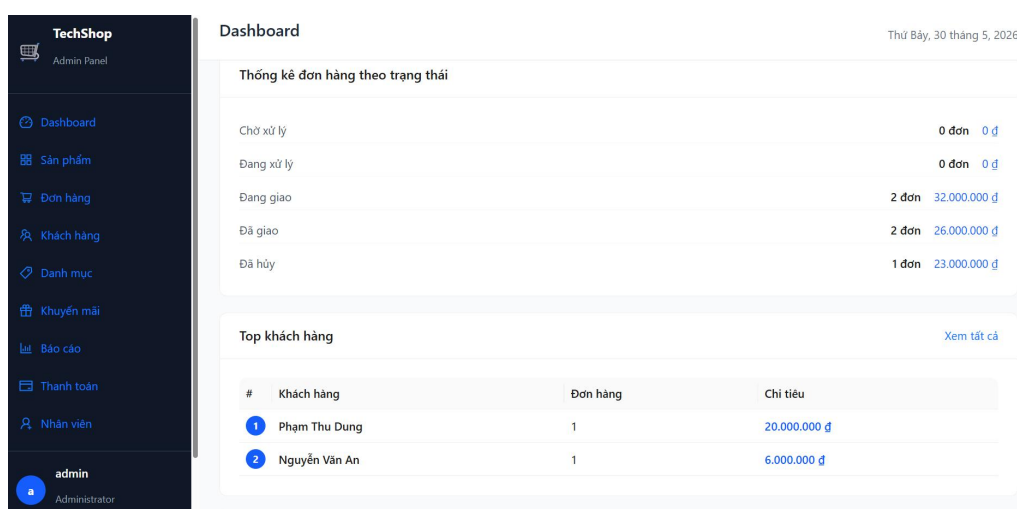


Figure 3.29: Order Statistics Screen

3.8.3 Customer Statistics Screen

Highlights the platform's most valuable users based on their total historical spending, aiding in targeted marketing and loyalty recognition.

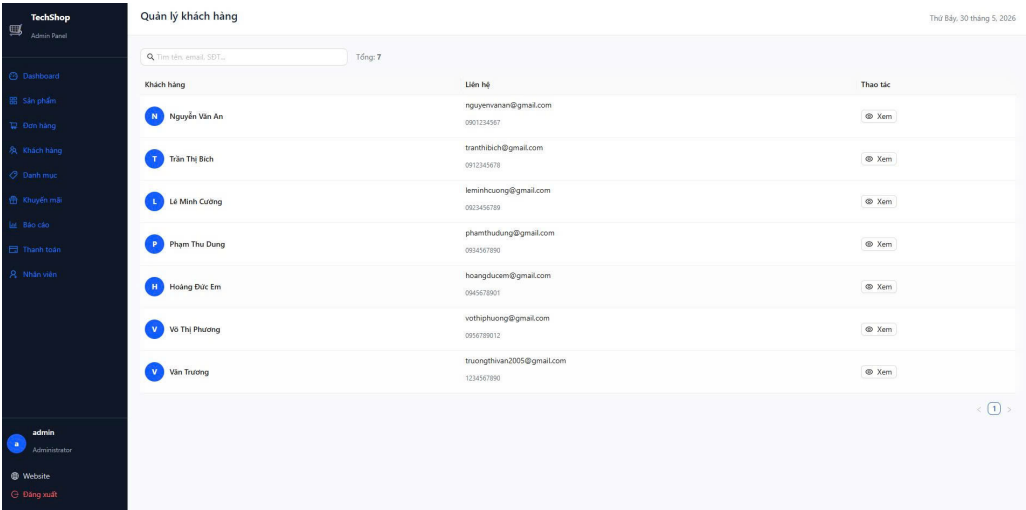


Figure 3.30: Customer Statistics Screen

Chapter 4

Related Work

The landscape of digital commerce infrastructure has undergone a profound paradigm shift over the past two decades, evolving from bespoke, rigidly monolithic on-premise deployments to highly distributed, cloud-native software ecosystems. Early implementations of e-commerce platforms demanded massive upfront capital expenditures, dedicated on-site server clusters, and intensive ongoing technical maintenance, which inherently restricted digital retail to massively capitalized enterprises. However, as global internet penetration deepened and consumer expectations for frictionless, instantaneous digital retail accelerated, the market rapidly transitioned toward standardized software models. Today, the operational viability, financial efficiency, and competitive agility of an online retail enterprise are inextricably linked to the underlying architecture of its chosen e-commerce platform. In contemporary digital markets, where industry data reveals that fewer than 2% of global online shopping visits culminate in a completed purchase, the importance of minimizing checkout friction, accelerating page load speeds, and implementing robust localization features has become paramount.

Academic literature and industry analyses concerning e-commerce architectures frequently classify these systems based on their hosting models, extensibility paradigms, and target merchant volume. These systems generally fall into three distinct architectural classifications: Fully managed Software-as-a-Service (SaaS), open-source self-hosted solutions, and enterprise-grade Platform-as-a-Service (PaaS) or highly integrated cloud-hosted frameworks. Each classification represents a specific series of systemic trade-offs between speed-to-market, long-term scalability, Total Cost of Ownership (TCO), and absolute architectural autonomy. Cloud-based platforms prioritize rapid deployment, managed security, and high availability by abstracting infrastructure complexities away from the merchant, whereas open-source frameworks provide unbounded code-level customization at the explicit expense of drastically increased technical complexity, security liability, and continuous developer dependency.

The concept of Total Cost of Ownership has become the central, overriding metric by which these platforms are evaluated in modern commerce literature. TCO fundamentally extends far beyond the base subscription or initial licensing fees; it comprehensively encompasses implementation burdens, mandatory third-party application subscriptions, transaction processing tariffs, specialized developer retainers, hosting overhead, and the

perpetual cost of maintaining Payment Card Industry Data Security Standard (PCI DSS) compliance. For instance, a platform may offer a negligible or even non-existent upfront licensing fee, yet impose severe long-term operational costs through mandatory revenue-sharing agreements, high-friction development environments that protract feature deployment, or cascading plugin conflicts. Conversely, multi-tenant SaaS platforms may offer highly predictable baseline infrastructure costs but introduce a phenomenon documented as "app-subscription creep," wherein merchants must rely on dozens of independently billed third-party integrations simply to achieve functional parity with monolithic enterprise systems.

Furthermore, the simultaneous rise of omnichannel retail and globalization has fundamentally redefined the baseline of what constitutes a "core" e-commerce feature. Modern digital enterprises are no longer confined to single-region Direct-to-Consumer (DTC) models; they require native, highly performant support for multi-currency transactions, complex algorithmic tax and duty calculations across diverse geopolitical jurisdictions, advanced inventory routing across disparate fulfillment centers, and seamless Business-to-Business (B2B) capabilities integrated directly alongside traditional retail storefronts. Platforms that inherently lack these native capabilities force merchants to construct fragile, interconnected architectures composed of stacked plugins and middleware. This patchwork approach inherently increases operational risk, network latency, and critical security vulnerabilities, often precipitating the need for a total systemic replatforming as the business scales. The average replatforming project in the current market spans approximately nine months, representing a massive allocation of resources and a significant interruption to standard business operations.

In evaluating the related work and prevailing paradigms within this domain, it is analytically essential to deeply examine the five most iconic e-commerce systems currently dictating the global market share: Shopify, WooCommerce, BigCommerce, Adobe Commerce (formerly Magento), and Salesforce Commerce Cloud. By systematically deconstructing their operational philosophies, underlying technical foundations, and economic models, a comprehensive empirical baseline can be established. This baseline is subsequently utilized to critically evaluate the architectural superiority, economic efficiency, and competitive advantages of the proposed custom E-Commerce Management System. The proposed system deliberately eschews the compromises of commercial off-the-shelf platforms by leveraging modern decoupled microservices, advanced Unit Testing protocols, high-performance GraphQL APIs, and state-of-the-art transformer-based behavioral analytics.

4.1 Existing E-Commerce Platforms

An exhaustive evaluation of the contemporary e-commerce market necessitates a highly detailed, platform-specific examination of the systems that currently dictate industry standards and best practices. The following five platforms: Shopify, WooCommerce, BigCommerce, Adobe Commerce, and Salesforce Commerce Cloud will cover the vast majority of medium to enterprise-scale retail deployments worldwide. They each embody radically divergent philosophies regarding digital commerce infrastructure, data sovereignty, and technical scalability.

4.1.1 Shopify and Shopify Plus

Shopify has unequivocally cemented its position as the globally dominant e-commerce platform by championing a fully hosted Software-as-a-Service (SaaS) model meticulously designed for speed, reliability, and unparalleled ease of use. As of current market evaluations, Shopify powers roughly one-third of the entire hosted commerce sector. Architecturally, Shopify operates on a proprietary, globally distributed multi-tenant cloud infrastructure that entirely abstracts server management, database scaling, caching, and security patching away from the merchant. It natively guarantees Level 1 PCI DSS compliance across all tiers and utilizes a proprietary global Content Delivery Network (CDN) to ensure highly stable performance and uptime during high-concurrency traffic events, such as algorithmic product drops or seasonal sales.

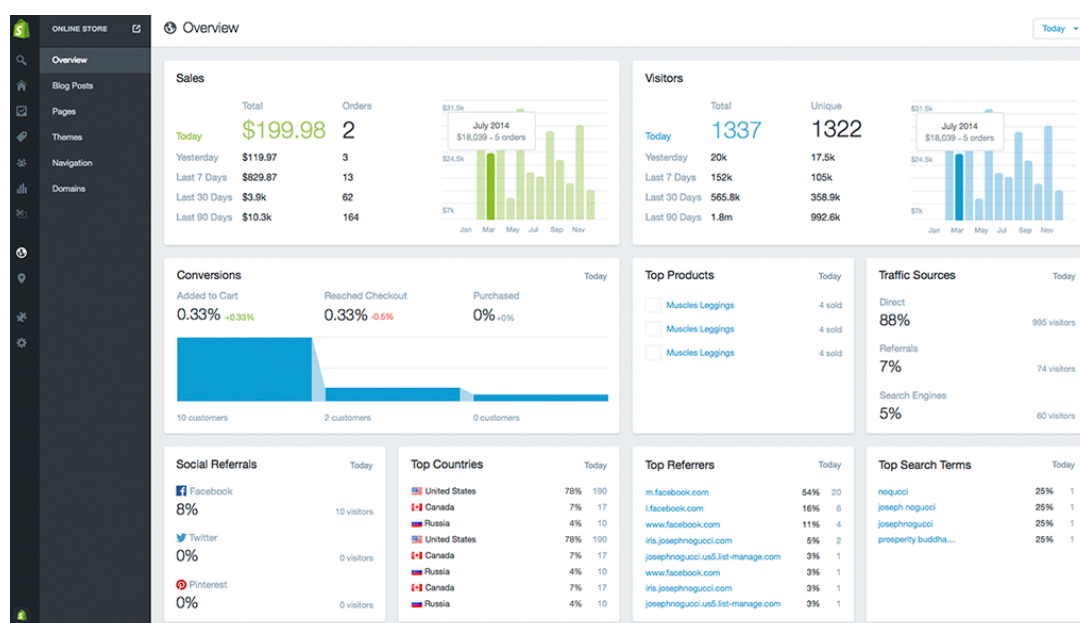


Figure 4.1: The Shopify admin overview dashboard.

Shopify's overarching market strategy relies heavily on the democratization of commerce technology via its vast application ecosystem. The platform boasts a centralized app marketplace containing over 8,000 highly vetted integrations, allowing merchants to bolt on functionality spanning from complex automated marketing and SMS recovery to sophisticated inventory forecasting and augmented reality displays, largely without requiring custom code. For entry-level and mid-market merchants, Shopify offers highly accessible standard plans ranging from \$39 to \$399 per month, which fundamentally commoditizes standard e-commerce functionality. However, to serve large-scale global enterprises processing tens to hundreds of millions in Gross Merchandise Value (GMV), Shopify offers Shopify Plus. Starting at a baseline of approximately \$2,300 per month and scaling upward based on volume, Plus introduces critical enterprise features such as checkout extensibility, API-driven composable commerce via its React-based Hydrogen framework, and native wholesale B2B channels that allow unified management of DTC and B2B orders within a single administrative backend. Furthermore, features like Shopify Markets allow Plus merchants to manage localized storefronts across more than 150 countries from a centralized instance.

Despite its overwhelming market dominance and operational reliability, Shopify presents distinct structural and economic limitations that become highly pronounced at the enterprise scale. Technologically, its rigid proprietary data model enforces strict URL taxonomies (e.g., mandating `/products/` and `/collections/` in URL routing), which inherently limits absolute Search Engine Optimization (SEO) control for brands with complex legacy content architectures. Economically, Shopify utilizes a dual-revenue stream model that heavily penalizes merchants who do not adopt its proprietary payment gateway, Shopify Payments. Merchants utilizing third-party payment processors for specific regional or strategic reasons are subject to punitive transaction fees ranging from 0.5% to 2.0% on standard tiers, and a negotiated rate of roughly 0.2% on the Plus tier. At enterprise GMV scales, these seemingly fractional percentages compound into massive, unavoidable operational liabilities, frequently driving high-volume, low-margin brands to reconsider their platform dependency.

4.1.2 WooCommerce

WooCommerce represents the absolute antithesis of the closed-SaaS model championed by Shopify. Built as a fundamentally open-source e-commerce extension integrated directly on top of the WordPress Content Management System (CMS), WooCommerce is the preeminent choice for organizations that demand total data ownership, unbounded code-level customization, and the seamless, organic integration of content and commerce. Because it operates on a self-hosted architecture, merchants maintain direct, unrestricted access to the underlying PHP codebase and the MySQL relational database.

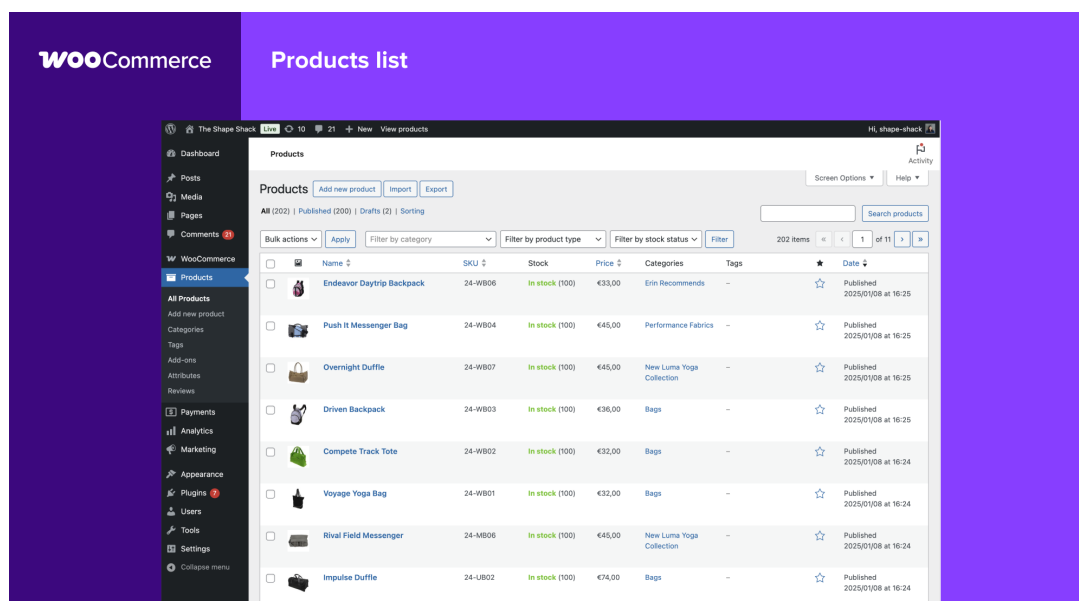


Figure 4.2: The WooCommerce product management interface integrated within the WordPress CMS.

The primary theoretical advantage of WooCommerce lies in its extraordinary architectural flexibility and cost-flexibility at the entry level. The core WooCommerce plugin is entirely free to license, and its surrounding open-source ecosystem comprises over 59,000 plugins that cover nearly every conceivable digital retail use case, from esoteric regional payment gateways to highly specialized shipping logic. For brands whose business models rely heavily on rich organic content, unconstrained editorial control, and advanced SEO mapping, WooCommerce offers unparalleled freedom. It allows for complete control over URL structures, deeply nested product categories, and integrated blogging that SaaS platforms simply cannot replicate natively. Furthermore, as an open-source platform, it imposes absolutely no proprietary transaction fees or arbitrary platform constraints on sales volume.

However, the "free" nature of WooCommerce is an economic construct that rarely applies at commercial scale. The operational burden of a self-hosted, plugin-dependent system means that the merchant organization is entirely responsible for provisioning its own high-performance hosting architecture, implementing continuous security patches, optimizing server-level caching, and absorbing the full legal and technical liabilities associated with PCI compliance. As a WooCommerce storefront scales past \$500,000 in annual GMV, the inherent fragility of a plugin-heavy architecture becomes acutely apparent. Premium plugin conflicts, database bloat resulting from inefficient query structures, and the necessity for continuous, highly skilled developer intervention drastically elevate the Total Cost of Ownership. Industry analyses consistently demonstrate that maintaining a performant, enterprise-grade WooCommerce stack for a business processing over \$2 million in GMV routinely requires an ongoing developmental and managed hosting budget rang-

ing from \$40,000 to \$150,000 annually, a figure that frequently exceeds the baseline cost of managed SaaS alternatives like Shopify Plus or BigCommerce.

4.1.3 BigCommerce

BigCommerce occupies a highly strategic middle ground in the e-commerce platform spectrum, heavily marketing itself as an "Open SaaS" solution tailored specifically for rapidly growing mid-market brands and established enterprise businesses. Similar to Shopify, BigCommerce is a fully hosted, cloud-based platform that entirely abstracts server maintenance, security patching, and uptime management away from the merchant. However, its architectural philosophy diverges significantly by embedding a far more robust, comprehensive suite of native functionalities directly into the core platform, thereby actively reducing merchant reliance on paid third-party applications.

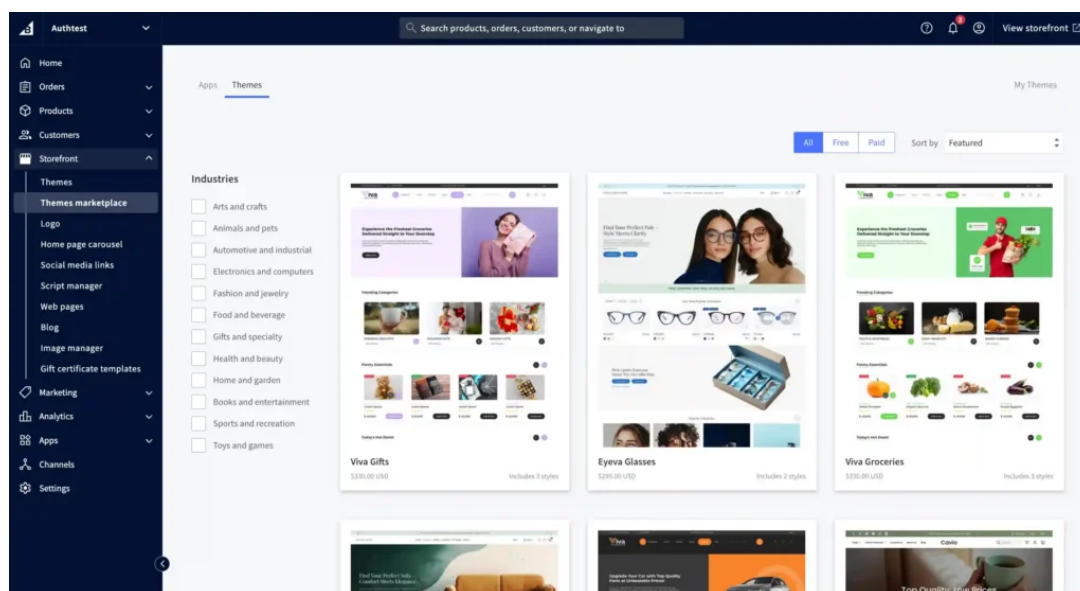


Figure 4.3: The BigCommerce Theme Marketplace.

BigCommerce distinguishes itself through superior API availability (supporting up to 400 API calls per second on enterprise tiers) and a highly flexible, catalog-centric data architecture. It permits deep catalog complexity, native multi-storefront management, and advanced internationalization features directly out of the box without requiring middleware. Crucially, BigCommerce has made deep, highly successful inroads into the B2B sector with its native "B2B Edition." This native functionality supports complex corporate buyer hierarchies, custom corporate quoting, negotiated net payment terms, and dynamic catalog visibility tailored to specific customer groups, capabilities that often require complex, expensive app-based workarounds on competing SaaS platforms.

Economically, BigCommerce represents one of the most predictable and transparent TCO

models in the enterprise space. Its standard plans range from \$39 to \$399 per month, scaling into custom-negotiated pricing for enterprise deployments based on order volume rather than an arbitrary percentage of revenue. Unlike Shopify, BigCommerce explicitly guarantees zero proprietary transaction fees across all plans, regardless of the merchant's chosen payment gateway. This gateway agnosticism is a massive financial incentive for high-GMV retailers operating on thin margins. While its third-party app ecosystem is undeniably smaller than Shopify's, which can occasionally result in a lack of hyper-niche marketing integrations, BigCommerce effectively compensates by providing robust, enterprise-ready features natively, thus maintaining a highly balanced cost-to-value ratio.

4.1.4 Adobe Commerce (Magento)

Magento, which now operates its enterprise-tier deployments under the Adobe Commerce moniker following its acquisition, is historically and currently recognized as the most powerful, complex, and infinitely scalable e-commerce framework available to digital retailers. Operating primarily as an open-source framework or a cloud-hosted Platform-as-a-Service (PaaS) solution, Magento utilizes a highly intricate PHP-based architecture designed specifically to accommodate immense, deeply nested product catalogs, complex multi-regional storefront deployments managed from a unified backend, and exceedingly bespoke business logic that off-the-shelf SaaS cannot accommodate.

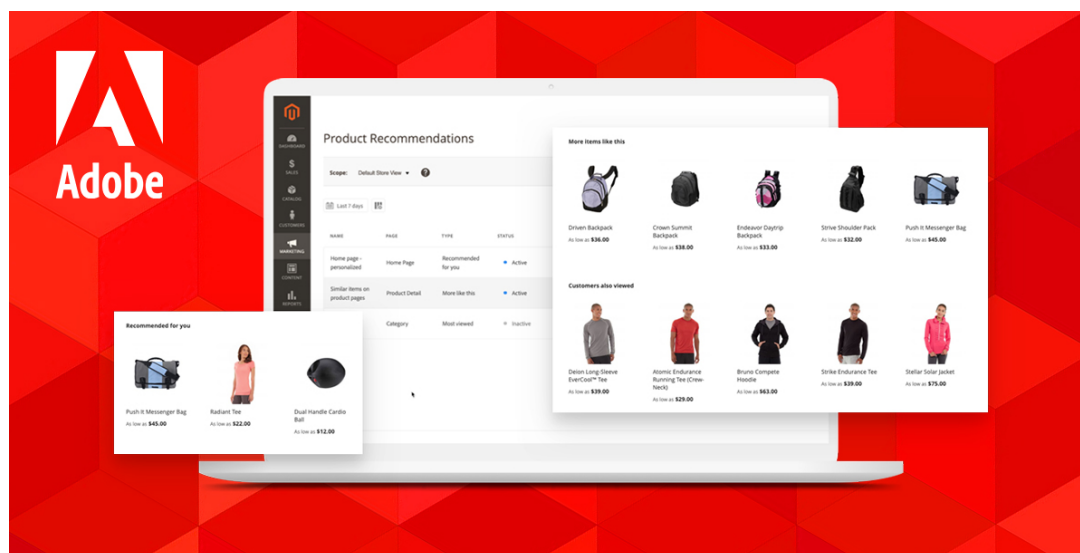


Figure 4.4: The Adobe Commerce backend interface.

Magento is built explicitly for massive global enterprises that view their digital commerce infrastructure not merely as a rented utility, but as a core, proprietary corporate asset. The platform provides absolute code-level freedom, allowing organizations to interface their commerce engine deeply and synchronously with legacy Enterprise Resource

Planning (ERP) systems, proprietary warehouse management and procurement networks, and complex regional fulfillment algorithms. Its native B2B capabilities are considered industry-leading, and its deep integration with Adobe Sensei provides premium, artificial intelligence-driven merchandising and personalization tools that dynamically alter storefronts based on user behavior.

However, Magento's profound architectural power is inherently coupled with exceptional technical difficulty and exorbitant financial costs. The platform requires dedicated internal teams or highly expensive agency retainers consisting of specialized PHP developers, database administrators, and server architects to deploy, optimize, and maintain. Implementation timelines for Adobe Commerce frequently span many months, with initial deployment and replatforming costs routinely exceeding \$150,000 and easily scaling past \$350,000 for complex corporate builds. Furthermore, its Entity-Attribute-Value (EAV) database model, while endlessly flexible for cataloging, is notoriously resource-intensive, requiring highly aggressive, multi-layered caching strategies (e.g., Varnish, Redis) merely to maintain acceptable page load speeds. Consequently, Magento is rarely deemed suitable for any retail entity operating below the large enterprise threshold.

4.1.5 Salesforce Commerce Cloud (SFCC)

Salesforce Commerce Cloud (formerly known as Demandware prior to acquisition) is an elite, enterprise-grade, cloud-hosted SaaS platform tailored predominantly for massive global brands that are already deeply entrenched within the broader Salesforce corporate ecosystem. SFCC operates on a highly controlled, proprietary architectural model that seeks to seamlessly unify digital storefronts (B2C and B2B), physical point-of-sale retail systems, customer service modules (Service Cloud), and advanced marketing automation (Marketing Cloud) into a single, cohesive, omni-channel data environment.

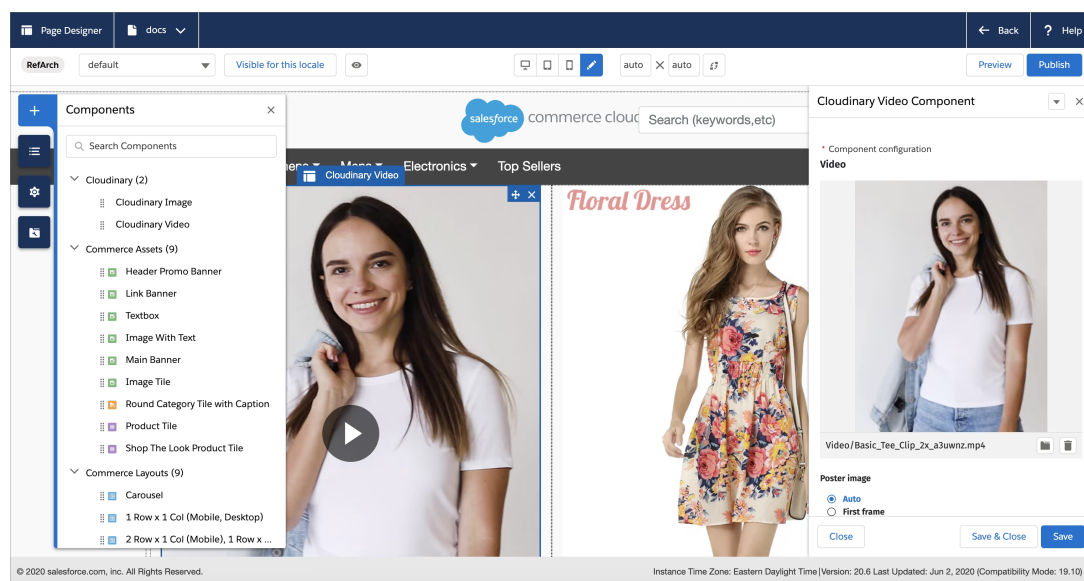


Figure 4.5: The Salesforce Commerce Cloud Page Designer.

A major functional advantage of SFCC is its deep, native integration of Einstein AI. This proprietary artificial intelligence engine delivers real-time predictive product recommendations, highly dynamic merchandising sorting rules, and complex behavioral insights directly into the merchant's workflow without requiring disparate third-party plugins. The platform is engineered to handle extreme operational scale, excelling at managing incredibly complex catalogs often exceeding 500,000 SKUs, and executing sophisticated, multi-national deployment strategies that demand stringent corporate compliance and security standards.

Despite these formidable capabilities, SFCC is frequently criticized in industry evaluations for its highly prohibitive cost structures and somewhat antiquated developer experience. Salesforce employs a revenue-sharing economic model, typically claiming between 1% and 3% of a brand's total GMV, alongside massive base licensing fees that generally begin around \$60,000 annually. This structure inherently punishes high-growth brands by exponentially increasing software costs alongside retail success, making it one of the most expensive platforms on the market over a multi-year timeline. Technologically, SFCC relies on a proprietary development environment known as Business Manager and utilizes custom JavaScript controllers. This legacy architecture creates significant operational friction; simple feature additions or user interface modifications can take weeks to navigate through SFCC's mandatory certification and release cycles. Furthermore, recruiting certified SFCC developers in the modern talent market is notably difficult and substantially more expensive compared to hiring software engineers proficient in ubiquitous modern frameworks like React, Node.js, or standard GraphQL.

4.2 Feature Comparison

To rigorously evaluate the relative strengths, operational trade-offs, and economic implications of the aforementioned iconic platforms against modern composable standards, a highly structured comparative analysis is required. The assessment is divided into key dimensions: overarching architectural metrics, economic models, and advanced enterprise functionality. By isolating these variables, the operational friction inherent in legacy platforms becomes quantifiable, setting the stage for the introduction of a highly optimized custom architecture.

Platform	Architectural Model		Core Stack	Technology	Target Market Demographic	Typical Implementation Velocity	
Shopify / Shopify Plus	Fully SaaS	Managed	Ruby on Rails, Liquid, React		SMBs to Mid-Market / Fast-scaling	Extremely Fast (Hours to Weeks)	
WooCommerce	Self-Hosted Source	Open-	PHP, WordPress Core, MySQL		Content-First Brands, Tech-Savvy SMBs	Moderate (Days to Weeks)	
BigCommerce	Open Hosted	SaaS /	API-First, PHP/React (Catalyst)		Mid-Market to Scaling Enterprises	Fast (Days to Weeks)	
Adobe Commerce	PaaS / Self-Hosted / Cloud		PHP, EAV Architecture		Large Global Enterprises, Complex B2B	Slow (Months)	
Salesforce Commerce Cloud	Enterprise SaaS	Cloud	Proprietary JS, Salesforce Core		Massive Brands (Salesforce Eco.)	Very Slow (6 - 18 Months)	

Table 4.1: High-Level Platform Architecture, Demographic Fit, and Implementation Velocity

As Table 4.1 illustrates, there is a direct, observable inverse correlation between implementation velocity and absolute architectural control. Shopify and BigCommerce permit highly accelerated go-to-market strategies precisely because their multi-tenant SaaS architectures inherently restrict merchants from altering backend server logic or database schemas. This standardization ensures platform stability but limits bespoke backend processes. Conversely, Adobe Commerce and Salesforce Commerce Cloud require protracted, highly managed deployment cycles (often spanning up to 18 months for SFCC) precisely because they involve deep, custom integrations with legacy corporate systems, customized checkout logic, and bespoke data mapping. The time-to-market delay with SFCC is particularly notable; industry data suggests it is 89% more expensive and significantly slower to launch compared to agile SaaS competitors.

Metric	Shopify Plus	WooCommerce	BigCommerce Ent.	Adobe Commerce	Commerce	Salesforce Commerce Cloud	Commerce
Base Licensing	~\$24k - \$36k/yr	Free Software	~\$24k - \$60k/yr	~\$22k - \$100k+/yr	-	\$60k - \$500k+/yr	
Transaction Fees	0% (only with prop. gateway); else 0.2%+	Gateway dependent; No platform fee	0% universally regardless of gateway	Gateway dependent; No platform fee		1% to 3% of GMV	
Est. 3-Year TCO	\$252k – \$528k	\$180k – \$420k	\$300k – \$650k	\$636k – \$1.5M+		Exceeds \$1.5M+ (Scales with GMV)	
Development Burden	Low (2-4 devs)	Medium (3-6 PHP devs)	Low to Medium (2-5 devs)	Extremely High (4-10+ specialized devs)		Extremely High (Certified req.)	High talent

Table 4.2: Economic Models and Three-Year Total Cost of Ownership (TCO)

The economic analysis detailed in Table 2 reveals fundamental, systemic differences in revenue generation strategies among software vendors. Salesforce Commerce Cloud utilizes a predatory GMV-based tax model, which significantly and predictably inflates the 3-Year TCO as a business scales its revenue. Shopify’s pricing is superficially attractive at the baseline, but the punitive transaction fees levied against third-party payment gateways act as a powerful mechanism to lock merchants into the Shopify financial ecosystem. Furthermore, the app-subscription creep on Shopify often pushes the TCO far beyond the base license. BigCommerce represents the most transparent economic model for high-growth firms by completely eliminating GMV percentages and transaction taxation. WooCommerce, while highly popular, presents a deeply misleading entry cost; while the core software is free, the subsequent mandatory requirements for managed hosting, continuous plugin coordination, and security liability management push its actual TCO well above managed SaaS platforms for businesses clearing significant commercial revenue.

Capability	Shopify Plus	WooCommerce	BigCommerce	Adobe Commerce	Commerce	Salesforce Commerce Cloud	Commerce
Native B2B	Strong (Portal features)	Weak (Plugins req.)	Excellent	Industry	Lead-	Strong (Handles complex hierarchies)	
App & Plugin Ecosystem	Massive (>8,000 apps)	Enormous (>59k plugins)	Moderate	Extensive (Enterprise)	(En-	Closed (Salesforce-centric)	
Headless / Composable	Excellent (Hydrogen)	Decoupled DIY	Strong (Native APIs)	Mature (Studio)	(PWA)	Strong (Composable Storefront)	
AI and Personalization	Shopify Magic (Native ML)	Plugin-based	Limited Native	Premium Add-on (Sensei)		World-Class Native (Einstein AI)	
Data Ownership	Low (High Vendor Lock-in)	Absolute	Medium	Absolute		Low (Ecosystem Lock-in)	

Table 4.3: Technical Capabilities, Ecosystem Depth, and Omnichannel Readiness

Technologically, the feature comparison outlined in Table 4.3 highlights the growing industry dichotomy between monolithic convenience and modular freedom. The current market shows that 27% of enterprise brands are actively running or migrating to headless commerce architectures. Platforms like Shopify Plus and BigCommerce have heavily invested in GraphQL APIs to facilitate this "composable" commerce, a paradigm where the frontend presentation layer (frequently built on Next.js or React) is entirely decoupled from the backend e-commerce engine. This architecture allows merchants to deliver highly bespoke, lightning-fast frontend experiences across mobile and web without sacrificing the backend security, checkout stability, and PCI compliance of a managed SaaS provider. Conversely, platforms like Adobe Commerce and WooCommerce offer absolute data ownership and database portability but require the merchant organization to orchestrate, optimize, and secure the entire technical stack manually.

While these five commercial platforms dominate the existing landscape, they each force a significant operational compromise upon the enterprise. SaaS solutions universally mandate vendor lock-in, restrict profound backend architectural alterations, and penalize growth through transaction fees. Self-hosted monolithic solutions demand exorbitant developmental overhead, present severe security liabilities, and inevitably suffer from legacy codebase bloat that degrades site performance over time. To circumvent these systemic limitations entirely, organizations with highly specific data models, real-time transaction needs, and complex multi-channel requirements are increasingly turning to purpose-built, microservices-driven architectures.

4.3 Competitive Advantages of the System

To directly address and neutralize the inherent compromises, economic penalties, and architectural bottlenecks of commercial off-the-shelf platforms, we propose a proprietary E-Commerce Management System built upon a highly modular, decoupled microservices architecture. Unlike legacy monolithic systems constrained by bloated PHP codebases (e.g., Magento/WooCommerce) or inflexible proprietary databases that restrict URL mapping (e.g., Shopify), the proposed System utilizes a modern, cloud-native technological stack. This stack is specifically optimized for ultra-high concurrency, absolute data sovereignty, real-time omni-channel synchronization, and the seamless integration of highly sophisticated machine learning models.

The System operates on a Node.js/Nest.js backend, a profoundly robust framework renowned for its enterprise-grade structural rigor, dependency injection, and native, out-of-the-box support for distributed microservices. This foundational architecture immediately resolves the primary bottlenecks experienced by traditional vertical applications during massive traffic surges. By extracting core business logic into distinctly isolated microservices such as order management, inventory control, payment processing, and user

authentication, the System achieves unparalleled high availability and fault tolerance. If the payment service experiences a surge, it can scale independently without requiring the entire application to duplicate, thereby avoiding the massive, inefficient hardware requirements and server loads typical of traditional platforms.

The competitive advantages of this custom-engineered System manifest distinctly across five critical dimensions: architectural agility and real-time data processing, sophisticated automated testing methodologies, absolute economic control and sovereign security, deeply customized B2B/B2C logic, and state-of-the-art predictive behavioral analytics.

4.3.1 Uncompromising Reliability through Advanced Testing Frameworks

Commercial platforms often present a black-box reliability model; merchants are forced to implicitly trust the vendor's universal update cycles, which occasionally introduce breaking changes to specialized third-party themes or critical plugins, resulting in devastating downtime. The proposed System systematically mitigates these regression risks through a rigorously implemented, multi-layered testing protocol natively integrated into the Continuous Integration/Continuous Deployment (CI/CD) pipeline.

Utilizing Jest within the Nest.js ecosystem, the System enforces comprehensive Unit Testing and Integration Testing across all discrete operational boundaries. Business logic encapsulated within the Service layer, complex data access and aggregation mechanisms within the Repository layer, and strict request validation within the Controller layer are all exhaustively tested in isolation prior to deployment. This ensures that the introduction of a new, highly complex feature, such as a bespoke B2B volume-discounting algorithmic rule or a new regional tax calculation, cannot inadvertently corrupt existing, stable checkout logic. This unparalleled level of test coverage guarantees the structural integrity of the application, rendering the System highly resilient and virtually eliminating the "plugin conflict" vulnerabilities inherent to the WordPress/WooCommerce ecosystem.

4.3.2 Absolute Economic Control and Sovereign Security

By operating as a sovereign, custom-built, headless architecture, the System entirely bypasses the predatory economic models of the commercial SaaS sector. Organizations deploying this System are not subjected to the punitive 1% to 3% GMV tax levied by Salesforce Commerce Cloud, nor are they coerced into utilizing a specific proprietary payment processor under threat of margin-eroding transaction fees, a tactic synonymous with Shopify. The enterprise pays only for its underlying, highly optimized cloud host-

ing infrastructure, resulting in a TCO that remains flat and predictable even as Gross Merchandise Value scales into the hundreds of millions.

Furthermore, security is handled holistically at the architectural level rather than as an afterthought. Rather than relying on obscure, frequently updated third-party plugins for user authentication, the System natively implements enterprise-grade security protocols. This includes JSON Web Tokens (JWT) coupled with Refresh Tokens for secure, stateless API authentication, seamless Google OAuth 2.0 integrations, and mandatory Two-Factor Authentication (2FA) for all administrative and high-level B2B portal access. A sophisticated Role-Based Access Control (RBAC) matrix ensures highly precise administrative partitioning, preventing unauthorized internal access to sensitive financial data. All data payloads are strictly validated using Zod schemas before hitting the database, and data at rest is heavily encrypted. This robust security posture ensures highly secure transactions and seamlessly addresses the fundamental technical requirements for achieving and maintaining Level 1 PCI DSS compliance.

4.3.3 Next-Generation Behavioral Analytics via HT-Transformer Integration

Perhaps the most profound and unique competitive advantage of the proposed System is its native integration of highly advanced machine learning models directly into the primary data processing pipeline. Specifically, the System utilizes the state-of-the-art History Token-Transformer (HT-Transformer) architecture for deep event sequence classification. While legacy platforms like SFCC rely on generalized, proprietary AI models (Einstein), and platforms like Shopify rely on basic rule-based recommendation engines, the proposed System is custom-engineered to process highly granular consumer event sequences natively.

In the context of modern e-commerce, consumer interactions, such as rapid product views, micro-dwell times on specific variants, cart additions, and search queries, form continuous, highly complex temporal sequences. Traditional machine learning models, particularly those utilizing standard Next-Token Prediction (NTP), struggle immensely to contextualize long-term user history without encountering catastrophic computational overhead due to the quadratic scaling of attention mechanisms. The HT-Transformer architecture elegantly bypasses this critical limitation by accumulating and summarizing long-term contextual information via highly specialized History Tokens. By processing continuous user interactions through this compressed mechanism, the System can generate incredibly accurate, predictive behavioral embeddings without bogging down the main server thread.

The business implications of this native intelligence are massive. It allows the System

to dynamically restructure the storefront presentation layer in real-time, anticipate cart abandonment with microscopic precision based on navigation hesitation, and execute hyper-personalized product recommendations based on the deep sequential context of an individual user's entire historical lifecycle, not just their current session. Because this intelligence is built directly into the System's microservices architecture via high-speed internal APIs, the severe network latency typically associated with calling external third-party personalization engines is virtually eliminated, ensuring that the storefront remains lightning-fast even while rendering highly individualized content.

In conclusion, while iconic platforms like Shopify, WooCommerce, BigCommerce, Adobe Commerce, and Salesforce Commerce Cloud offer viable, well-documented solutions for generalized retail scenarios, they each demand significant, often painful compromises regarding total cost, technical capability, or absolute architectural control. The proposed custom E-Commerce Management System decisively resolves these systemic tensions. By combining the immense scalability, fault tolerance, and developer agility of a Nest.js/GraphQL microservices architecture with rigorous automated testing protocols, sovereign economic control, and the unprecedented predictive power of HT-Transformer analytics, the System represents a structurally superior framework. It is explicitly designed for modern enterprises seeking absolute digital autonomy, zero proprietary revenue taxation, and peak operational performance in an increasingly competitive digital marketplace.

Chapter 5

Conclusion and Future Works

5.1 Conclusion

The E-Commerce Management System project represents a comprehensive and ambitious effort to model, design, and implement a fully functional digital retail platform. The primary objective of this project was to establish a scalable, maintainable, and secure software architecture capable of handling modern online retail operations for small to medium-sized businesses. By leveraging a modern technology stack comprising ASP.NET Core 8 Web API for the backend, Entity Framework Core for data access, SQL Server for relational data storage, and React for the frontend user interface, the project successfully established a decoupled, microservices-inspired monolithic architecture.

The completed phases of the project have successfully demonstrated the core capabilities required to operate an online storefront. The system supports a robust product catalog, allowing administrators and staff to manage complex product variants, categories, and brands. The strategic integration of Cloudinary for external image hosting ensures that high-resolution media assets are delivered efficiently to the end-user without burdening the primary application servers or consuming excessive database storage. Furthermore, the implementation of JSON Web Token authentication provides a stateless, secure method for managing user sessions and enforcing role-based access control across the Customer, Staff, and Administrator roles.

Operational workflows, including shopping cart management, order creation, payment processing logic, and promotional voucher application, were successfully mapped and implemented during the development phase. The database schema was meticulously designed to preserve historical accuracy. For example, utilizing snapshot techniques in the `OrderDetails` and `Orders` tables ensures that future product price changes or voucher expirations do not retrospectively alter historical financial records or customer order summaries. The architectural decision to incorporate dependency injection, global exception handling middleware, and a strict Controller-Service-Repository pattern ensures that the codebase remains modular, readable, and highly testable.

However, the transition from architectural design to practical implementation revealed

several critical disparities between the intended system behavior and the actual codebase. An exhaustive review of the system documentation, the codebase summary, and the testing parameters indicates that while the standard "happy path" of the application functions correctly, the system possesses significant vulnerabilities when subjected to edge cases, unauthorized access attempts, and unexpected user behavior.

The evaluation identified explicit logical errors in the system specifications, severe friction points in the user experience design, critical omissions in data validation, and potential security leaks within the data access layer. Furthermore, the packaging of the system for review necessitated the disabling of automated security checks, highlighting a gap in the continuous integration and deployment pipeline. Because a digital commerce platform is directly responsible for processing highly sensitive personal identification and financial transaction data, these weaknesses cannot be overlooked.

Consequently, the conclusion of this initial development phase does not mark the end of the software engineering lifecycle. Instead, it establishes a functional baseline from which targeted, data-driven improvements must be made. The current iteration proves the fundamental viability of the architecture, but the system is not yet prepared for live production deployment. The subsequent section outlines a highly detailed, prioritized roadmap for future works. This roadmap focuses exclusively on feasible, high-impact remediations designed to patch existing loopholes, fortify system security, complete omitted workflows, and elevate the user experience to meet modern commercial standards.

5.2 Future Works

The future development roadmap is directly informed by the limitations, logical errors, and security vulnerabilities identified during the system evaluation phase. The core requirement for future work is to develop upon existing loopholes and weak points while remaining technically feasible to implement within the established ASP.NET Core and React architecture. To ensure a structured and systematic approach to system refinement, the future works are categorized into six primary domains: Security and Identity Management, User Experience and Workflow Optimization, API Stability and Input Validation, Functional Completeness of the Support Module, Testing and Infrastructure Standardization, and Advanced Analytical Expansions.

5.2.1 Remediation of Security Vulnerabilities and Identity Management

Security in an e-commerce platform is the most critical component of the system architecture, as the platform serves as the central custodian of personal identification and financial transaction data. The current implementation exhibits several severe access control flaws, data exposure vulnerabilities, and provisioning ambiguities that must be addressed immediately to protect the system from malicious actors.

Restriction of Administrative Account Creation

The most critical security vulnerability identified in the system specifications is the unrestricted nature of the Administrator account creation process. Currently, the system specification for the AD-EC01 function (Register account) allows the creation of an administrative account with the condition explicitly listed as "No requirement". This structural flaw theoretically permits any public user, guest, or automated bot to interface with the API and elevate their privileges to the highest system level. If exploited, an attacker could gain full control over the E-Commerce Management System, allowing them to modify product prices, view sensitive customer data, and intercept financial transactions.

Future work will mandate the complete removal of public registration endpoints for the Admin role. The `POST /api/user/create-staff-admin` endpoint will be strictly protected by JWT authorization policies configured within the ASP.NET Core middleware. This policy will require the requesting user to already possess a valid "Super-Admin" role token to execute the request. For the initial bootstrapping of the system in a production environment, a master administrator account will be securely seeded directly into the SQL Server database via Entity Framework Core migrations. This initial account will utilize a highly complex, BCrypt-hashed password. Subsequent administrative accounts will only be created through a secure, internal dashboard, ensuring that public actors cannot exploit the registration payload to inject elevated roles.

Prevention of Sensitive Data Exposure via DTO Mapping

An in-depth analysis of the `AdminService.cs` implementation revealed a severe data leakage vulnerability during the retrieval of user lists. The current implementation utilizes Entity Framework Core's eager loading mechanism, specifically invoking `_context.Users.Include(u => u.Role)`, to fetch user records and their associated role data from the database. While this approach is functionally correct for retrieving related data, it directly serializes the entire `User` entity to the client frontend. In the context of the `Users` table schema, this entity includes highly sensitive fields such as the `Password` (which

stores the BCrypt hash), contact details, and internal system flags like `IsDeleted`. Exposing cryptographic hashes to the frontend environment, even if they are hashed, provides malicious actors with the material necessary to execute offline brute-force or rainbow table attacks.

To rectify this loophole, future work will involve the strict implementation of Data Transfer Objects (DTOs) across all service layer outputs. The architecture will integrate an object mapping library, such as AutoMapper, to enforce explicit projection boundaries. Instead of returning the raw `User` entity, the service will utilize the `.Select()` LINQ method to project the database query directly into a `UserListResponseDto`. This DTO will exclusively contain safe properties required by the React frontend, such as `Id`, `Name`, `Email`, and `RoleName`. By shifting the data projection to the database query level, the system will not only eliminate the security vulnerability by keeping cryptographic data confined to the backend, but it will also significantly reduce the memory footprint and network payload size, optimizing overall API performance.

Standardization of Staff Account Provisioning

The current system logic specifies a staff login function (ST-EC01) which requires a "Registered account". However, unlike the Administrator and Customer roles, the documentation completely omits a clear, standardized workflow for how staff members register or receive their accounts in the first place. This ambiguity leads to operational confusion and potential security workarounds where administrators might share generic staff credentials, destroying individual accountability and audit trails.

Future development will build a dedicated Human Resources provisioning module within the Admin React dashboard. Administrators will be able to generate secure invite links containing time-limited, cryptographically signed JWT tokens. Prospective staff members will receive these tokens via email and use them to access a hidden secure registration form where they can establish their own personal credentials. The backend will validate the invite token before creating the account and assigning the Staff role. This ensures that the Staff role remains a tightly controlled environment, eliminating the need for ambiguous self-registration flows and ensuring that every staff action can be audited to a specific individual.

Identified Security Vulnerability	Current System State	Proposed Technical Remediation	Expected Security Outcome
Unrestricted Admin Registration	AD-EC01 allows account creation with "No requirement".	Implement strict RBAC on the creation endpoint; require an existing Admin token to provision new Admins.	Total elimination of unauthorized privilege escalation from public endpoints.
Sensitive Data Exposure	<code>AdminService.cs</code> returns the complete <code>User</code> entity, including password hashes.	Implement <code>UserListResponseDto</code> and EF Core <code>.Select()</code> projection.	Cryptographic data and internal system flags remain permanently confined to the secure backend boundary.
Vague Staff Provisioning	ST-EC01 requires an account, but no registration flow is defined in the specs.	Develop a token-based invitation system generated by authorized Administrators.	Establishment of a clear, secure, and auditable staff onboarding process.

Table 5.1: Security Vulnerabilities and Proposed Remediations

5.2.2 User Experience Optimization and Workflow Friction Reduction

A successful e-commerce platform must balance rigorous backend security with a frictionless, intuitive frontend user experience. The evaluation of the current system design revealed areas where strict system rules inadvertently create barriers to customer conversion and continuous engagement.

Implementation of Anonymous Browsing and Guest Carts

The most detrimental user experience flaw in the current system design is the mandatory authentication requirement for basic platform interaction. In the modern digital retail landscape, forcing users to create an account before they can add items to their cart or

calculate totals results in exceptionally high bounce rates and lost revenue.

Future works will fundamentally redesign the authentication middleware and the Shopping Cart module to fully support anonymous sessions. The product catalog endpoints will be explicitly configured to remain completely public, allowing unrestricted browsing, filtering, and search functionality for all visitors.

For the shopping cart functionality, the system will integrate temporary session-based management. When an unauthenticated user adds an item to their cart, the React frontend will generate a unique UUID representing an anonymous session. The ASP.NET Core backend will store this anonymous cart data in a Redis cache rather than the persistent SQL Server database, utilizing the UUID as the key. This ensures that abandoned guest carts naturally expire via Time-To-Live (TTL) settings and do not bloat persistent storage.

When the user eventually decides to proceed to checkout and creates an account, the backend will intercept the registration flow, retrieve the anonymous cart from Redis using the session UUID, and seamlessly migrate the items into a permanent Cart record in the SQL Server database. This technical solution entirely removes the friction of mandatory early login while maintaining perfect data consistency for the order processing pipeline.

Rectification of Payment Logic Specifications

A critical logical error was identified in the system documentation for the payment function. The specification for CU-EC06 (Payment) incorrectly describes the action as "Download learning materials provided by the teacher". This represents a severe mismatch in system specifications, likely resulting from a documentation copy-paste error from an unrelated educational software project.

Future work will require the complete rewriting of the specification and the underlying controller code for the payment gateway integration. The corrected workflow will align with standard e-commerce financial processing. The customer will select VNPay as the supported online payment method, and the React frontend will trigger the `POST /api/orders` endpoint. The backend will lock the inventory to prevent overselling, calculate the final total including any applied vouchers and shipping fees, and generate a secure payment link via a third-party payment provider API.

The system will then expose a dedicated webhook endpoint to listen for asynchronous payment confirmations from the payment provider. Upon receiving a cryptographically verified success signal, the system will update the `Orders` table status from "Pending" to "Processing" and formalize the inventory deduction. If the payment fails or times out, the system will release the inventory lock and notify the user. This completely replaces the erroneous "learning materials" logic with a robust, production-ready financial transaction

pipeline.

Correction of UI/UX Copywriting and Localization

The system evaluation noted several typographical and spelling errors within the user interface specifications. For example, the customer support function CU-EC09 is written as "Khiếu nại đơn hàng", which contains a spelling typo and should be the grammatically correct Vietnamese term "Khiếu nại đơn hàng" (Order complaint). While seemingly minor compared to architectural flaws, typographical errors severely erode consumer trust in the platform's security and professional operation.

Future work will involve a comprehensive audit of all frontend text assets within the React application. Rather than hardcoding text strings directly into the React components, a centralized localization dictionary, such as `react-i18next`, will be implemented. This will ensure that all terminology is consistent, grammatically correct, and easily manageable. Furthermore, implementing a proper localization framework immediately prepares the platform for future expansion into international markets by allowing seamless language switching without requiring code changes.

5.2.3 API Stability, Input Validation, and Data Integrity

The stability of the ASP.NET Core backend API relies heavily on its ability to handle unexpected, malformed, or malicious data inputs gracefully. The current system exhibits significant weaknesses in how it processes incoming client requests, leading to potential runtime crashes and database corruption.

Prevention of Null Reference Exceptions in Query Filtering

The implementation of the `GET /api/support/all` endpoint contains a critical flaw in its data filtering logic. The endpoint accepts a nullable query parameter for filtering tickets by status (`[FromQuery] string? status`). However, the internal LINQ query attempts to normalize the input using the expression `.Where(s => s.Status.ToLower() == status.ToLower())`.

If a client calls this API without providing the `status` parameter, the `status` variable resolves to null. The subsequent invocation of the `.ToLower()` method on a null object will instantly trigger a `NullReferenceException`, causing the API to crash and return a 500 Internal Server Error to the React frontend. Furthermore, if the database contains records where the status itself is null, the query translation in Entity Framework Core may fail unpredictably depending on the specific SQL Server provider configuration.

Future work will refactor this endpoint to utilize safe query execution patterns. The logic will be updated to first verify the existence of the parameter before applying the filter to the database query. The corrected LINQ expression will utilize the `string.Equals` method with `StringComparison.OrdinalIgnoreCase`, which naturally handles null values without throwing exceptions.

The revised logic will follow this exact structure in the controller:

- Initialize the base query:

```
var query = _context.SupportRequests.AsQueryable();
```
- Apply conditional filtering only if the parameter exists:

```
if (!string.IsNullOrEmpty(status))  
{ query = query.Where(s => s.Status == status); }
```
- Execute the query and return the result to the client.

This approach ensures that the API remains highly resilient. If a staff member navigates to the support page without selecting a filter, the API will safely return the full, unfiltered list of tickets, rather than crashing the application.

API Endpoint	Identified Weakness	Proposed Remediation	Technical Benefit
GET /api/support/all	NullReferenceException crash on null status query parameter.	Implement conditional query execution and null-safe string comparison.	Eliminates 500 Internal Server Errors; improves API uptime and reliability.

Table 5.2: API Stability Issue and Proposed Remediation

5.2.4 Expanding Functional Completeness: Support Request Lifecycle

An e-commerce platform must provide comprehensive tools for staff and administrators to manage customer interactions and resolve complaints. The current implementation of the Support Request module suffers from a severe functional omission that renders the system practically unusable for real-world customer service operations.

Implementation of Ticket Update and Resolution Endpoints

The system documentation indicates that endpoints exist for customers to create tickets (POST /api/support), for customers to view their own tickets, and for Staff/Admin to view all tickets (GET /api/support/all). However, there are absolutely no endpoints implemented to allow Staff or Administrators to update, reply to, or change the status of these tickets. Consequently, the Staff interface is entirely view-only. Once a customer submits a complaint, it remains permanently in its initial state, as the core "Update" component of the CRUD paradigm was completely omitted from development.

Future work will prioritize the completion of the Support Request lifecycle. The development team will implement a new endpoint: PATCH /api/supports/{id}/status. This endpoint will be protected by JWT authorization, ensuring that only users with the Staff or Admin role can invoke it.

The system will enforce a strict Finite State Machine (FSM) for the ticket status lifecycle. The allowed transitions will be defined programmatically in the Service layer to prevent illogical updates:

- **Pending (0):** The initial state upon customer submission.
- **In Progress (1):** Transitioned by a staff member when they begin investigating the issue. This action will automatically record the **AssignedStaffID** in the database, linking the ticket to the specific staff member handling it.
- **Resolved (2):** Transitioned by a staff member when a solution is provided to the customer.
- **Closed (3):** Transitioned automatically by a background worker service after 7 days of inactivity following a "Resolved" state, or manually closed by an Administrator.

Development of Customer-Staff Message Threading

Simply updating the status of a ticket is insufficient; staff must be able to communicate with the customer. A secondary endpoint, POST /api/supports/{id}/messages, will be developed to allow back-and-forth communication between the customer and the assigned staff member.

This will require an update to the database schema. **SupportTicketMessages**, will be created as a new database table and linked via a foreign key to the primary **SupportRequests** table. This new table will store the message content, the sender's user ID, and a timestamp. The React frontend will be updated to display these messages in a chat-like

interface within the Support Ticket Detail screen. This enhancement transforms the module from a static, view-only list into a dynamic, fully functional customer relationship management (CRM) tool, significantly improving the operational capabilities of the platform.

5.2.5 Testing Infrastructure, Security Scanning, and Deployment Standardization

While a significant portion of future work focuses on application code, the operational environment and testing infrastructure of the codebase itself require urgent attention. The project summary explicitly warned that security checks were disabled during the packaging of the system, and that the packed files may contain highly sensitive credentials, API keys, or private backend details. Furthermore, binary files and gitignored files were excluded, meaning the current repository snapshot is incomplete. Deploying code to a production environment with disabled security checks and potentially hardcoded credentials is an unacceptable risk that could lead to an immediate data breach.

Restoration of Security Checks and CI/CD Pipeline Hardening

Future development will mandate the implementation of a rigorous Continuous Integration and Continuous Deployment (CI/CD) pipeline, utilizing industry-standard tools such as GitHub Actions or Azure DevOps. The pipeline will enforce strict automated gates that code must pass before it can be merged or deployed.

The first gate will be Automated Secret Scanning. Tools like GitGuardian or Trivy will be integrated into the repository to scan every single commit. This ensures that no JWT secret keys, Cloudinary API keys, SQL database connection strings, or third-party payment gateway credentials are ever hardcoded into the source code. All configuration variables must be injected dynamically via secure environment variables or a secure vault service during deployment. If a developer accidentally commits a secret, the pipeline will immediately fail, and the repository administrators will be notified to rotate the compromised key.

The second gate will be Static Application Security Testing (SAST). Tools like SonarQube will analyze the C# and JavaScript source code during the build process to identify code smells, architectural flaws, and vulnerabilities. This automated analysis is highly effective at catching issues like the null reference vulnerability identified in the support controller, or unvalidated input pathways, long before the code reaches a testing environment.

Expansion of Unit and Integration Testing Suites

The project directory indicates the presence of a testing suite (`Backend/E-commerce.Tests/`) containing files like `ProductServiceTests.cs` and `SupportControllerTests.cs`. However, given the presence of the `NullReferenceException` vulnerability and the lack of input validation, it is highly probable that the current tests focus exclusively on the "happy path" where all inputs are perfectly formatted.

Future work will require a vast expansion of this test suite. Developers will be required to adopt a test-driven mindset, prioritizing edge cases and error handling. For example, new unit tests must be written that intentionally send null parameters to query filters, excessively long strings to DTOs, and unauthorized JWT tokens to protected endpoints. The tests must assert that the API responds with the correct 400 Bad Request or 403 Forbidden HTTP status codes, rather than failing internally and returning a 500 Internal Server Error. A minimum test coverage threshold (e.g., 80% line coverage) will be enforced by the CI/CD pipeline, ensuring that all future modifications are thoroughly verified before deployment.

5.2.6 Preparing for Advanced Analytics and Performance Scaling

While the primary focus of the future works is the remediation of existing flaws and the completion of omitted features, it is also highly feasible to prepare the architecture for advanced data processing and performance scaling. The core system documentation explicitly mentions the competitive advantage of integrating next-generation behavioral analytics via HT-Transformer integration.

Data Pipeline Construction for HT-Transformer Analytics

Standard machine learning models often struggle to contextualize long-term user history without encountering massive computational overhead due to the complexities of processing sequential data. The HT-Transformer architecture elegantly solves this by compressing long-term historical event sequences into specialized History Tokens. However, to make this advanced capability feasible as a future work, the ASP.NET Core backend must first be modified to act as a highly efficient, non-blocking data collection pipeline.

Future work will involve creating lightweight middleware in the ASP.NET Core application that asynchronously captures user interaction events. These events include actions such as category navigation, specific product variant dwell times, search queries, and

cart abandonment actions. Rather than writing these events directly to the SQL Server database—which would cause massive database bottlenecks and degrade the performance of the checkout system—the API will publish these event payloads to a message queue.

The system will implement BullMQ, an enterprise-grade queueing system backed by Redis, to handle these event streams. A separate, completely decoupled Python-based microservice will consume these messages from the queue, tokenize the user sequences, and feed them into the HT-Transformer model for processing.

This asynchronous, decoupled approach ensures that the primary e-commerce backend remains exceptionally fast and responsive to customer requests, while the heavy machine learning inference occurs entirely in the background. Once the model predicts a high likelihood of cart abandonment or identifies a highly relevant cross-sell opportunity for a specific user, it will update a personalized cache in Redis. The React frontend will subsequently query this Redis cache to dynamically render personalized product recommendations or targeted discount pop-ups in real-time, providing a highly tailored shopping experience without compromising system performance.

Database Indexing and Query Optimization

As the platform scales and the number of products, users, and orders grows, the performance of the SQL Server database will become a critical factor. The current documentation outlines the basic table structures but does not detail a comprehensive indexing strategy.

Future work will involve a thorough review of the Entity Framework Core queries generated by the application. Using tools like SQL Server Profiler, the development team will identify slow-running queries, particularly those involving complex joins between the `Products`, `Product Variants`, and `Categories` tables. Non-clustered indexes will be applied to frequently filtered columns, such as `CategoryId`, `BrandId`, and `Status` in the `Orders` table, to ensure that database lookups remain instantaneous even with millions of records.

In conclusion, the execution of these detailed future works will systematically eliminate the security vulnerabilities, logical errors, and user experience friction points present in the current iteration of the E-Commerce Management System. By enforcing strict DTO projections to protect sensitive data, implementing anonymous guest carts to boost conversion rates, completing the CRM lifecycle endpoints for effective staff operations, and hardening the automated deployment pipelines, the system will evolve from a functional academic prototype into a highly resilient, secure, and commercially viable digital enterprise platform capable of supporting advanced analytics and significant future growth.

Appendix A

Appendix

A.1 API Documentation

This appendix summarizes the main RESTful API endpoints of the E-Commerce Management System, including HTTP methods, endpoint paths, access roles, and business purposes.

Method	Endpoint	Description
1. Authentication		
POST	/api/auth/login	User Login. <i>Roles:</i> Public <i>Response:</i> Returns single data containing token, name, and roleName.
POST	/api/auth/register	User Registration. <i>Roles:</i> Public <i>Response:</i> Creates a new Customer account.
2. User Profile		
GET	/api/user/profile	Retrieve Personal Profile. <i>Roles:</i> Customer, Staff, Admin <i>Response:</i> Returns single data with the current user's profile details.
PUT	/api/user/profile	Update Personal Profile. <i>Roles:</i> Customer, Staff, Admin.
PUT	/api/user/password	Change Password. <i>Roles:</i> Customer, Staff, Admin.
DELETE	/api/user/profile	Delete Personal Profile. <i>Roles:</i> Customer, Staff, Admin.
POST	/api/user/ create-staff-admin	Create Staff or Admin Account. <i>Roles:</i> Admin.
3. Categories		

GET	/api/categories	Retrieve Category List. <i>Roles:</i> Public <i>Response:</i> Returns a list of categories as an array inside data .
POST	/api/categories	Create New Category. <i>Roles:</i> Admin.
PUT	/api/categories/{id}	Update Category by ID. <i>Roles:</i> Admin.
DELETE	/api/categories/{id}	Delete Category by ID. <i>Roles:</i> Admin.
4. Brands		
GET	/api/brands	Retrieve Brand List. <i>Roles:</i> Public <i>Response:</i> Returns a list of brands as an array inside data .
POST	/api/brands	Create New Brand. <i>Roles:</i> Admin.
PUT	/api/brands/{id}	Update Brand by ID. <i>Roles:</i> Admin.
DELETE	/api/brands/{id}	Delete Brand by ID. <i>Roles:</i> Admin.
5. Products		
GET	/api/products	Retrieve Product List. <i>Roles:</i> Public <i>Response:</i> PagedResponse<ProductResponse> supporting query filters such as page , pageSize , and q .
GET	/api/products/{id}	Retrieve Product Details by ID. <i>Roles:</i> Public <i>Response:</i> Returns single data of the product.
POST	/api/products	Create New Product. <i>Roles:</i> Admin.
PUT	/api/products/{id}	Update Product by ID. <i>Roles:</i> Admin, Staff.
DELETE	/api/products/{id}	Delete Product by ID. <i>Roles:</i> Admin.
6. Product Variants		
GET	/api/products/{productId}/variants	Retrieve Variants of a Product. <i>Roles:</i> Public.

POST	/api/products/{productId}/variants	Create New Variant for a Product. <i>Roles:</i> Admin, Staff.
PUT	/api/variants/{id}	Update Variant. <i>Roles:</i> Admin, Staff <i>Note:</i> Mainly used by frontend to update stock quantity.
DELETE	/api/variants/{id}	Delete Variant by ID. <i>Roles:</i> Admin, Staff.
7. Product Images		
POST	/api/products/{productId}/images	Add Image URL to Product Record. <i>Roles:</i> Admin, Staff.
DELETE	/api/images/{imageId}	Delete Image from Product Record by ID. <i>Roles:</i> Admin, Staff.
POST	/api/images/upload	Upload Image File to Cloudinary. <i>Roles:</i> Admin, Staff <i>Format:</i> multipart/form-data with field file. <i>Response:</i> Returns direct URL string in single data.
8. Cart		
GET	/api/cart	Retrieve Current Cart. <i>Roles:</i> Private (Token required) <i>Response:</i> Returns a list of cart items.
POST	/api/cart/add	Add Item to Cart. <i>Roles:</i> Private (Token required) <i>Body:</i> { productId, quantity }.
PUT	/api/cart/update	Update Cart Item Quantity. <i>Roles:</i> Private (Token required) <i>Body:</i> { productId, quantity }.
DELETE	/api/cart/remove	Remove Item from Cart. <i>Roles:</i> Private (Token required) <i>Query:</i> ?productId={id}.
DELETE	/api/cart	Clear Entire Cart. <i>Roles:</i> Private (Token required).
9. Orders		
POST	/api/orders	Create New Order / Checkout. <i>Roles:</i> Customer <i>Body:</i> CreateOrderRequest; send cartItemIds: null to buy all items.

GET	/api/orders	Retrieve All Orders. <i>Roles:</i> Admin, Staff <i>Response:</i> Returns a list of all orders in the system.
GET	/api/orders/my	Retrieve My Orders. <i>Roles:</i> Customer.
GET	/api/orders/{id}	Retrieve Order Details by ID. <i>Roles:</i> Private (Token required).
PUT	/api/orders/{id}/status	Update Order Status. <i>Roles:</i> Admin, Staff.
PUT	/api/orders/{id}/cancel	Cancel Order. <i>Roles:</i> Customer; must be the owner of the order.
10. Shipping Addresses		
GET	/api/shipping-addresses	Retrieve User's Shipping Addresses. <i>Roles:</i> Private (Token required).
POST	/api/shipping-addresses	Create New Shipping Address. <i>Roles:</i> Private (Token required).
PUT	/api/shipping-addresses/{id}	Update Shipping Address by ID. <i>Roles:</i> Private (Token required).
DELETE	/api/shipping-addresses/{id}	Delete Shipping Address by ID. <i>Roles:</i> Private (Token required).
PATCH	/api/shipping-addresses/{id}/default	Set Selected Address as Default. <i>Roles:</i> Private (Token required).
11. Payment Methods		
GET	/api/payment-methods	Retrieve Payment Methods List. <i>Roles:</i> Public.
POST	/api/payment-methods	Create New Payment Method. <i>Roles:</i> Admin.
PUT	/api/payment-methods/{id}	Update Payment Method. <i>Roles:</i> Admin.
DELETE	/api/payment-methods/{id}	Delete Payment Method. <i>Roles:</i> Admin.
12. VNPay Payment		

POST	/api/payment/vnpay/create	Create VNPay Payment URL. <i>Roles:</i> Private (Token required) <i>Body:</i> CreatePaymentRequest containing OrderId. <i>Response:</i> Returns a VNPay payment URL string for the selected order.
GET	/api/payment/vnpay/ipn	Handle VNPay IPN Callback. <i>Roles:</i> Public callback from VNPay. <i>Behavior:</i> Validates the VNPay signature, updates the order to Processing when payment succeeds, or Cancelled when payment fails. <i>Response:</i> Returns RspCode and confirmation message.
13. Vouchers		
GET	/api/vouchers	Retrieve All Vouchers. <i>Roles:</i> Public <i>Response:</i> Returns a list array.
GET	/api/vouchers/{id}	Retrieve Voucher Details by ID. <i>Roles:</i> Public.
POST	/api/vouchers/validate	Validate Voucher Eligibility. <i>Roles:</i> Public <i>Body:</i> { code, orderAmount }.
POST	/api/vouchers	Create New Voucher. <i>Roles:</i> Admin.
PATCH	/api/vouchers/{id}	Partially Update Voucher. <i>Roles:</i> Admin.
DELETE	/api/vouchers/{id}	Delete Voucher. <i>Roles:</i> Admin.
14. Product Reviews		
GET	/api/products/{productId}/reviews	Retrieve Product Reviews. <i>Roles:</i> Public.
GET	/api/products/{productId}/can-review	Check User's Review Eligibility. <i>Roles:</i> Customer <i>Response:</i> Returns { canReview: true/false }.
POST	/api/products/{productId}/reviews	Create New Product Review. <i>Roles:</i> Customer; must have purchased the product.

PUT	/api/products/ {productId}/reviews/ {reviewId}	Update Review Content. <i>Roles:</i> Customer; must be the owner of the review.
DELETE	/api/products/ {productId}/reviews/ {reviewId}	Delete Product Review. <i>Roles:</i> Customer (Owner) or Admin.
15. Customer Support		
GET	/api/supports	Retrieve Personal Support Tickets. <i>Roles:</i> Customer.
GET	/api/supports/all	Retrieve All Support Tickets. <i>Roles:</i> Staff.
POST	/api/supports	Create New Support Ticket. <i>Roles:</i> Customer.
PATCH	/api/supports/{id}/ status	Update Support Ticket Status. <i>Roles:</i> Staff <i>Enums:</i> Pending, InProgress, Resolved.
16. Notifications		
GET	/api/notifications	Retrieve User Notifications. <i>Roles:</i> Private (Token required) <i>Response:</i> Returns all notifications belonging to the authenticated user.
GET	/api/notifications/ unread-count	Retrieve Unread Notification Count. <i>Roles:</i> Private (Token required) <i>Response:</i> Returns { count } for the authenticated user.
PATCH	/api/notifications/ {id}/read	Mark Notification as Read. <i>Roles:</i> Private (Token required) <i>Path Parameter:</i> id identifies the notification record.
PATCH	/api/notifications/ read-all	Mark All Notifications as Read. <i>Roles:</i> Private (Token required) <i>Response:</i> Marks every notification of the authenticated user as read.
17. Staff Management		
GET	/api/staffs	Retrieve Staff List. <i>Roles:</i> Admin <i>Response:</i> Returns a list array.
GET	/api/staffs/{id}	Retrieve Staff Details by ID. <i>Roles:</i> Admin.

POST	/api/staffs	Create New Staff Account. <i>Roles:</i> Admin.
PUT	/api/staffs/{id}	Update Staff Information. <i>Roles:</i> Admin.
DELETE	/api/staffs/{id}	Delete Staff Account. <i>Roles:</i> Admin.
18. Reports & Statistics		
GET	/api/reports/revenue	Revenue Report. <i>Roles:</i> Admin <i>Query:</i> ?startDate={ISO}&endDate={ISO} <i>Response:</i> Returns totalRevenue and totalOrders.
GET	/api/reports/orders	Order Statistics by Status. <i>Roles:</i> Admin <i>Response:</i> Returns list data: status, count, totalRevenue, averageOrderValue.
GET	/api/reports/customers	Top Customers Report. <i>Roles:</i> Admin <i>Note:</i> Lists customers with the highest total spending.

Table A.1: API Documentation